



**UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE
CAMPUS AVANÇADO DE NATAL
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

MARCOS ANDRÉ AZEVEDO DE ASSIS

**UM SISTEMA PARA GERENCIAMENTO DE REMANEJAMENTOS EM PROJETOS
DE FUNDAÇÕES DE APOIO INSTITUCIONAL**

**NATAL
2025**

MARCOS ANDRÉ AZEVEDO DE ASSIS

UM SISTEMA PARA GERENCIAMENTO DE REMANEJAMENTOS EM PROJETOS
DE FUNDAÇÕES DE APOIO INSTITUCIONAL

Monografia apresentada à Universidade do
Estado do Rio Grande do Norte - UERN -
como requisito obrigatório para obtenção
do título de Bacharel em Ciência da Com-
putação.

Orientador: Prof. Me. André Gustavo Pe-
reira da Silva

NATAL
2025

MARCOS ANDRÉ AZEVEDO DE ASSIS

UM SISTEMA PARA GERENCIAMENTO DE REMANEJAMENTOS EM PROJETOS
DE FUNDAÇÕES DE APOIO INSTITUCIONAL

Monografia apresentada à Universidade do
Estado do Rio Grande do Norte - UERN -
como requisito obrigatório para obtenção
do título de Bacharel em Ciência da Com-
putação.

Aprovado em ____/____/____.

Banca Examinadora

Prof. Me. André Gustavo Pereira da Silva (Orientador)
Universidade do Estado do Rio Grande do Norte - UERN

Prof. Dr. Wilfredo Blanco Figuerola
Universidade do Estado do Rio Grande do Norte - UERN

Prof^a. Dra. Bartira Paraguaçu Falcão Dantas Rocha
Universidade do Estado do Rio Grande do Norte - UERN

Dedico este trabalho primeiramente a Deus, aos meus pais, irmão, avós, tios, primos e amigos que sempre estiveram ao meu lado em cada etapa da minha jornada. Vocês são os pilares da minha formação e, de maneiras únicas, contribuíram para tornar este sonho uma realidade. Sem o apoio e a presença de cada um de vocês, nada disso seria possível. Agradeço por serem a base e a força que realmente importam em minha vida.

AGRADECIMENTOS

Gostaria de dedicar um sincero agradecimento a todos que contribuíram para esta importante etapa da minha vida acadêmica. Embora estas palavras não possam mencionar cada um individualmente, saibam que todos estão igualmente presentes em meu coração e em minha gratidão.

Primeiramente, agradeço a Deus por me guiar ao longo do curso, proporcionando-me saúde e forças para superar os desafios e concluir esta jornada.

À minha família, expresso meu mais profundo reconhecimento. Sem o apoio incondicional e o encorajamento constante de vocês, alcançar este objetivo teria sido extremamente difícil.

Meu agradecimento especial ao meu orientador, Prof. Me. André Gustavo Pereira da Silva, cuja paciência, sabedoria e orientação foram fundamentais para o desenvolvimento deste trabalho.

Aos professores do curso de Ciência da Computação da Universidade do Estado do Rio Grande do Norte, sou grato pela excelência técnica e pelo empenho em compartilhar conhecimento, que enriqueceram minha formação acadêmica.

Aos colegas de graduação, que estiveram ao meu lado em inúmeros desafios, agradeço pela colaboração e pelo espírito de equipe que tornaram essa jornada mais significativa.

Agradeço também à Secretaria do Curso pela cooperação e pela atenção dedicada ao longo de todo o processo.

Aos meus colegas de trabalho do setor de Assessoria de Tecnologia da Informação (AsTI), que sempre me incentivaram a buscar constante aprimoramento e qualificação.

Por fim, minha profunda gratidão a todos que fizeram parte desta jornada. Continuarei aplicando e aprimorando os conhecimentos adquiridos para minha trajetória futura.

“Seja você quem for, seja qual for a posição social que você tenha na vida, a mais alta ou a mais baixa, tenha sempre como meta muita força, muita determinação e sempre faça tudo com muito amor e com muita fé em Deus, que um dia você chega lá. De alguma maneira você chega lá.”
(Ayrton Senna)

RESUMO

As fundações de apoio institucional têm como missão promover o conhecimento científico e tecnológico e suas inovações, gerenciando projetos de pesquisa, ensino, extensão e cultura. Dentro desse contexto, é crucial que essas instituições estejam preparadas para lidar com diversos cenários desafiadores durante a execução de projetos, incluindo o remanejamento de recursos. O remanejamento envolve a realocação de recursos, equipes e outros elementos do projeto, e frequentemente é uma tarefa complexa devido à falta de ferramentas computacionais adequadas. A dependência de planilhas eletrônicas para armazenamento, consulta e geração de relatórios pode levar a inconsistências nos dados, dado que essas planilhas são suscetíveis a erros. Diante dessa necessidade, buscamos ferramentas especializadas para o processo de remanejamento, mas sem sucesso. O presente trabalho visa preencher essa lacuna com o desenvolvimento de um sistema que abrange o cadastro de projetos, acompanhamento, remanejamentos e geração de relatórios. Utilizamos tecnologias como PHP, JavaScript, o *framework* Laravel, a arquitetura MVC e o banco de dados PostgreSQL. Para assegurar um desenvolvimento ágil e eficiente, adotamos a metodologia Scrum. As análises foram baseadas nas demandas específicas de remanejamento em projetos geridos por uma fundação de apoio institucional. Esperamos que este sistema ofereça uma solução eficaz para os desafios encontrados nesta fase da gestão de projetos, simplificando e otimizando o processo de remanejamento e, conseqüentemente, aumentando a eficiência.

Palavras-chave: Sistema; Remanejamentos; Gestão de Projetos; Desenvolvimento; Planilhas;

ABSTRACT

The mission of Institutional Support Foundations is to promote scientific and technological knowledge and innovations, managing research, teaching, extension and cultural projects. Within this context, it is crucial that these institutions are prepared to deal with several challenging scenarios during the execution of projects, including the reallocation of resources. Reallocation involves the reallocation of resources, teams and other elements of the project, and is often a complex task due to the lack of adequate computational tools. The dependence on spreadsheets for storage, consultation and generation of reports can lead to inconsistencies in the data, since these spreadsheets are susceptible to errors. Given this need, we sought specialized tools for the reallocation process, but without success. This work aims to fill this gap by developing a system that covers project registration, monitoring, reallocations and generation of reports. We use technologies such as PHP, JavaScript, the Laravel framework, the MVC architecture and the PostgreSQL database. To ensure agile and efficient development, we adopted the Scrum methodology. The analyses were based on specific reallocation demands in projects managed by an Institutional Support Foundation. We hope that this system will offer an effective solution to the challenges encountered at this stage of project management, simplifying and optimizing the reallocation process and, consequently, increasing efficiency.

Keywords: System; Reallocations; Project management; Development; Spreadsheets;

LISTA DE ILUSTRAÇÕES

Figura 1 – Ciclo do Scrum	22
Figura 2 – Arquitetura MVC.	24
Figura 3 – Ilustração do <i>back-end</i> e <i>front-end</i> do sistema.	25
Figura 4 – Estatísticas de uso de linguagens de programação do lado do servidor.	27
Figura 5 – Fluxo de requisições da arquitetura MVC com Laravel.	28
Figura 6 – Estatísticas de pesquisa sobre <i>frameworks</i> PHP.	28
Figura 7 – Estatísticas da pesquisa realizada pelo Stack Overflow em 2024.	30
Figura 8 – Estatísticas de uso de servidores web.	31
Figura 9 – Estatísticas de uso do distribuições Linux na web.	32
Figura 10 – Fluxograma do cadastro de projeto	48
Figura 11 – Fluxograma do cadastro de ofício de remanejamento	50
Figura 12 – Protótipo de tela de login	52
Figura 13 – Protótipo de tela de cadastro de usuário	53
Figura 14 – Protótipo de tela de cadastro de projeto	54
Figura 15 – Protótipo de tela de adição de rubrica e itens	56
Figura 16 – Protótipo de tela de cadastro de ofício	57
Figura 17 – Protótipo de tela de remanejamento de itens	59
Figura 18 – Protótipo de tela de conclusão do cadastro de ofício	60
Figura 19 – Diagrama de classe do sistema.	63
Figura 20 – Tela de Autenticação.	64
Figura 21 – Código para validação de valores de autenticação.	65
Figura 22 – Código para autenticação de usuário.	66
Figura 23 – Classe de Usuário.	66
Figura 24 – Tela de cadastro de usuários.	67
Figura 25 – Código para cadastro de usuário.	68
Figura 26 – Tela de listagem de usuários.	69
Figura 27 – Código para deletar usuário.	70
Figura 28 – Classe de Rubrica	71
Figura 29 – Tela de cadastro de rubricas.	72
Figura 30 – Código <i>front-end</i> do formulário de cadastro de rubrica.	73
Figura 31 – Tela de listagem de rubricas.	74
Figura 32 – Código para deletar rubrica.	74
Figura 33 – Classe de Projeto.	75
Figura 34 – Tela de cadastro de projeto.	76
Figura 35 – Tela de cadastro de projeto - Funcionalidade de salvamento parcial.	77
Figura 36 – Tela principal do sistema - Listagem de projetos.	78

Figura 37 – Código para deletar projeto.	80
Figura 38 – Classe de ofício.	81
Figura 39 – Tela de cadastro de ofício.	82
Figura 40 – Tela de cadastro de ofício - Plano de aplicação atual do projeto. . .	83
Figura 41 – Classe de aditivo de vigência.	84
Figura 42 – Tela de cadastro de ofício - Aditivo de vigência.	85
Figura 43 – Tela de cadastro de ofício - Aditivo de alteração de item.	86
Figura 44 – Tela de cadastro de ofício - Aditivo de alteração de item.	87
Figura 45 – Classe de aditivo de orçamento.	87
Figura 46 – Tela de cadastro de ofício - Aditivo de orçamento.	88
Figura 47 – Classe de aditivo de uso de rendimento.	89
Figura 48 – Tela de cadastro de ofício - Uso de rendimento.	90
Figura 49 – Classe de ofício de remanejamento de item.	91
Figura 50 – Tela de cadastro de ofício - Remanejamento de item.	92
Figura 51 – Classe de ofício de remanejamento de rendimento.	93
Figura 52 – Tela de cadastro de ofício - Remanejamento de rendimento.	93
Figura 53 – Tela de listagem de ofícios.	94
Figura 54 – Tela de visualização de ofício.	95
Figura 55 – Tela de relatório de ofícios.	97

LISTA DE TABELAS

Tabela 1 – Cronograma de Sprints 43

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Motivação	17
1.2	Justificativa	18
1.3	Objetivo Geral	18
1.4	Objetivos Específicos	18
1.5	Organização do Trabalho	19
2	FUNDAMENTAÇÃO TEÓRICA	20
2.1	Projetos	20
2.1.1	Gerenciamento de projetos	20
2.1.2	Gerenciamento de projetos em fundações de apoio institucional	20
2.2	Motodologia Ágil Scrum	21
2.3	Paradigma Orientado a Objetos	23
2.4	Arquitetura MVC	23
2.5	<i>Back-end</i>	25
2.5.1	Linguagem PHP	26
2.5.2	<i>Framework</i> Laravel	27
2.5.3	Linguagem SQL	29
2.5.4	Banco de Dados PostgreSQL	29
2.5.5	Apache HTTP Server	30
2.5.6	Ubuntu Server	31
2.6	<i>Front-end</i>	33
2.6.1	Linguagem HTML	33
2.6.2	Linguagem CSS	34
2.6.3	Linguagem JavaScript	35
2.6.4	<i>Framework</i> Bootstrap	35
2.6.5	Biblioteca JQuery	35
2.7	Git	36

2.8	Considerações finais do capítulo	36
3	TRABALHOS RELACIONADOS	38
3.1	<i>Identificação e análise de variáveis de desempenho e sucesso que mais influenciam no gerenciamento de projetos nas fundações de apoio</i>	38
3.2	<i>Framework Laravel: Um Estudo de Caso Full Stack Development</i>	39
3.3	<i>Desenvolvimento de uma ferramenta de gestão para construção civil utilizando o Framework Laravel MVC</i>	40
3.4	Considerações finais do capítulo	40
4	METODOLOGIA	42
4.1	Tipo de pesquisa	42
4.2	Metodologia ágil Scrum	42
4.3	<i>Sprint 1 - Levantamento de requisitos do sistema</i>	44
4.3.1	Requisitos funcionais	44
4.3.2	Requisitos não funcionais	46
4.4	<i>Sprint 2 – Elaboração de fluxogramas do sistema</i>	47
4.4.1	Fluxograma do cadastro de projeto	47
4.4.2	Fluxograma do cadastro de ofício de remanejamento	49
4.5	<i>Sprint 3 - Elaboração de protótipos de tela.</i>	51
4.5.1	Protótipo de tela de login	51
4.5.2	Protótipo de tela de cadastro de usuário	52
4.5.3	Protótipo de tela de cadastro de projeto	53
4.5.4	Protótipo de tela de adição de rubrica e itens	55
4.5.5	Protótipos de tela de cadastro de ofício	56
4.5.5.1	Protótipo de tela de cadastro de tipo de ofício	57
4.5.5.2	Protótipo de tela de ofício de remanejamento de itens	58
4.5.5.3	Protótipo de tela de conclusão do cadastro de ofício	59
4.6	<i>Sprint 4 - Definição de ferramentas e início de desenvolvimento</i>	60
4.7	Considerações finais do capítulo	61
5	DESENVOLVIMENTO	62

5.1	<i>Sprint 5 - Elaboração de diagrama de classes do sistema</i>	62
5.2	<i>Sprint 6 - Implementação de autenticação e cadastro de usuários</i>	64
5.2.1	Autenticação de usuários	64
5.2.2	Modelagem do diagrama de classe de Usuário	66
5.2.3	Cadastro de usuários	67
5.2.4	Listagem e gerenciamento de usuários	69
5.3	<i>Sprint 7 - Implementação de módulos de rubricas e projetos.</i>	70
5.3.1	Modelagem do diagrama de classe de Rubrica	71
5.3.2	Cadastro de rubricas	71
5.3.3	Listagem e gerenciamento de rubricas	73
5.3.4	Modelagem do diagrama de classe de Projeto	75
5.3.5	Cadastro de projeto	76
5.3.6	Cadastro de projeto – Funcionalidade de salvamento parcial	77
5.3.7	Listagem e gerenciamento de projetos	78
5.4	<i>Sprint 8 - Implementação do módulo de cadastro de ofícios e relatórios.</i>	80
5.4.1	Modelagem do diagrama de ofícios	80
5.4.2	Cadastro de ofício	81
5.4.3	Cadastro de ofício - Plano de aplicação atual do projeto	83
5.4.4	Modelagem do diagrama de Aditivo de vigência	84
5.4.5	Cadastro de ofício - Aditivo de vigência	84
5.4.6	Modelagem do diagrama de Aditivo de alteração de item	85
5.4.7	Cadastro de ofício - Aditivo de alteração de item	86
5.4.8	Modelagem do diagrama de Aditivo de orçamento	87
5.4.9	Cadastro de ofício - Aditivo de orçamento	88
5.4.10	Modelagem do diagrama de Aditivo de uso de rendimento	89
5.4.11	Cadastro de ofício - Aditivo de uso de rendimento	90
5.4.12	Modelagem do diagrama de aditivo de remanejamento de item	90
5.4.13	Cadastro de ofício - Aditivo de remanejamento de item	91
5.4.14	Modelagem do diagrama de Aditivo de remanejamento de rendimento	92
5.4.15	Cadastro de ofício - Remanejamento de rendimento	93
5.4.16	Listagem de ofícios	94
5.4.17	Visualizar ofício	95
5.4.18	Relatório de ofícios	96
5.5	<i>Considerações finais do capítulo</i>	98
6	<i>CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS</i>	99

REFERÊNCIAS 101

1 INTRODUÇÃO

As fundações de apoio institucional são entidades cuja missão é promover o conhecimento científico, tecnológico e suas inovações por meio da gestão de projetos. Atuando em parceria com instituições públicas e privadas, as fundações apoiam pesquisadores ao fornecer assessoria administrativa e financeira para a execução de projetos de ensino, pesquisa, extensão, desenvolvimento institucional, desenvolvimento científico e tecnológico, e fomento à inovação, contribuindo assim para o avanço da ciência e tecnologia no contexto geral (CULTURA, 2019).

A partir das negociações que resultam na formalização do instrumento jurídico (convênio, contrato, termo de parceria, acordo de cooperação, dentre outros), todo o desenvolvimento do projeto é acompanhado, desde o cadastramento até prestação de contas.

Como observado, essas instituições abrangem uma ampla gama de áreas e, como consequência, enfrentam uma necessidade constante na gestão de projetos: a adaptação e a realocação de recursos, equipes e outros elementos necessários ao longo da execução desses projetos. Essa prática comumente chamada de remanejamento é essencial para garantir a eficácia na utilização dos recursos.

Um exemplo clássico de remanejamento é a transferência de valor entre rubricas, que envolve a realocação de recursos de um item orçamentário para outro, dentro do plano de aplicação do projeto (PROPLAN, 2025).

Uma rubrica pode ser entendida como uma verba prevista como despesa em orçamentos públicos e destinada a fins específicos. Sua existência é obrigatória para que haja pagamento de qualquer despesa pública (SENADO, 2025).

Os remanejamentos são fundamentais para garantir que os projetos alcancem seus objetivos de forma eficiente. No entanto, na ausência de ferramentas computacionais adequadas, esse processo se torna manual e está sujeito a erros, devido à grande quantidade de informações envolvidas. Diante desse desafio, realizamos consultas a diversas fundações, como a Fundação Norte-Rio-Grandense de Pesquisa e Cultura (FUNPEC), a Fundação de Apoio a Serviços Técnicos, Ensino e Fomento a Pesquisas (ASTEF), a Fundação de Apoio ao Desenvolvimento da Universidade Federal de Pernambuco (FADE), entre outras, para identificar aquelas que solucionaram esse problema. Constatamos que essas instituições ainda dependem de planilhas eletrônicas para armazenar, consultar e gerar relatórios relacionados aos remanejamentos em projetos.

Um estudo conduzido por Ray Panko (PANKO, 2005), professor de gerenciamento de tecnologia da informação da Universidade do Havaí, revelou que cerca de 88% das planilhas corporativas apresentam erros. “As planilhas, mesmo após

um planejamento cuidadoso, contêm erros em 1% ou mais de todas as células com fórmulas. Em grandes planilhas com milhares de fórmulas, haverá dezenas de erros não detectados. Apesar dessa evidência de longa data, a maioria das corporações tem prestado pouca atenção à possibilidade de erros graves em planilhas.”. Essa constatação corrobora os dados de um levantamento da empresa americana de softwares de Recursos Humanos, BambooHR, que aponta que 90% das planilhas de organizações contêm erros significativos (CONVENIA, 2023).

Neste contexto, a hipótese deste trabalho é que a implementação de uma solução em sistema de computador possa aumentar a eficiência e a precisão na alocação de recursos em projetos geridos por fundações de apoio institucional. Uma solução desenvolvida para proporcionar a centralização de dados e a automatização de tarefas, como será discutido adiante, pode superar as limitações do uso de planilhas eletrônicas, proporcionando uma gestão mais eficaz e transparente dos projetos.

Quando nos referimos à centralização de dados no parágrafo anterior, nos referimos à consolidação de todas as informações em um único sistema de banco de dados, acessível por meio de uma interface — o *front-end*, ou, didaticamente falando, as telas do sistema. Essa centralização traz vantagens, como, por exemplo:

- **Unificação dos dados:** Em vez de consultar várias planilhas ou documentos dispersos, todos os dados sobre os projetos, suas rubricas e remanejamentos ficam disponíveis em um único sistema. Isso evita a duplicidade de dados e facilita a consulta em tempo real do plano orçamentário de cada projeto.
- **Camada básica de segurança:** Ao centralizar os dados em um sistema, é possível implementar práticas de segurança, como a autenticação de usuários, que garantem que apenas pessoas autorizadas possam realizar alterações. Pesquisas indicam que 73% das planilhas utilizadas pelas empresas não possuem o mínimo de proteção, sendo arquivos abertos (SIGO, 2023).

Já quando nos referimos à automatização de tarefas, referimo-nos à automatização de uma série de atividades rotineiras que, por possuírem um padrão bem definido, tornam-se ideais para serem automatizadas. Algumas dessas tarefas incluem:

- **Validação de valores de remanejamentos:** Algumas regras de remanejamento podem ser verificadas automaticamente, como limites de transferência de valores que cada rubrica disponibiliza e que podem ser realocados de uma rubrica para outra, garantindo conformidade dos valores do plano orçamentário do projeto.
- **Cálculo de valores de remanejamentos:** O sistema pode calcular automaticamente a redistribuição de recursos entre rubricas após a consolidação do ato de remanejamento entre rubricas no projeto, evitando erros humanos comuns em cálculos manuais.

- **Geração de relatórios:** O sistema pode gerar relatórios do plano orçamentário do projeto e dos remanejamentos realizados durante sua vigência, economizando tempo na compilação manual de dados em planilhas e, principalmente, agilizando a prestação de contas.

Para executar essa hipótese, desenvolvemos um sistema utilizando tecnologias robustas e bem consolidadas no campo do desenvolvimento de sistemas, como PHP, JavaScript, SQL, o *framework* Laravel, o banco de dados PostgreSQL, o servidor Apache e os sistemas operacionais Linux (Ubuntu e Ubuntu Server). A metodologia Scrum foi empregada para termos um desenvolvimento ágil e alinhado com as expectativas do cliente. A aplicabilidade da solução foi experimentada com base nas demandas de remanejamentos em uma fundação de apoio institucional, que, por motivos patrimoniais, solicitou-nos para não ser mencionada.

O impacto que buscamos gerar com este trabalho é aprimorar os processos de remanejamento de recursos em projetos geridos por fundações de apoio institucional, oferecendo um sistema computacional que aumente a precisão dos dados, reduza a carga de trabalho relacionada à manipulação de planilhas eletrônicas e minimize os erros associados a estes processos.

Embora todas as fundações de apoio institucional sigam o mesmo padrão de gestão de projetos, controlando o orçamento por meio de rubricas e seus respectivos itens, o trabalho aqui desenvolvido ainda enfrenta uma limitação que pode restringir sua total aplicabilidade: tivemos a oportunidade de avaliar o uso do sistema em apenas uma instituição, o que pode não refletir plenamente as demandas e necessidades de outras instituições semelhantes.

1.1 Motivação

A prática de remanejamento de recursos é fundamental para assegurar a eficiência na gestão de recursos de projetos ao longo de sua execução. No entanto, a dependência de planilhas eletrônicas para o armazenamento, acompanhamento e geração de relatórios apresenta riscos significativos, conforme apontado pelos colaboradores da instituição analisada:

1. **Tempestividade da informação:** O preenchimento das planilhas demanda um tempo considerável, além da necessidade de conciliar as informações com o plano orçamentário do projeto. Pesquisas mostram que funcionários gastam, em média, 12 horas por mês consolidando e verificando planilhas compartilhadas. Mesmo com todo esse tempo investido, os erros ainda podem acontecer, gerando prejuízos para empresas (MINDTEK, 2021).

2. **Risco de perda de dados:** Os arquivos podem ser acidentalmente excluídos, danificados ou corrompidos, resultando na perda de todas as informações referentes aos remanejamentos já implementados.
3. **Erros nos valores informados:** Pequenos deslizes, como erros de digitação ou seleção incorreta de células, podem fazer com que as informações registradas não estejam em conformidade com o orçamento real do projeto.

Essas limitações evidenciam a necessidade de uma ferramenta computacional mais adequada, o que motivou o desenvolvimento deste trabalho.

1.2 Justificativa

Este trabalho baseia-se na necessidade de aprimorar o processo de remanejamento de recursos em projetos geridos por fundações de apoio institucional. As falhas recorrentes no uso de ferramentas manuais, como planilhas eletrônicas, resultam em diversos problemas que afetam negativamente a integridade dos dados.

Dessa forma, o desenvolvimento deste sistema se justifica pela busca de maior eficiência e segurança na administração dos recursos, ao mesmo tempo em que automatiza a execução das tarefas e melhora a precisão das informações. O sistema proposto visa não apenas corrigir as deficiências identificadas, mas também manter-se alinhado às demandas de eficiência e prestação de contas.

1.3 Objetivo Geral

O objetivo geral deste trabalho é solucionar a ineficiência, a vulnerabilidade a erros e a falta de segurança no processo de remanejamento de recursos em projetos geridos por fundações de apoio institucional. Atualmente, essas instituições ainda dependem de planilhas eletrônicas para o controle orçamentário, o que torna o processo propenso a falhas. Para resolver essa questão, foi desenvolvido um sistema que abrange desde o cadastro dos projetos e o acompanhamento de sua execução até a geração de relatórios. O sistema proposto visa minimizar os riscos associados ao uso de planilhas e otimizar a gestão de projetos dentro dessas instituições.

1.4 Objetivos Específicos

Para alcançar o resultado descrito no objetivo geral deste trabalho, estabeleceu-se os seguintes objetivos específicos:

1. **Analisar demandas relacionadas a remanejamentos em projetos geridos por fundação de apoio institucional:** Esta análise foi realizada como parte do estudo

de caso, visando compreender as necessidades relacionadas ao remanejamento de recursos em projeto.

2. **Realizar o levantamento de requisitos:** Nessa fase, foram realizadas reuniões juntos aos colaboradores da instituição, com o objetivo de identificar as funcionalidades essenciais e entender os fluxos de trabalho relacionados ao processo de remanejamento de recursos.
3. **Identificar as tecnologias adequadas:** Após o levantamento de requisitos, iniciou-se a fase de escolha das tecnologias a serem empregadas. A seleção incluiu ferramentas de desenvolvimento ágil, linguagens de programação bem consolidadas, bancos de dados eficientes para lidar com o grande volume de informações, além de servidor web confiável e por último, sobre o que tudo iria rodar, um sistema operacional.
4. **Desenvolver o sistema computacional:** Nesta fase, iniciou-se o desenvolvimento do sistema, onde foram implementadas a arquitetura, a lógica de funcionamento, as regras de negócio e as telas do sistema.
5. **Disponibilizar as funcionalidades imprescindíveis do sistema:** Cadastro de projetos, cadastro de rubricas, cadastro de itens de rubricas, realização de remanejamento em projetos e geração de relatórios detalhados com informações sobre o orçamento do projeto.
6. **Realizar testes de homologação com os colaboradores internos:** Antes de cada nova funcionalidade, testes são feitos, envolvendo colaboradores da fundação de apoio. Essa etapa de homologação teve como objetivo validar a usabilidade das novas funcionalidades. Os retornos e avaliações dos colaboradores foram importantes para que os ajustes necessários fossem realizados.
7. **Realizar melhoria contínuas:** É fundamental para garantir que o produto evolua constantemente. Isso envolve a implementação de novas funcionalidades, melhoria na documentação e a correção de falhas que forem sendo constatadas.

1.5 Organização do Trabalho

Esta dissertação está estruturada da seguinte maneira: No Capítulo 2, são apresentados os fundamentos teóricos necessários para embasar este trabalho. O Capítulo 3 aborda trabalhos relacionados. No Capítulo 4, são discutidas as metodologias empregadas. O Capítulo 5 compreende a exposição do desenvolvimento, incluindo os conceitos utilizados, além das limitações e desafios enfrentados durante este processo. Por fim, no Capítulo 6 de Conclusão, são apresentadas as considerações finais para esta dissertação.

2 FUNDAMENTAÇÃO TEÓRICA

Esta seção tem como objetivo proporcionar uma compreensão mais aprofundada da base teórica deste trabalho, detalhando as principais áreas de estudo abordadas.

Para compreender a proposta apresentada, é fundamental explorar conceitos-chave, uma vez que se trata de um programa de computador classificado como sistema web, conforme definido pela Lei nº 9.609/1998 (BRASIL, 1998). Nesse contexto, serão discutidos os fundamentos que orientam o desenvolvimento desse tipo de sistema, como a metodologia Scrum, o paradigma da programação orientada a objetos, a arquitetura MVC, as camadas de *back-end* e *front-end*, além do uso do sistema de controle de versão Git. Também serão abordados temas menos recorrentes na área da Ciência da Computação, como por exemplo o princípio de gerenciamento de projetos.

2.1 Projetos

Um projeto é um esforço temporário realizado para criar um produto, serviço ou resultado exclusivo. Sua natureza temporária significa que ele possui um início e um término bem definidos, conhecidos como vigência. O término ocorre quando os objetivos do projeto são alcançados, quando ele é interrompido devido à impossibilidade de atingir seus objetivos, quando a necessidade do projeto desaparece ou quando chega a data estabelecida para seu fim. Além disso, um projeto pode ser encerrado a pedido do cliente, patrocinador ou financiador (PMI, 2013).

2.1.1 Gerenciamento de projetos

O gerenciamento de projetos é a aplicação sistemática e integrada de conhecimentos, competências, ferramentas e técnicas às atividades do projeto, com base em processos bem definidos, visando o alcance dos objetivos estabelecidos em termos de escopo, tempo, custo, qualidade e riscos (CRUZ, 2013).

2.1.2 Gerenciamento de projetos em fundações de apoio institucional

Os projetos geridos por fundações de apoio são iniciativas de ensino, pesquisa e desenvolvimento executadas por universidades públicas com apoio de fundações privadas, que gerenciam recursos de forma mais ágil e flexível, conforme prevê a Lei nº 8.958 (BRASIL, 1994).

2.2 Metodologia Ágil Scrum

Adotamos a metodologia Scrum no desenvolvimento do trabalho com o objetivo de realizar entregas rápidas e iterativas, promover a colaboração contínua entre as equipes e garantir flexibilidade na adaptação a mudanças de requisitos (SCRUM.ORG, 2025).

Criado no início dos anos 1990 por Jeff Sutherland e Ken Schwaber, o Scrum é uma metodologia ágil e leve, desenvolvido para ajudar indivíduos, equipes e organizações a gerar valor por meio de soluções adaptativas para problemas complexos. Os dois profissionais apresentaram o Scrum pela primeira vez na Conferência OOPSLA, em 1995 (SCHWABER; SUTHERLAND, 2020).

O Scrum adota um fluxo iterativo e incremental, o que significa que o trabalho é realizado em camadas. A cada ciclo, iniciamos uma nova iteração, focando em entregar uma parte relevante do todo. Após a entrega, validamos o que foi feito e, então, reiniciamos o processo, sempre em ciclos de dias ou semanas. Esse modelo é sustentado por reuniões diárias, ciclos semanais e uma estratégia mensal, tudo isso visando a construção de um negócio, produto ou serviço de sucesso (AUDY, 2015).

O Scrum é estruturado de maneira robusta, com um time composto por três papéis principais cada um com responsabilidades bem definidas:

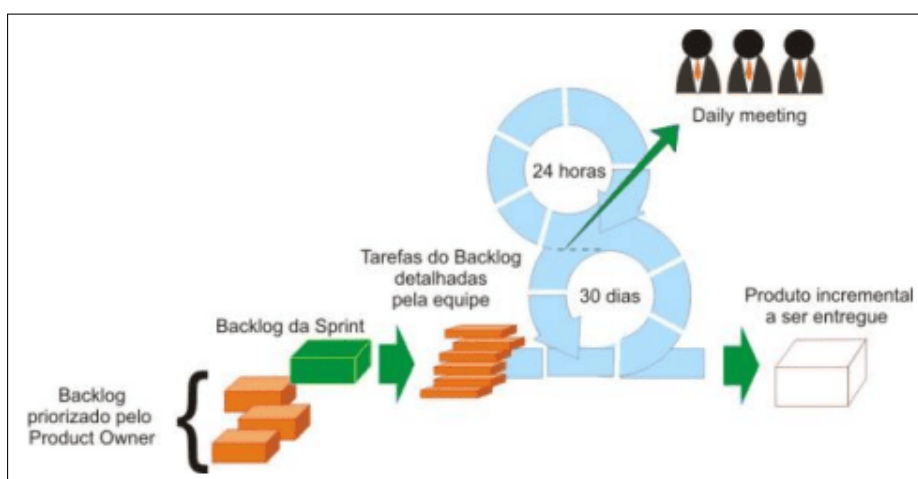
1. **Product Owner:** Papel de maior visibilidade e o guia do time. Ele é responsável por tomar as decisões estratégicas, definir as ações prioritárias, validar as soluções e garantir que a palavra final do negócio seja trazida para aprovação;
2. **Scrum Master:** Especialista no método e nas técnicas do Scrum, responsável por oferecer treinamentos e reciclagens, organizar os eventos, remover obstáculos e promover a harmonia do time dentro do seu ecossistema;
3. **Development Team:** Equipe de desenvolvimento composta por profissionais responsáveis por entregar versões utilizáveis do produto, que agrega valor e está “pronta” ao final de cada *Sprint*, contribuindo para o contínuo aprimoramento do projeto.

O ciclo do Scrum (Figura 1) segundo Pereira (2007) ocorre da seguinte forma:

1. **Backlog priorizado pelo Product Owner:** O *Product Owner* é responsável por criar e priorizar o *Product Backlog*, que é uma lista de todas as funcionalidades desejadas no produto.
2. **Backlog da Sprint:** Em reuniões de planejamento (*Sprint Planning*), o time seleciona os itens mais prioritários do *backlog* para compor o *Backlog da Sprint*, ou seja, o conjunto de tarefas que será desenvolvido durante a próxima *Sprint*.

3. **Tarefas detalhadas pela equipe:** A equipe de desenvolvimento detalha essas tarefas e as organiza para execução dentro da *Sprint*.
4. ***Sprint*:** Um ciclo de desenvolvimento com duração fixa — neste caso, 30 dias — durante o qual a equipe trabalha para transformar os itens do *Sprint Backlog* em um produto funcional.
5. ***Daily Meeting*** Durante a *Sprint*, a equipe realiza reuniões diárias de curta duração (normalmente 15 minutos), chamadas *Daily*, para sincronizar as atividades e planejar as próximas 24 horas.
6. **Produto incremental a ser entregue:** Ao final da *Sprint*, é entregue um incremento de produto, ou seja, uma versão funcional e potencialmente utilizável do produto.

Figura 1 – Ciclo do Scrum



Fonte: Imagem retirada de (PEREIRA; TORREÃO; MARÇAL, 2007)

Cada *Sprint* começa com uma reunião de planejamento, onde o time define e estima as tarefas junto ao *Product Owner*. Durante a execução, ocorrem reuniões diárias de até 15 minutos para acompanhar o progresso, monitorado por um gráfico chamado *Sprint Burndown*. Ao final da *Sprint*, o time realiza uma revisão do produto entregue (*Sprint Review*) e uma Retrospectiva para identificar melhorias no processo. O Scrum é ideal para projetos com mudanças frequentes nos requisitos, mas exige compreensão clara de seus papéis, responsabilidades e artefatos (PEREIRA; TORREÃO; MARÇAL, 2007).

O Scrum tem sido adotado com sucesso por organizações de diversos tamanhos e setores, desde grandes multinacionais até *startups*, e por empresas conhecidas ou emergentes. Originalmente desenvolvido para projetos de *software*, seu uso se expandiu e hoje é aplicado em diferentes mercados, como *marketing*, desenvolvimento

de *hardware*, entre outros. Ele é amplamente utilizado na produção de *softwares* comerciais, sites, *softwares* embarcados, aplicativos móveis, sistemas financeiros e jogos (AUDY, 2015).

2.3 Paradigma Orientado a Objetos

No desenvolvimento deste trabalho adotamos o paradigma da programação orientada a objetos, por oferecer uma abordagem estruturada e eficiente para a construção de sistemas. Esse paradigma favorece a modularidade, a reutilização de funcionalidades e a facilidade na manutenção e evolução do código-fonte do sistema (FARINELLI, 2007).

A orientação a objetos é um paradigma que representa uma filosofia para a construção de programas. Ao invés de criar um programa composto por um conjunto de procedimentos e variáveis, muitas vezes desconectadas do contexto, como era comum em linguagens estruturadas (como Cobol, Clipper, Pascal), a orientação a objetos adota uma abordagem mais alinhada ao mundo real. Trabalhamos com objetos, estruturas familiares do nosso cotidiano, sobre as quais temos uma compreensão mais intuitiva (DALL’OGLIO, 2018).

O conceito de programação orientada a objetos (POO) surgiram no final da década de 1960, quando a linguagem Simula-68 introduziu os princípios de objetos e troca de mensagens para a construção de programas. Esses conceitos foram posteriormente refinados e expandidos na década de 1970 pela linguagem de programação Smalltalk, desenvolvida no laboratório de pesquisa da Xerox, nos Estados Unidos. Um dos principais responsáveis por popularizar os conceitos de POO foi o cientista Alan Kay, da Xerox, um dos criadores do Smalltalk. Durante suas pesquisas, Kay propôs a ideia de que programas poderiam ser construídos utilizando conceitos e abstrações do mundo real, como objetos e troca de mensagens (O’REGAN, 2015).

2.4 Arquitetura MVC

A aplicação da arquitetura MVC (*Model-View-Controller*) em conjunto com os princípios da programação orientada a objetos proporciona ganhos significativos, especialmente na fase de manutenção do sistema. A separação clara entre as camadas — *model*, *view* e *controller* — aliada ao encapsulamento e à modularidade oferecidos pela POO, resulta em um sistema mais organizado e coeso. Essa estrutura facilita a localização e a correção de possíveis problemas, bem como a evolução do código ao longo de seu ciclo de vida (ROCHA, 2018).

A arquitetura MVC foi idealizada em 1979 pelo cientista da computação norueguês Trygve Reenskaug, então pesquisador na Xerox PARC. Sua proposta visava criar um padrão arquitetural que dividisse o sistema em camadas distintas, minimi-

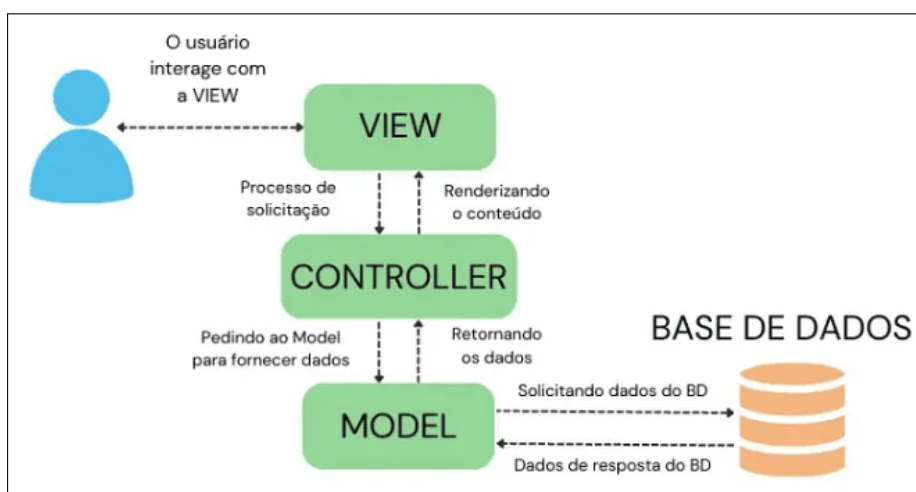
zando a dependência entre elas e facilitando a manutenção e evolução do código-fonte (REENSKAUG, 2006).

Na arquitetura MVC, o código-fonte do sistema é organizado em três camadas:

1. **Model** (Modelo): Representa as regras de negócio do sistema. Fazem parte dessa camada as classes responsáveis por realizar transações de negócio. Essa parte do código é responsável também por se comunicar com o banco de dados ou com outros sistemas existentes.
2. **View** (Visão): Camada responsável pela apresentação da resposta, ou seja, pela montagem das telas de resposta do sistema. Ele é ativado pelo *controller* e recebe informações da camada de *model*.
3. **Controller** (Controlador): Camada responsável por receber os comandos do usuário e acionar as ações corretas a serem realizadas no sistema, incluindo a seleção de qual componente de apresentação da resposta deverá ser utilizado.

Desta forma a arquitetura MVC segue o seguinte fluxo: O usuário interage com a *View*, que é responsável pela interface e apresentação. A *View* envia a solicitação ao *Controller*, que atua como intermediário, processando as ações e requisitando os dados ao *Model*. O *Model* acessa a base de dados, obtém as informações e as retorna ao *Controller*, que então envia os dados de volta para a *View*. Por fim, a *View* renderiza o conteúdo atualizado para o usuário (Figura 2).

Figura 2 – Arquitetura MVC.

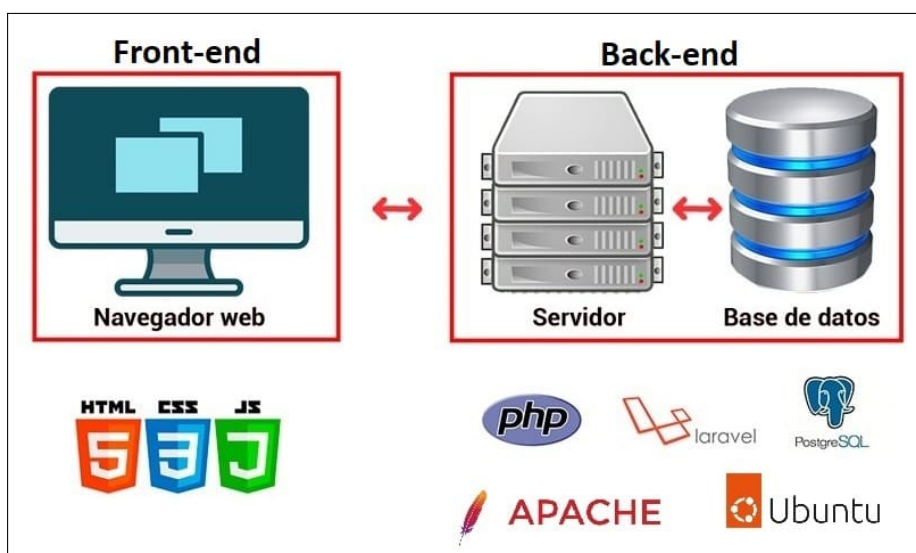


Fonte: Imagem retirada de (NORMANDO, 2021).

2.5 Back-end

Para assegurar uma melhor modularização do código-fonte e promover um controle mais eficaz sobre suas funcionalidades, decidiu-se pela separação do sistema em duas camadas principais: *back-end* e *front-end* (Figura 3).

Figura 3 – Ilustração do *back-end* e *front-end* do sistema.



Fonte: Imagem elaborada pelo autor.

O *back-end* corresponde ao lado servidor da aplicação, sendo responsável pelo armazenamento e gerenciamento de dados, além da implementação das regras de negócio. Trata-se de uma camada que opera de forma invisível ao usuário final, uma vez que os dados e funcionalidades por ela disponibilizados são consumidos e apresentados por meio do *front-end* da aplicação (JÚNIOR et al., 2022).

A camada de *back-end* do sistema desenvolvido é responsável por toda a lógica de funcionamento, além do armazenamento e manipulação de dados. De forma resumida, seu fluxo operacional funciona da seguinte maneira:

1. O servidor Apache recebe uma requisição HTTP proveniente do *front-end*.
2. O Apache repassa a requisição ao interpretador PHP.
3. O interpretador PHP, gerenciado pelo *framework* Laravel, executa a lógica da aplicação.
4. Quando necessário, o Laravel realiza consultas SQL ao banco de dados PostgreSQL para acessar ou modificar informações.
5. Os dados retornados do banco são processados pelo Laravel.
6. O Laravel gera uma resposta em HTML com base nos dados processados.

7. O Apache envia essa resposta HTML de volta ao *front-end*.
8. Todo esse processo é executado sobre um servidor Ubuntu, que hospeda os principais componentes do sistema.
9. O *back-end* garante que funcionalidades críticas — como autenticação, transações e atualizações de dados — ocorram de forma segura e eficiente.

A seguir, serão abordados os fundamentos das seguintes ferramentas utilizadas no *back-end* do sistema: a linguagem de programação PHP, o *framework* Laravel, a linguagem SQL, o banco de dados PostgreSQL, o servidor Apache HTTP Server e o ambiente Ubuntu Server.

2.5.1 Linguagem PHP

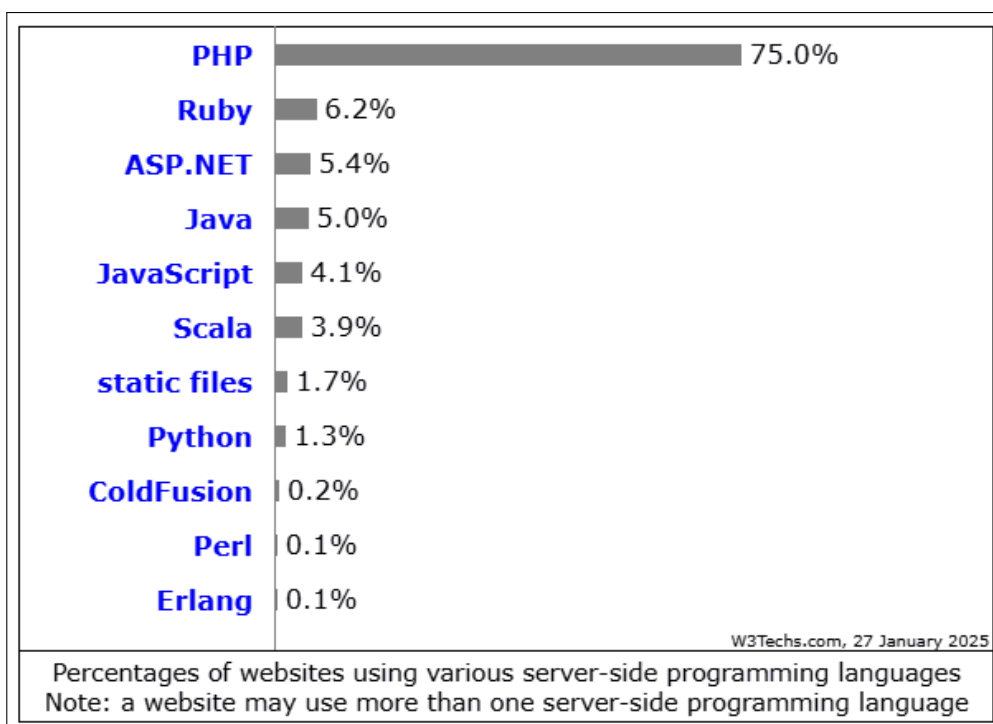
A escolha da linguagem de programação PHP para o desenvolvimento do sistema foi fundamentada em fatores como sua ampla adoção pela comunidade de desenvolvimento web, a facilidade de integração com bancos de dados e a disponibilidade de diversos *frameworks* e bibliotecas. Sendo uma linguagem de código aberto, o PHP é amplamente acessível e constantemente aprimorado pela comunidade. Sua compatibilidade com diferentes plataformas e servidores, como o Apache HTTP Server, torna-o uma opção interessante para a construção de sistemas web. Além disso, as frequentes atualizações focadas em desempenho e segurança reforçam sua relevância.

O PHP (um acrônimo recursivo de “PHP: *Hypertext Preprocessor*”) é uma linguagem de programação classificada como software livre e de código aberto para uso geral, disponível sob a licença PHP License, amplamente utilizada e especialmente adequada para o desenvolvimento de sistemas e páginas web (BAKKEN; SURASKI; SCHMID, 2000).

Criada em 1994 pelo programador canadense-dinamarquês Rasmus Lerdorf, o PHP evoluiu ao longo dos anos para se tornar uma das linguagens mais populares no desenvolvimento web. Recentemente, em 22 de novembro de 2021, a data ficou marcado como “A nova vida do PHP”, pois foi quando nasceu a The PHP Foundation, uma organização sem fins lucrativos cuja missão é garantir a longevidade e a prosperidade da linguagem PHP, coletando fundos para financiar desenvolvedores que contribuam para o avanço e a prosperidade da linguagem (THE PHP FOUNDATION, 2025).

Segundo levantamento conduzido pela W3Techs (2025), empresa especializada na análise de tecnologias empregadas na web, são realizadas coletas sistemáticas de dados a partir de informações publicamente disponíveis, como cabeçalhos HTTP e elementos acessíveis por meio de requisições aos servidores. A metodologia adotada permite mapear a distribuição de tecnologias de desenvolvimento utilizadas em milhões de sites. Com base nesses dados, foi identificado que a linguagem PHP está presente em aproximadamente 75% dos servidores web analisados (Figura 4).

Figura 4 – Estatísticas de uso de linguagens de programação do lado do servidor.



Fonte: Imagem retirada de (W3TECHS, 2025).

2.5.2 Framework Laravel

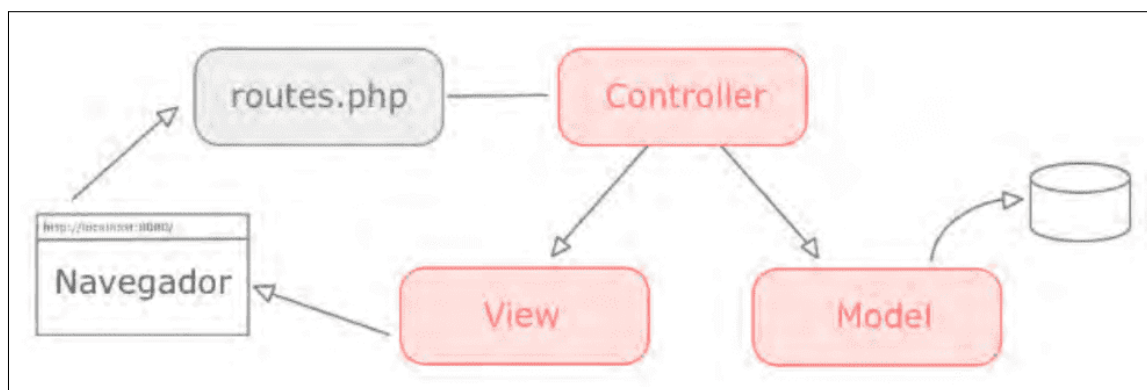
Com o objetivo de potencializar o desenvolvimento do sistema, optou-se pela adoção do *framework* Laravel, que oferece uma estrutura robusta e um amplo conjunto de funcionalidades integradas, como roteamento, autenticação, entre outras. Esses recursos contribuem diretamente para a eficiência do processo de desenvolvimento, possibilitando a redução do tempo necessário para a implementação de funcionalidades essenciais da aplicação.

O Laravel é um *framework* PHP de código aberto criado pelo programador Taylor B. Otwell como uma alternativa mais avançada ao *framework* CodeIgniter. Sua primeira versão foi lançada em 2011. Desde então, tem recebido atualizações frequentes e contribuições tanto do autor quanto da comunidade, consolidando-se como um verdadeiro ecossistema para o desenvolvimento de sistemas web. Com foco em uma sintaxe simples e concisa, seu lema é “O *framework* PHP dos artesãos da web” (GABARDO, 2017).

O Laravel adota a arquitetura MVC, que tem como principal objetivo separar as regras de negócio em três camadas distintas, cada uma com uma responsabilidade bem definida. Para entender melhor como o MVC funciona no Laravel, é importante observar o fluxo das requisições (Figura 5). Quando um usuário envia uma requisição pelo navegador, ela é inicialmente recebida pelo arquivo *routes.php*, que é responsável por direcionar a requisição ao *controller*. O *controller*, por sua vez, determina o que

deve ser feito com a requisição, interagindo com a camada *Model*, que lida com o acesso ao banco de dados e a execução das regras de negócio. Em seguida, o *controller* encaminha a resposta para a *View*, que é responsável por apresentar o conteúdo no navegador do usuário (TURINI, 2015).

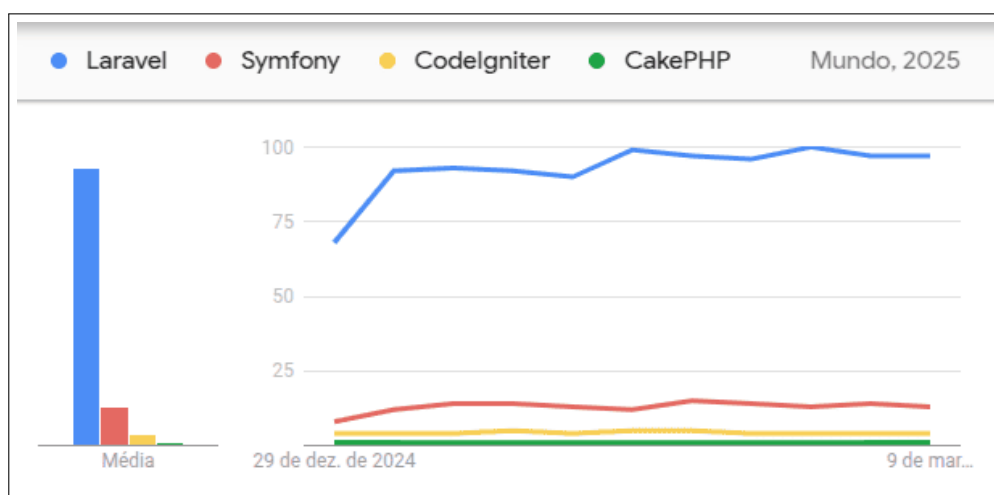
Figura 5 – Fluxo de requisições da arquitetura MVC com Laravel.



Fonte: Imagem retirada de (TURINI, 2015)

Realizamos uma pesquisa rápida e simples utilizando a ferramenta gratuita Google Trends, que permite analisar a popularidade de termos de pesquisa ao longo do tempo no mecanismo de busca do Google. A ferramenta apresenta gráficos e dados detalhados sobre como as pessoas estão pesquisando diferentes tópicos. Com base nisso, selecionamos quatro *frameworks* que consideramos os mais relevantes no ecossistema PHP. A partir dos resultados obtidos, concluímos que o Laravel se destaca entre eles, com uma vantagem considerável, seguido por Symfony, CodeIgniter e CakePHP, respectivamente (Figura 6).

Figura 6 – Estatísticas de pesquisa sobre *frameworks* PHP.



Fonte: Imagem retirada de (TRENDS, 2025)

Atualmente o Laravel está plenamente consolidado no mercado de desenvolvimento web, com mais de 80 mil estrelas em seu repositório público no GitHub, mais de

420 milhões de downloads e mais de 650 programadores contribuindo para seu avanço; posicionando-o como um dos *frameworks* mais populares para o desenvolvimento web (LARAVEL, 2025).

2.5.3 Linguagem SQL

Utilizou-se a linguagem SQL como componente essencial no desenvolvimento do sistema, desempenhando papel fundamental no gerenciamento e na manipulação dos dados armazenados no banco de dados. Por meio de suas instruções, é possível realizar operações como criação, consulta, atualização e exclusão de registros de forma estruturada e eficiente. Dessa forma, o SQL viabiliza a comunicação entre a aplicação e sua base de dados, assegurando a integridade, o armazenamento adequado e a recuperação das informações necessárias ao correto funcionamento do sistema.

A linguagem SQL foi desenvolvida no início dos anos 1970 nos laboratórios da IBM em San Jose, como parte do projeto System R. Inicialmente, a linguagem foi chamada SEQUEL, um acrônimo para *Structured English Query Language* (Linguagem de Consulta Estruturada em Inglês). Embora a IBM tenha sido a responsável pela criação do SQL, a linguagem rapidamente gerou diversas outras linguagens desenvolvidas por outras empresas e programadores. Essa proliferação levou à necessidade de estabelecer um padrão para a linguagem, o que foi feito pela American National Standards Institute (ANSI) em 1986 e pela International Organization for Standardization (ISO) em 1987. Desde então, o SQL passou por três revisões, culminando na versão SQL:2003 (BEIGHLEY, 2009).

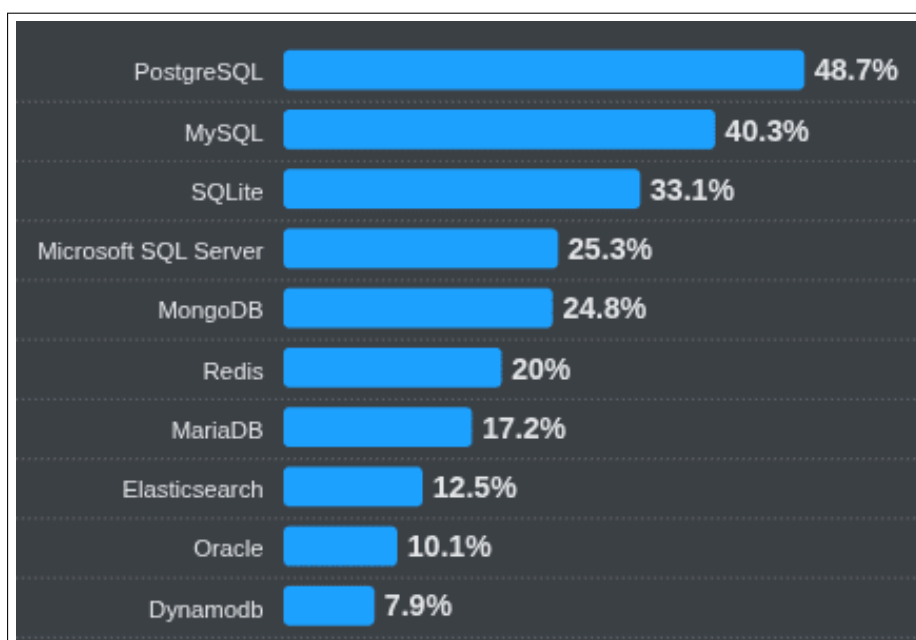
2.5.4 Banco de Dados PostgreSQL

Para garantir a persistência dos dados optou-se pelo uso do sistema de banco de dados PostgreSQL. Essa escolha se deve à sua sólida consolidação no mercado de desenvolvimento de programas, sendo amplamente reconhecido pela sua robustez, confiabilidade e conformidade com os padrões da linguagem SQL.

O PostgreSQL é um sistema de gerenciamento de banco de dados objeto-relacional de código aberto, robusto e altamente eficiente, que estende a linguagem SQL e oferece uma ampla variedade de recursos para armazenar e escalar com segurança até as cargas de trabalho de dados mais complexas. Sua origem remonta a 1986, no contexto do projeto POSTGRES da Universidade da Califórnia em Berkeley, e desde então, acumula mais de 35 anos de desenvolvimento contínuo e aprimoramento. Reconhecido por sua arquitetura sólida, confiabilidade, integridade de dados, extensibilidade e um conjunto abrangente de funcionalidades, o PostgreSQL conquistou uma forte reputação. Além disso, sua comunidade ativa e comprometida de código aberto garante a entrega constante de soluções inovadoras e de alto desempenho (MILANI, 2008).

A forte reputação do PostgreSQL é constatada pelos dados da pesquisa conduzida pela Stack Overflow (2024), na qual mais de 65 mil desenvolvedores compartilharam informações sobre as tecnologias e ferramentas que utilizam, aquelas que desejam aprender e suas experiências no ambiente de trabalho. Um dado interessante é que o PostgreSQL fez sua estreia na pesquisa de desenvolvedores em 2018, quando 33% dos participantes relataram utilizá-lo, enquanto a opção mais popular daquele ano, o MySQL, era usado por 59% dos desenvolvedores. Seis anos depois, o PostgreSQL é utilizado por 48,7% dos desenvolvedores e mantém-se como o banco de dados mais popular, conquistando esse título pelo segundo ano consecutivo, à frente do MySQL (40,3%), conforme mostrado na Figura 7 .

Figura 7 – Estatísticas da pesquisa realizada pelo Stack Overflow em 2024.



Fonte: Imagem retirada de (STACKOVERFLOW, 2024).

2.5.5 Apache HTTP Server

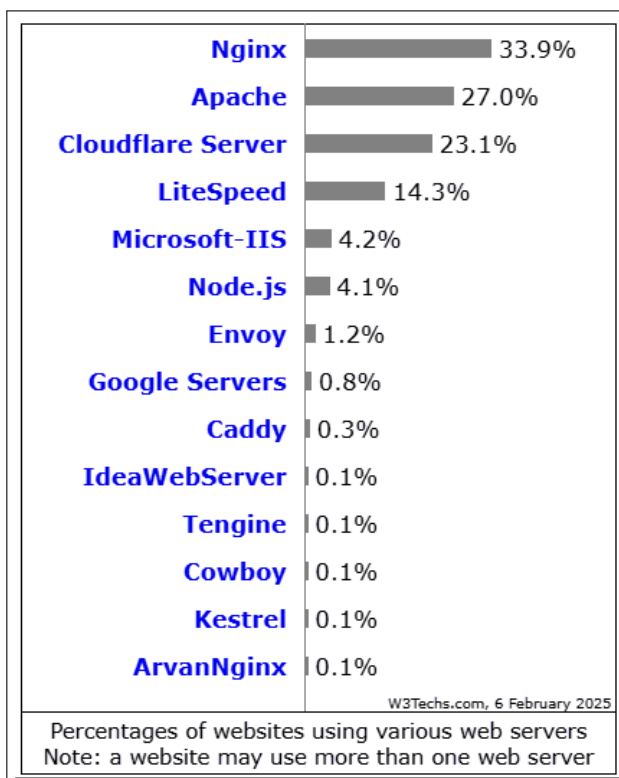
À medida que o desenvolvimento do sistema avançava, tornou-se possível considerar a necessidade de garantir sua disponibilidade na internet. Para isso, era essencial realizar sua hospedagem em um servidor web. Optou-se pelo Apache HTTP Server devido à sua robustez, flexibilidade, confiabilidade e por ser uma solução gratuita e amplamente adotada no mercado. O Apache é um servidor web que opera sobre sistemas operacionais como o Ubuntu Server, sendo responsável por processar requisições HTTP e disponibilizar os arquivos e recursos solicitados pelos usuários.

O Apache HTTP Server foi lançado em 1995 como um esforço colaborativo de desenvolvimento de software, com o objetivo de criar uma implementação robusta,

de nível comercial, rica em recursos e de código aberto de um servidor HTTP (Web) (FIELDING; KAISER, 2002).

De acordo com os dados fornecidos pela W3Techs (2025), o gráfico sobre o uso de servidores web indica que o Apache ocupa a segunda posição entre os servidores mais utilizados na internet, com 27% de participação entre os sites cuja tecnologia de servidor é conhecida (Figura 8).

Figura 8 – Estatísticas de uso de servidores web.



Fonte: Imagem retirada de (W3TECHS, 2025)

O projeto é gerido por um grupo de voluntários de diversas partes do mundo, que utilizam a internet para comunicação, planejamento e desenvolvimento tanto do servidor quanto de sua documentação. O Apache HTTP Server faz parte da Apache Software Foundation, uma organização dedicada a fornecer software de código aberto para o benefício público (KABIR, 2002).

Desde o seu lançamento, o Apache HTTP Server tem demonstrado sua relevância no mercado de aplicações web, mantendo-se como um dos principais servidores web da Internet até os dias atuais.

2.5.6 Ubuntu Server

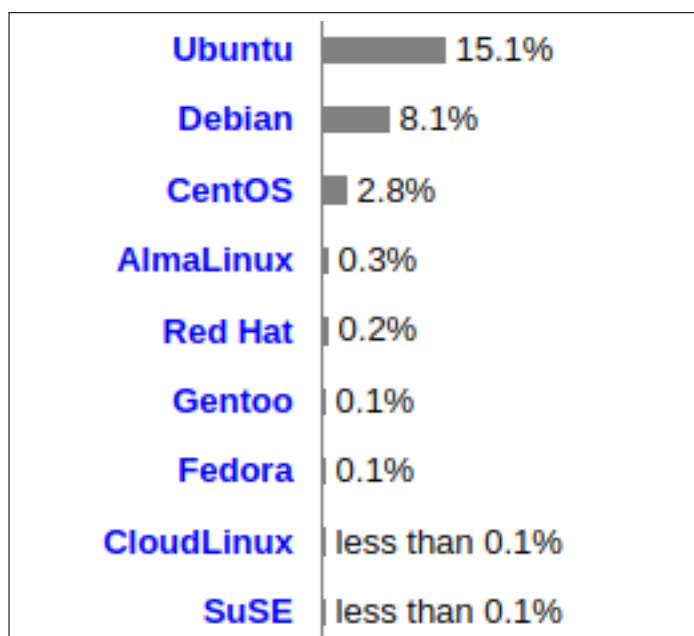
Optamos pelo sistema operacional Ubuntu Server em nosso projeto devido à sua robustez, flexibilidade e confiabilidade. Amplamente reconhecido por sua facilidade de instalação e manutenção, o Ubuntu Server também se destaca por contar com uma

comunidade ativa de desenvolvedores. Sua compatibilidade com diversas ferramentas e tecnologias essenciais para o desenvolvimento web, como servidores Apache e bancos de dados PostgreSQL, entre outros, assegura um ambiente estável, seguro e otimizado para atender às demandas do projeto.

Para compreender o funcionamento do Ubuntu Server, é essencial conhecer os fundamentos do Linux, que serve como núcleo (*kernel*) deste sistema operacional. O Linux é um *kernel* de código aberto criado em 1991 por Linus Torvalds, então estudante de Ciência da Computação na Universidade de Helsinque. Desde então, conquistou uma base de usuários globalmente massiva e é a plataforma escolhida para os 500 supercomputadores mais potentes do mundo. A popularidade do Linux se deve a diversos fatores, incluindo seus recursos robustos de segurança, flexibilidade e a ampla personalização que oferece. O *kernel* do Linux é mantido por uma comunidade global de desenvolvedores e entusiastas de código aberto, resultando em centenas de distribuições únicas que atendem a diferentes necessidades e preferências (FILHO, 2012).

De acordo com dados da W3Techs (2025), o Linux é utilizado por 56,2% dos servidores da web. O gráfico apresentado (Figura 9), mostra o uso de sistemas operacionais Linux na web, destacando o Ubuntu como a distribuição mais utilizada nesse universo, presente em 15,1% dos servidores que operam com Linux.

Figura 9 – Estatísticas de uso do distribuições Linux na web.



Fonte: Imagem retirada de (W3TECHS, 2025).

O Ubuntu Server é uma distribuição Linux desenvolvida especificamente para ambientes de servidor. Baseado no Ubuntu, ele oferece um sistema operacional completo e altamente otimizado, com suporte a uma ampla variedade de softwares essenciais, como SSH para acesso remoto, servidores web como o Apache HTTP

Server, bancos de dados como o PostgreSQL, suporte a linguagens de programação como PHP, entre muitas outras ferramentas fundamentais para a gestão de servidores (HAGEN, 2007).

2.6 *Front-end*

A camada de *front-end* do sistema desenvolvido, já representada na Figura 3, é responsável pela interface e experiência do usuário. Ele abrange tudo o que é exibido na tela e com o que o usuário interage diretamente, como layouts, botões, formulários e animações. Essa camada é composta principalmente por HTML (estrutura), CSS (estilo) e JavaScript (funcionalidade e interatividade). De forma resumida, seu funcionamento segue o seguinte fluxo:

1. O HTML define a estrutura da página.
2. O CSS aplica estilos visuais aos elementos da página.
3. O *framework* Bootstrap complementa o CSS, proporcionando um layout responsivo e componentes visuais pré-definidos.
4. O JavaScript adiciona interatividade à página, permitindo comportamentos dinâmicos.
5. A biblioteca jQuery facilita a manipulação dos elementos HTML e a execução de ações em resposta às interações do usuário.

Em síntese, o *front-end* representa a parte visível e interativa do sistema. Trata-se da camada da aplicação com a qual o usuário interage diretamente, sendo responsável pela interface gráfica e pela experiência de uso. Ele se comunica com o *back-end* por meio de requisições e envio de dados, integrando-se a esse ambiente para exibir informações e executar funcionalidades de forma dinâmica e responsiva (JÚNIOR et al., 2022).

2.6.1 Linguagem HTML

O HTML foi utilizado no desenvolvimento do sistema na estruturação e organização do conteúdo, permitindo a criação de interfaces para o usuário. Oferecendo a base para a construção de todos os elementos visuais e interativos, como formulários, tabelas, links e imagens, garantindo que o sistema seja compatível com diferentes navegadores. Além disso, o HTML facilita a integração com outras tecnologias, como CSS e JavaScript, para melhorar a estética e as funcionalidades.

O HTML foi criado em 1991 pelo cientista da computação britânico Tim Berners-Lee com o objetivo de facilitar o compartilhamento de pesquisas entre ele e seu grupo

de colegas. Trata-se de uma linguagem de marcação utilizada na construção de páginas web. Diferente de linguagens de programação, em que o programador tem liberdade para definir comportamentos a partir da entrada de dados, o HTML é utilizado para descrever a estrutura e a apresentação do conteúdo, permitindo que seja exibido corretamente aos usuários. Atualmente, o HTML é a base de qualquer site na internet, sendo essencial para a criação e organização de seu conteúdo (HICKSON, 2014).

A estrutura básica do HTML a seguir define os principais elementos que compõem uma página web. Ela começa com a declaração `<!DOCTYPE html>`, seguida pelo elemento `<html>`, que envolve todo o conteúdo. Dentro dele, o `<head>` contém informações como título, codificação e links para estilos, enquanto o `<body>` abriga o conteúdo visível, como textos, imagens, botões e formulários. Essa estrutura organiza e apresenta os elementos de forma que possam ser interpretados corretamente pelos navegadores.

```
[index.html]
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1.0">
    <title>Document</title>
  </head>
  <body>
  </body>
</html>
```

2.6.2 Linguagem CSS

Utilizou-se o CSS para a estilização e o aprimoramento visual da interface do usuário. Por meio do CSS, foi possível definir cores, fontes e outros elementos visuais, que contribuem para uma melhor experiência do usuário.

A linguagem HTML foi criada com o objetivo de marcar e estruturar páginas web, sem se preocupar com o estilo de apresentação dos elementos, como cores, tamanhos de fonte e outros aspectos visuais. Para resolver essa limitação, em 1994 surgiu a proposta das Folhas de Estilo em Cascata (CSS – *Cascading Style Sheets*), uma linguagem responsável pela formatação e estilização das páginas. Assim, enquanto o HTML se dedica à organização e estruturação do conteúdo, o CSS cuida da aparência visual desse conteúdo, proporcionando uma experiência mais agradável e personalizada ao usuário (WATRALL, 2009).

2.6.3 Linguagem JavaScript

O JavaScript foi empregado ao código-fonte do sistema para que fosse possível adicionar interatividade às interfaces, permitindo a manipulação dinâmica de conteúdos, a validação de formulários e, assim, aprimorando a experiência do usuário.

O desenvolvimento do JavaScript teve início em 1995, quando a empresa Netscape reconheceu a necessidade de tornar as páginas web mais dinâmicas. Observou-se que o HTML carecia de uma linguagem de programação simples e acessível, tanto para designers quanto para programadores iniciantes. Foi nesse contexto que o programador Brendan Eich criou o primeiro protótipo da linguagem e apresentando-o em maio de 1995. Ao longo dos anos, a linguagem foi padronizada e, em 1997, teve sua primeira versão oficial, chamada ECMAScript 1. Desde então, o JavaScript evoluiu, culminando no lançamento da ECMAScript 2020 em 2020 (SILVA, 2010).

2.6.4 *Framework* Bootstrap

Optamos por utilizar o Bootstrap em nosso sistema para garantir uma interface responsiva e adaptável a diferentes dispositivos, proporcionando uma experiência de usuário mais fluida e intuitiva. Além disso, sua ampla documentação e a facilidade de personalização nos permitiram acelerar o processo de desenvolvimento.

O Bootstrap é um *framework* CSS criado para facilitar o desenvolvimento de conteúdos da web responsivos, inicialmente desenvolvida pelo Twitter (atualmente X), e que se tornou uma das estruturas de *front-end* e projetos de código aberto mais populares do mundo. O Bootstrap oferece um conjunto de arquivos CSS e JavaScript que proporcionam um design profissional e responsivo, ajustando automaticamente o layout para diferentes dispositivos e tamanhos de tela (MORETO, 2016).

2.6.5 Biblioteca JQuery

O jQuery foi utilizado para simplificar a manipulação dos elementos DOM (*Document Object Model*), que representam todos os componentes de uma página web. Além disso, otimizou a criação de animações e facilitou a implementação de requisições AJAX (*Asynchronous JavaScript and XML*), uma técnica que permite atualizar partes de uma página web sem a necessidade de recarregá-la completamente.

O jQuery é uma biblioteca JavaScript leve, rápida e repleta de recursos. Lançada em 2006 por John Resig e disponibilizada como software livre e aberto, ou seja, de emprego e uso regido segundo licença MIT e GPL. Ela simplifica tarefas como manipulação de documentos HTML, gestão de eventos, animações e requisições Ajax, oferecendo uma API intuitiva que funciona em diversos navegadores (SILVA, 2013).

2.7 Git

Utilizamos o sistema de controle de versão Git no processo de desenvolvimento do sistema para permitir um monitoramento das alterações no código-fonte e assegurar a integridade do projeto ao longo do tempo. Com o Git, conseguimos realizar um controle preciso das versões, testar novas funcionalidades sem comprometer a estabilidade do sistema.

O núcleo (*kernel*) do Linux é um projeto de código aberto de grande escala. Durante a maior parte de sua fase de manutenção, entre 1991 e 2002, as alterações no código eram compartilhadas principalmente como correções e *patches*. Em 2002, o desenvolvimento do núcleo passou a adotar um sistema distribuído de controle de versão proprietário chamado BitKeeper. No entanto, em 2005, a relação entre a comunidade de desenvolvedores do Linux e a empresa responsável pelo BitKeeper se deteriorou, e a ferramenta passou a ser paga. Esse acontecimento alertou a comunidade, especialmente Linus Torvalds, seu criador, que decidiu criar uma solução própria. Assim nasceu o Git, um sistema de controle de versão distribuído, gratuito e de código aberto, projetado para gerenciar projetos de qualquer escala (CHACON; STRAUB, 2014).

No Git, os clientes não apenas acessam o estado mais recente dos arquivos, mas duplicam localmente todo o repositório. Dessa forma, se um servidor falhar e os sistemas estiverem colaborando por meio dele, qualquer repositório de cliente pode ser copiado de volta para o servidor para restaurá-lo. Cada clone funciona, na prática, como um backup completo de todos os dados.

2.8 Considerações finais do capítulo

A fundamentação teórica apresentada neste capítulo estabeleceu as bases conceituais necessárias para a compreensão do desenvolvimento do sistema proposto. Inicialmente, discutiram-se aspectos relacionados a projetos e ao gerenciamento aplicado em fundações de apoio institucional, evidenciando a importância de processos organizados e metodologias adequadas para alcançar os objetivos estabelecidos. Em seguida, a adoção da metodologia ágil Scrum foi contextualizada como estratégia fundamental para promover entregas incrementais, alinhamento contínuo com as necessidades do projeto e capacidade de adaptação frente a mudanças.

No âmbito tecnológico, foram abordados os principais paradigmas, arquiteturas e ferramentas que sustentam a construção do sistema. A programação orientada a objetos e a arquitetura MVC forneceram diretrizes para uma estruturação modular, coesa e de fácil manutenção. A separação entre as camadas de *back-end* e *front-end* revelou-se essencial para organização.

No *back-end*, destacaram-se as tecnologias PHP, Laravel, SQL, PostgreSQL, Apache e Ubuntu Server. Já no *front-end*, o uso de HTML, CSS, JavaScript, Bootstrap e jQuery possibilitou a criação de interfaces dinâmicas, responsivas e amigáveis ao usuário. Para sustentar o desenvolvimento de forma segura e controlada, adotou-se ainda o Git como sistema de versionamento, assegurando rastreabilidade e estabilidade no ciclo evolutivo do código-fonte.

Em conjunto, esses elementos teóricos e tecnológicos compõem o alicerce necessário para o desenvolvimento da solução proposta, garantindo não apenas sua implementação, mas também sua manutenibilidade e aderência às boas práticas de engenharia de software.

3 TRABALHOS RELACIONADOS

Neste capítulo, são apresentados estudos que, de alguma forma, forneceram embasamento teórico e contribuíram com uma visão prática sobre os temas relacionados ao trabalho proposto. Foram buscados trabalhos cujo foco principal fosse o gerenciamento de projetos por fundações de apoio institucional, bem como aqueles que utilizaram ferramentas e metodologias semelhantes às adotadas no desenvolvimento desta aplicação. Contudo, constatou-se uma escassez de produções acadêmicas que abordem especificamente o gerenciamento de projetos no contexto das fundações de apoio, o que reforça a relevância e a originalidade da proposta desenvolvida.

3.1 *Identificação e análise de variáveis de desempenho e sucesso que mais influenciam no gerenciamento de projetos nas fundações de apoio*

O gerenciamento de projetos em fundações de apoio institucional envolve desafios que vão além da execução técnica das atividades previstas, estendendo-se à gestão orçamentária, monitoramento de desempenho e adaptação constante a demandas emergentes para cada projeto. Neste contexto, alguns estudos têm buscado compreender e propor soluções para aprimorar esse tipo de gestão, entre eles o trabalho de Pereira et al. (2022) que realizou análises quantitativas e qualitativas das variáveis críticas que afetam o sucesso dos projetos geridos por fundações de apoio, com base em entrevistas, questionários e análise de documentos. O estudo destaca que os principais fatores que impactam o sucesso na gestão de projetos são: a clareza na definição dos objetivos, a capacitação da equipe gestora, a eficiência na alocação de recursos financeiros, e por último e não menos importante, a adoção de ferramentas tecnológicas para apoio ao gerenciamento interno.

Um dos aspectos mais relevantes do estudo é a constatação da carência de sistemas informatizados específicos para apoiar rotinas críticas, como a alocação e o remanejamento de recursos — tema central deste trabalho. Um sistema desenvolvido com essa finalidade deve abranger todo o ciclo de vida do projeto, com ênfase nas etapas de planejamento, controle, encerramento e comunicação, atendendo tanto aos requisitos normativos (como os estabelecidos pela ISO 21500, conforme Krause (2014)) quanto às necessidades práticas apontadas pelos gestores das fundações de apoio. Segundo os autores, a maioria dessas instituições ainda utiliza planilhas eletrônicas para o controle orçamentário, o que contribui para falhas, retrabalho e baixa rastreabilidade das operações financeiras — cenário também identificado na presente trabalho.

Portanto, neste trabalho daremos continuidade à direção proposta pelo estudo

mencionado, ao sair do diagnóstico e entrar na esfera da intervenção prática, entregando uma aplicação voltada a um problema identificado. A construção de uma ferramenta tecnológica que possa contribuir diretamente para o aumento da transparência, eficiência e segurança na gestão orçamentária de projetos conduzidos por fundações de apoio institucional.

3.2 Framework Laravel: Um Estudo de Caso Full Stack Development

O desenvolvimento de sistemas web para gerenciamento de processos específicos tem sido cada vez mais explorado na literatura, destacando-se a utilização de *frameworks* modernos que facilitam a implementação de soluções robustas e escaláveis. Um exemplo disso é o trabalho de Ferreira (2021), que desenvolveu uma aplicação web *full stack* utilizando o *framework* Laravel.

Ferreira desenvolveu um trabalho apresentado ao Curso de Ciência da Computação da Universidade Federal de Ouro Preto (UFOP) como requisito parcial para obtenção do título de Bacharel em Ciência da Computação. Este trabalho teve como objetivo principal desenvolver uma aplicação web que conecte um serviço de aluguel de quadras esportivas aos clientes finais. Ao longo do trabalho, foram analisadas as ferramentas essenciais para o desenvolvimento dessa aplicação, bem como os requisitos fundamentais para garantir seu funcionamento eficiente. Um dos focos principais é o estudo e aplicação do *framework* Laravel, buscando compreender e dominar essa ferramenta, bem como preparar etapas para o desenvolvimento de um projeto de aplicação web.

No estudo de caso apresentado por Ferreira, o Laravel foi escolhido devido à sua estrutura baseada no padrão arquitetural MVC, que promove uma separação clara das responsabilidades no desenvolvimento de aplicações web. A autora destaca que a escolha do Laravel proporcionou uma curva de aprendizado acessível mesmo para programadores com pouca experiência na ferramenta, além de oferecer uma ampla gama de recursos integrados que agilizam o processo de desenvolvimento, como roteamento de requisições, o mecanismo de *templates* Blade e o Eloquent ORM para manipulação de informações no banco de dados.

A aplicação desenvolvida por Ferreira visava conectar proprietários de quadras esportivas aos seus clientes, permitindo o gerenciamento de reservas, horários disponíveis e pagamentos. O sistema foi projetado para ser responsivo e de fácil utilização, atendendo às necessidades tanto dos administradores quanto dos usuários finais.

A relevância deste estudo para o presente trabalho reside na demonstração prática da aplicabilidade do *framework* Laravel no desenvolvimento de sistemas de gerenciamento. A experiência relatada por Ferreira reforça a viabilidade de utilizar este *framework* do ecossistema PHP como base para a construção de sistemas que demandam organização, segurança e eficiência.

O resultado final do trabalho não se limita apenas à entrega da aplicação conforme as especificações, mas também abrange o aprofundamento no entendimento e na utilização da ferramenta estudada, proporcionando um avanço significativo no conhecimento e compreensão dessa tecnologia e os estágios do desenvolvimento de um projeto de sistema web.

3.3 *Desenvolvimento de uma ferramenta de gestão para construção civil utilizando o Framework Laravel MVC*

O Trabalho de Conclusão de Curso (TCC), realizado por Silva (2019) , como parte do curso de Ciência da Computação na Universidade Federal de Goiás (UFG), teve como propósito a implementação de uma ferramenta para gestão na área da construção civil. No desenvolvimento deste projeto, foram empregadas várias tecnologias e metodologias específicas que também estão sendo utilizadas no desenvolvimento do projeto aqui proposto.

Silva propôs o desenvolvimento de uma ferramenta de gestão voltada para o setor da construção civil, com o objetivo de otimizar a gestão do tempo, dos custos operacionais e dos recursos disponíveis, por meio da substituição de processos manuais, como planilhas eletrônicas, por soluções automatizadas. Embora o foco do trabalho tenha sido a construção civil, as abordagens técnicas, metodológicas e as ferramentas utilizadas são aplicados ao processo de desenvolvimento da aplicação proposta no presente trabalho.

No desenvolvimento do trabalho de Silva, foram empregadas diversas tecnologias, entre elas as linguagens de programação PHP e JavaScript, a linguagem de marcação HTML, a linguagem de estilização CSS, o *framework* Laravel, a arquitetura MVC, a linguagem SQL para manipulação de dados e o *framework* Bootstrap para o desenvolvimento de interfaces de usuário. Essas tecnologias foram organizadas em camadas de *front-end* e *back-end*, abordagem também adotada no presente trabalho.

Por fim o autor conclui que as ferramentas descritas foram indispensáveis para o desenvolvimento do sistema, uma vez que, sem as mesmas, a organização do projeto seria custosa. Deste modo, foi possível focar nos requisitos necessários para o funcionamento da solução.

3.4 Considerações finais do capítulo

Os trabalhos analisados ao longo deste capítulo permitiram identificar contribuições relevantes para o desenvolvimento da proposta apresentada, embora também tenham evidenciado lacunas significativas na literatura, especialmente no que diz respeito ao gerenciamento de projetos em fundações de apoio institucional. O estudo de Pereira et al. (2022) destacou de forma clara a necessidade de ferramentas tecnoló-

gicas específicas para apoiar processos críticos, como o controle orçamentário e o remanejamento de recursos — exatamente o problema abordado por este trabalho.

Além disso, os estudos de Ferreira (2021) e de Silva (2019) demonstraram, por meio de aplicações reais, a eficácia do uso de tecnologias como PHP, Laravel, MVC, Bootstrap, SQL e demais componentes adotados na construção da solução aqui proposta. Esses trabalhos serviram como referências práticas tanto na organização arquitetural quanto na escolha das tecnologias, evidenciando sua capacidade de sustentar sistemas robustos.

Portanto, a análise dos trabalhos relacionados cumpre duas funções principais: fundamenta tecnicamente as escolhas metodológicas e tecnológicas do presente projeto e, simultaneamente, destaca a originalidade da solução desenvolvida, uma vez que poucos estudos abordam diretamente o contexto das fundações de apoio.

4 METODOLOGIA

Este capítulo descreve os métodos e procedimentos adotados para o desenvolvimento do sistema proposto, cujo objetivo é oferecer uma solução em sistema de computador que possa aumentar a eficiência e a precisão na alocação de recursos em projetos geridos por fundações de apoio institucional. A metodologia adotada abrange desde a análise das necessidades da instituição parceira até o desenvolvimento e validação do sistema, com ênfase na aplicação de técnicas da engenharia de software e metodologias ágeis.

4.1 Tipo de pesquisa

O presente trabalho caracteriza-se como uma pesquisa aplicada, uma vez que visa gerar conhecimento para aplicação prática, voltado à solução de problemas específicos relacionados ao remanejamento de recursos em projetos (PRODANOV; FREITAS, 2013). Para Vergara (2006), a pesquisa aplicada é motivada pela necessidade de resolver problemas concretos, imediatos ou não, tendo portanto finalidade prática. Trata-se também de uma pesquisa tecnológica de natureza qualitativa e exploratória, já que busca propor uma aplicação baseada na realidade observada em uma fundação de apoio institucional.

No ponto de vista dos objetivos, a metodologia deste trabalho pode ser classificada como um método exploratório. Segundo Selltiz (1975), esse método busca descobrir ideias e intuições na tentativa de encontrar maior familiaridade com o fenômeno estudado. Neste trabalho, visou-se compreender de forma mais aprofundada o processo de remanejamento em projetos geridos por fundações de apoio institucional, de modo a viabilizar o desenvolvimento de um sistema computacional que contribua para a melhoria desse processo.

4.2 Metodologia ágil Scrum

Como descrito na sessão 2.2 deste trabalho, utilizou-se a metodologia ágil Scrum para conduzir o desenvolvimento do sistema. O Scrum foi escolhido por permitir entregas incrementais e pela sua capacidade de adaptação a mudanças frequentes, o que é útil no contexto de sistemas que envolvem interação constante com usuários e adaptação de funcionalidades, conforme destaca Audy (2015).

O processo foi dividido em *Sprints*, cada uma com duração de trinta dias, conforme sugerido no ciclo padrão do Scrum. A cada início de *Sprint*, foi realizada uma reunião de planejamento (*Sprint Planning*), na qual foram definidos os requisitos a

serem desenvolvidos. Ao longo do ciclo, foram feitas reuniões diárias (*Daily Scrum*), com o objetivo de acompanhar o progresso das atividades e identificar impedimentos.

A Tabela 1 apresenta o cronograma das *sprints* planejadas para o desenvolvimento do sistema. O projeto foi dividido em oito *sprints*, cada uma com duração aproximada de um mês, permitindo entregas incrementais e contínuas. As quatro primeiras *sprints* serão apresentadas aqui no capítulo de metodologia, enquanto as quatro últimas serão descritas no capítulo de desenvolvimento, por estarem mais relacionadas à implementação do sistema.

Tabela 1 – Cronograma de Sprints

Sprint	Período	Entregas Principais
1	01/01/2025 – 31/01/2025	Levantamento de requisitos do sistema.
2	01/02/2025 – 28/02/2025	Elaboração de fluxogramas do sistema.
3	01/03/2025 – 31/03/2025	Elaboração de protótipos de tela.
4	01/04/2025 – 30/04/2025	Definição de ferramentas, configuração de ambiente e início de desenvolvimento.
5	01/05/2025 – 31/05/2025	Elaboração do diagrama de classe do sistema.
6	01/06/2025 – 30/06/2025	Implementação módulo de autenticação e cadastro de usuários.
7	01/07/2025 – 31/07/2025	Implementação do módulo de cadastro de rubricas e projetos.
8	01/08/2025 – 31/08/2025	Implementação do módulo de cadastro de ofícios e relatórios.

Ao final de cada *Sprint*, foram realizados os seguintes eventos:

- *Sprint Review*: Para demonstrar o que foi desenvolvido aos colaboradores da fundação de apoio e coletar os *feedbacks*.
- *Sprint Retrospective*: Uma reunião interna do time Scrum realizada após a *Sprint Review* e antes da próxima *Sprint Planning*, com o intuito de filtrar e alinhar as demandas posteriores.

O time Scrum, composto por funcionários da fundação de apoio, contou com os seguintes papéis:

- *Product Owner*: Ficou a cargo do gerente de projetos da fundação de apoio a responsabilidade de priorizar o *backlog* e validar as entregas;
- *Scrum Master*: O coordenador de tecnologia da informação da fundação de apoio estudada foi responsável por agendar as reuniões relacionadas aos processos do Scrum, bem como por analisar e filtrar as demandas das tarefas de desenvolvimento do sistema;

- *Development Team*: Como membro unitário da equipe de desenvolvimento, o autor deste trabalho ficou responsável tanto pela implementação do código-fonte do sistema quanto pela elaboração da documentação aqui apresentada.

4.3 *Sprint 1* - Levantamento de requisitos do sistema

Na primeira *sprint* do projeto, foram realizadas reuniões diárias (*Daily Scrum*) com os colaboradores internos da fundação. Esses encontros tiveram papel fundamental não apenas para alinhar a equipe de desenvolvimento, mas também para aproximar os futuros usuários do sistema ao processo de construção da solução. Nesse momento inicial, foi possível conduzir o levantamento de requisitos, etapa crucial para identificar as principais necessidades do sistema.

Os requisitos de um sistema consistem nas descrições do que ele deve realizar, nos serviços que deve oferecer e nas restrições que condicionam seu funcionamento. Esses requisitos refletem as necessidades dos clientes e definem as funcionalidades esperadas para que o sistema atenda a uma finalidade específica, como controlar um dispositivo, registrar um pedido ou disponibilizar informações (SOMMERVILLE, 2011).

4.3.1 Requisitos funcionais

Os requisitos funcionais de um sistema definem as funções e comportamentos que ele deve oferecer. Esses requisitos variam de acordo com o tipo de software a ser desenvolvido, o perfil de seus usuários e a abordagem adotada pela organização na sua especificação. Quando apresentados como requisitos de usuário, costumam ser descritos de forma mais abstrata, em linguagem acessível, de modo a facilitar a compreensão por parte dos futuros usuários do sistema (SOMMERVILLE, 2011).

A seguir apresentaremos os requisitos funcionais levantados na primeira *sprint* do projeto.

[RF01] Autenticar usuário

O sistema deve permitir que os usuários realizem autenticação antes de acessar suas funcionalidades. Esse requisito visa garantir a segurança do sistema, restringindo o acesso apenas a pessoas devidamente cadastradas.

[RF02] Validar autenticação de usuário

A autenticação deve ser realizada por meio da matrícula que é gerada pelo sistema integrado de gestão de fundações, em conjunto de uma senha pessoal. Essa abordagem reforça o vínculo do usuário com a instituição e assegura maior confiabilidade no processo de login.

[RF03] Controlar níveis de acesso de usuário

O sistema deve oferecer controle de níveis de acesso, diferenciando permissões de acordo com o perfil do usuário. Tal controle garante que cada usuário acesse somente as funcionalidades relacionadas às suas responsabilidades.

[RF04] Oferecer três tipos de perfil de usuário

O sistema deve contemplar três tipos de usuários: Controlador de orçamento, coordenador de projetos e administrador do sistema. Cada perfil possuirá permissões específicas, adequadas ao seu papel institucional.

[RF05] Delimitar permissões controlador de orçamento

O usuário com perfil de controlador de orçamento terá acesso apenas aos projetos sob sua responsabilidade. Essa limitação garante a segurança dos dados e evita interferência em projetos sob responsabilidades de outros técnicos com o mesmo perfil.

[RF06] Visualizar andamento de projetos para coordenador de projetos

O usuário com perfil de coordenador de projetos deve poder visualizar todos os projetos cadastrados, possibilitando um acompanhamento global dos projetos.

[RF07] Atualizar perfil de usuário

O sistema deve permitir que o usuário edite seu próprio perfil, possibilitando a atualização de informações pessoais.

[RF08] Cadastrar projeto

O sistema deve oferecer a funcionalidade de cadastro de projetos, contemplando os dados para acompanhamento e controle orçamentário. Trata-se de uma das principais funcionalidades do sistema.

[RF09] Cadastrar dados parciais de projeto

O processo de cadastro de projetos deve permitir que os dados sejam salvos parcialmente, possibilitando que o usuário continue o preenchimento posteriormente. Essa funcionalidade é importante devido ao alto volume de informações de determinados projetos.

[RF10] Exibir situação inicial do projeto

O sistema deve exibir o estado inicial do orçamento de cada projeto, registrado no momento do cadastro, e o estado atual, atualizado ao longo da execução. Isso permitirá o acompanhamento da evolução financeira do projeto.

[RF11] Alterar responsável do projeto

O sistema deve possibilitar a alteração do controlador de orçamento responsável por um projeto. Essa funcionalidade é necessária em situações de afastamento, como férias, garantindo a continuidade da gestão do projeto por meio de outro técnico controlador orçamentário.

[RF12] Cadastrar usuários

Apenas o administrador do sistema poderá realizar a inclusão e exclusão de usuários aptos a utilizar o sistema. Essa restrição assegura maior controle sobre os

acessos e reforça a segurança do ambiente.

[RF13] Cadastrar rubricas

O sistema deve permitir o cadastro de rubricas, possibilitando a categorização orçamentária de despesas dos projetos. Essa funcionalidade auxilia na organização e na análise financeira.

[RF14] Administrar rubricas

Apenas usuários com perfil de coordenador de projetos ou administrador do sistema poderão cadastrar, remover ou editar rubricas orçamentarias no sistema. Essa restrição evita modificações por usuários sem autoridade para alterar a estrutura orçamentária.

[RF15] Permitir cadastro de usuário de forma personalizada

O sistema deve permitir o registro de uma matrícula de usuários ou número de projeto de forma personalizada para casos em que a fundação de apoio não utilize o sistema integrado de gestão de fundações.

[RF16] Permitir geração de relatório do projeto

O sistema deve permitir a geração do relatório do projeto, incluindo o registro das alterações realizadas ao longo do tempo por meio dos ofícios.

[RF17] Gerar no relatório campo de assinaturas

No arquivo de relatório gerado pelo sistema, deve haver um campo destinado à assinatura do técnico controlador orçamentário, a ser utilizado em eventuais processos de auditoria.

4.3.2 Requisitos não funcionais

Os requisitos não funcionais, como o próprio nome indica, não estão diretamente vinculados às funcionalidades específicas disponibilizadas pelo sistema aos seus usuários. Em vez disso, referem-se a características de qualidade que determinam como o sistema deve operar. Tais requisitos estão frequentemente associados a propriedades emergentes, como confiabilidade, desempenho, tempo de resposta, usabilidade e eficiência na utilização de recursos, que influenciam diretamente a experiência do usuário e a robustez da aplicação (SOMMERVILLE, 2011).

A seguir apresentaremos os requisitos não funcionais levantados na primeira *sprint* do projeto.

[RFN01] Sistema em Web

O sistema deve ser desenvolvido como uma aplicação web, permitindo que os usuários acessem suas funcionalidades por meio de navegadores, sem a necessidade de instalação de programas adicionais. O sistema adota um modelo cliente-servidor baseado na Web. Nesse modelo, o cliente apenas envia requisições ao servidor, que é responsável por processá-las e retornar os resultados correspondentes. Todo o proces-

samento é realizado no servidor, enquanto os clientes acessam suas funcionalidades por meio da interface Web.

[RFN02] Ferramentas gratuitas

Todas as ferramentas utilizadas na implementação do projeto são gratuitas, incluindo a plataforma de programação, o sistema operacional, o sistema de gerenciamento de banco de dados e o servidor web.

[RFN03] Compatibilidade com dispositivos

O sistema deve ser compatível com diferentes tamanhos de tela e resoluções, garantindo usabilidade adequada em desktops, notebooks, tablets e smartphone. Dessa forma, todas as funcionalidades estarão acessíveis de forma consistente, independentemente do dispositivo utilizado pelo usuário, e a interface se ajustará automaticamente às dimensões da tela.

[RFN04] Tempo limite de inatividade

O sistema deve encerrar automaticamente a sessão do usuário após um período de inatividade de vinte minutos, garantindo maior segurança das informações contra acessos indevidos.

[RFN05] Implementação em PHP

Todas as rotinas de interação entre a interface gráfica do usuário e o banco de dados serão desenvolvidas em linguagem de programação PHP.

4.4 Sprint 2 – Elaboração de fluxogramas do sistema

Durante a execução da segunda *sprint* do projeto, foi possível começar a visualizar como determinados processos seriam estruturados. Essas discussões permitiram documentar o fluxo das atividades por meio da elaboração de fluxogramas, tornando mais clara a compreensão das etapas, decisões e interações envolvidas.

Os fluxogramas são ferramentas essenciais para representar visualmente a lógica de um processo, permitindo compreender a sequência em que os procedimentos devem ser executados, as decisões a serem tomadas e os caminhos resultantes de cada escolha. Quando elaborados com um nível adequado de detalhamento, tornam-se instrumentos valiosos para a definição do escopo, pois deixam claro e explícito o que precisa ser realizado, bem como os pontos de decisão e suas alternativas. Dessa forma, contribuem para alinhar o entendimento de todos os envolvidos e minimizar ambiguidades durante o desenvolvimento de um projeto (DEBASTIANI, 2016).

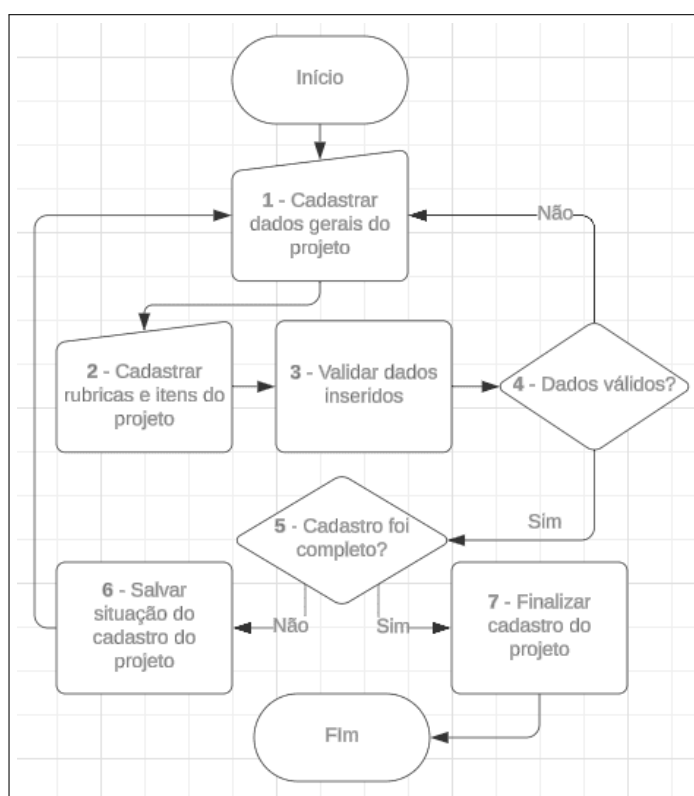
4.4.1 Fluxograma do cadastro de projeto

O fluxograma apresentado a seguir (Figura 10) descreve o fluxo do processo de cadastro de projetos no sistema. Com o objetivo de facilitar a compreensão, optou-se

por incluir no texto os números de referência correspondentes às etapas destacadas na figura.

O procedimento tem início na etapa de cadastro dos dados gerais do projeto (1), na qual são inseridas as informações básicas necessárias para a identificação do projeto.

Figura 10 – Fluxograma do cadastro de projeto



Fonte: Imagem elaborada pelo autor.

Na sequência, o sistema avança para a fase de cadastro das rubricas e itens do projeto (2), em que são adicionados os elementos orçamentários vinculados. Concluída essa etapa, realiza-se a validação dos dados inseridos (3), garantindo a consistência e a integridade das informações. Essa verificação contempla, por exemplo, a checagem de campos obrigatórios não preenchidos e a detecção de valores inválidos, como números negativos em registros orçamentários.

Caso a validação identifique inconsistências (4), o processo retorna para a etapa de cadastro de dados gerais (1), a fim de que as devidas correções sejam realizadas. Se os dados forem validados com sucesso, o sistema procede à verificação da completude do cadastro (5).

Quando o cadastro ainda não estiver concluído, o sistema disponibiliza a funcionalidade salvar situação do cadastro do projeto (6), permitindo que o processo seja interrompido e retomado posteriormente. Essa funcionalidade foi especificada no RF09 (Requisito Funcional 09), considerando que o preenchimento pode demandar

tempo significativo, especialmente em projetos de grande porte, que exigem elevado nível de detalhamento das rubricas e itens. Diante desse cenário, estabeleceu-se a necessidade de um recurso de salvamento parcial, possibilitando ao usuário pausar o cadastro sem risco de perda das informações já registradas.

Por fim, quando o cadastro estiver completo, o processo avança para a etapa de finalização do cadastro do projeto (7), encerrando o fluxo. A partir desse momento, o projeto e seu plano orçamentário ficam consolidados no sistema, tornando-se aptos a receber ofícios de remanejamento.

4.4.2 Fluxograma do cadastro de ofício de remanejamento

O próximo fluxograma (Figura 11) ilustra o processo de cadastro e aplicação de alterações orçamentárias em projetos a partir de ofícios de remanejamentos enviados pelos coordenadores de projetos.

Para solicitar o remanejamento no plano de aplicação do projeto, o coordenador deverá encaminhar à fundação de apoio institucional um ofício contendo a justificativa da solicitação e a descrição detalhada das alterações pretendidas, especificando-as por rubrica e item (CULTURA, 2019).

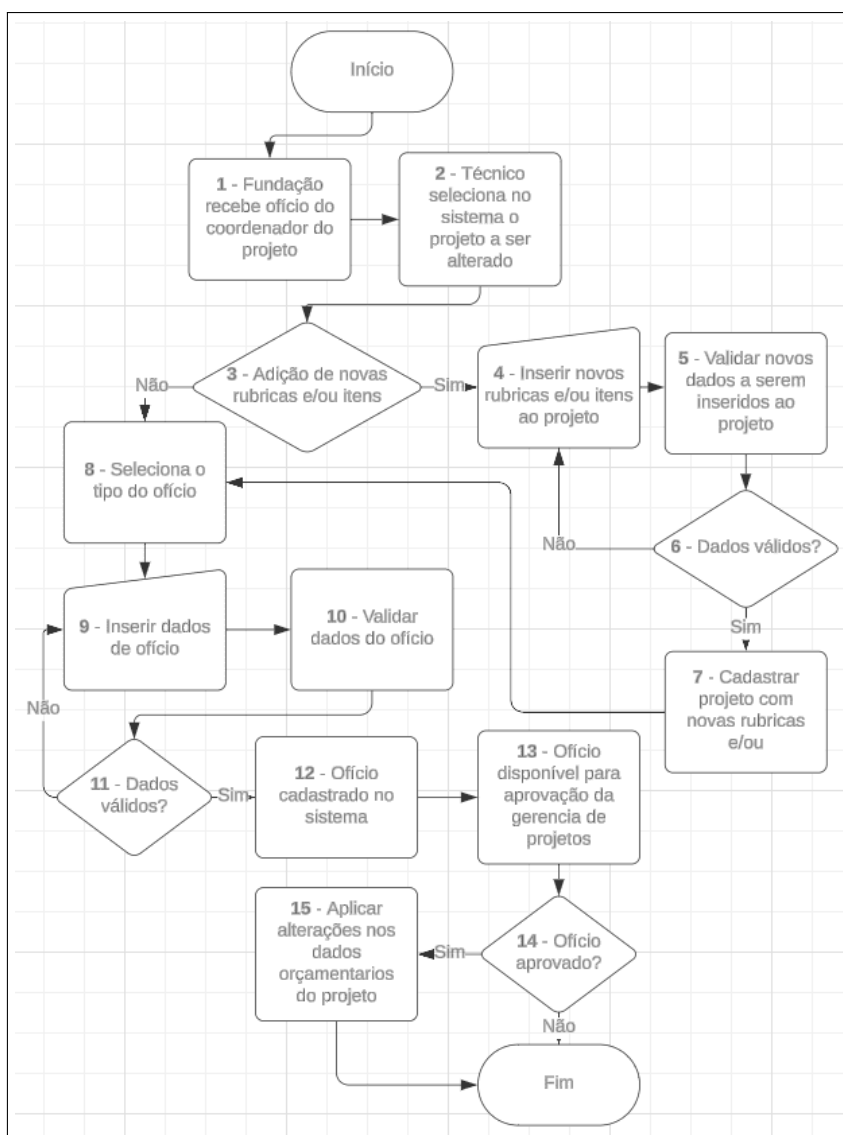
O fluxograma inicia-se com o recebimento, pela fundação, de um ofício enviado pelo coordenador do projeto (1). Em seguida, o técnico responsável acessa o sistema e seleciona o projeto a ser alterado (2).

A seguir, verifica-se a necessidade de adição de novas rubricas e/ou itens (3):

- Caso haja adições, estas são registradas no sistema (4) e, em seguida, submetidas à etapa de validação (5). Essa validação consiste em verificar se os dados inseridos não são nulos e se, no caso de valores orçamentários, não apresentam valores negativos. Se a validação (6) identificar inconsistências, o processo retorna à etapa de inserção para correção (4). Uma vez validados positivamente, o projeto é atualizado com as novas rubricas e/ou itens (7).
- Caso não haja adições, o técnico segue para a seleção do tipo de ofício (8), inserindo posteriormente as informações correspondentes (9).

As informações do ofício passam, então, por uma nova etapa de validação (10). Essa etapa ocorre, por exemplo, nos casos de ofícios de remanejamento de itens: quando um valor é transferido de uma rubrica para outra, o sistema verifica se o montante a ser removido do item de origem está efetivamente disponível. Se forem identificadas inconsistências (11), os dados retornam à etapa de inserção para correção (9). Após a validação positiva, o ofício é cadastrado no sistema (12) e disponibilizado para aprovação pela gerência de projetos (13).

Figura 11 – Fluxograma do cadastro de ofício de remanejamento



Fonte: Imagem elaborada pelo autor.

Na etapa de aprovação (14), se o ofício for rejeitado, o processo é encerrado. Caso seja aprovado, o sistema aplica as alterações nos dados orçamentários do projeto (15), concluindo o fluxo.

No contexto dos ofícios, as solicitações podem envolver diferentes tipos de remanejamento, de acordo com a natureza da alteração proposta:

- **Aditivo de Orçamento:** Inclusão de novos recursos financeiros ao projeto, ampliando o valor total aprovado inicialmente.
- **Aditivo de Vigência:** Extensão do prazo de execução do projeto, garantindo tempo adicional para cumprimento das atividades previstas.
- **Alteração de Item:** Modificação ou substituição de itens previamente aprovados no orçamento, ajustando-os às novas necessidades do projeto.

- Remanejamento de Item: Realocação de valores entre diferentes itens orçamentários, sem alteração do montante global do projeto.
- Remanejamento de Rendimento: Redistribuição dos valores oriundos de rendimentos financeiros do projeto, direcionando-os para rubricas específicas.
- Uso de Rendimentos: Aplicação direta dos rendimentos financeiros gerados pelos recursos do projeto, conforme regras estabelecidas pela fundação e pela instituição de fomento.

4.5 *Sprint 3* - Elaboração de protótipos de tela.

Na execução da terceira *sprint* do projeto, deu-se início a definição de interfaces do sistema. Nesse processo, foram criados protótipos de telas de baixa fidelidade, com o objetivo de orientar o desenvolvimento da camada de apresentação (*front-end*). Esses protótipos iniciais possibilitaram a validação visual com os usuários antes do início da implementação do sistema.

Um protótipo é uma versão inicial de um programa de computador, criada com o propósito de demonstrar conceitos, explorar alternativas de projeto e aprofundar a compreensão do problema a ser solucionado, bem como de suas possíveis soluções. O desenvolvimento ágil e iterativo do protótipo é essencial para que os usuários finais possam experimentar e avaliar, desde cedo, as características do que futuramente será o produto final (SOMMERVILLE, 2011).

A ferramenta utilizada para a geração dos protótipos foi o Balsamiq, que é uma ferramenta de *wireframing* (processo de criação de esboços ou estruturas visuais para interfaces) que auxilia equipes de desenvolvimento enxutas a transformar ideias iniciais em diretrizes claras e acionáveis, de forma rápida e eficiente (FARANELLO, 2012).

4.5.1 Protótipo de tela de login

A Figura 12 apresenta o protótipo da tela de login do sistema. Essa é a primeira interface exibida ao usuário e tem como finalidade realizar o processo de autenticação, assegurando o acesso apenas a usuários autorizados, em conformidade com o requisito funcional RF01.

No centro da tela encontra-se um formulário composto por dois campos principais — Matrícula e Senha — conforme definido no requisito funcional RF02. Esses campos devem ser preenchidos com as credenciais do usuário cadastradas na base de dados. Abaixo, há um botão rotulado como “Entrar”, responsável por enviar as informações ao *back-end* do sistema para validação. Em caso de autenticação bem-sucedida, o usuário terá acesso ao sistema; caso contrário, será exibida uma mensagem informando a inconsistência nos dados inseridos.

Figura 12 – Protótipo de tela de login

O protótipo de tela de login é apresentado dentro de uma janela simulando um navegador web. No topo da janela, há uma barra de endereço com o texto "Sistema de Remanejamentos - Início" e ícones de navegação (setas e uma casa). Abaixo da barra, o título "Sistema de Remanejamentos" é exibido em um cabeçalho azul. O corpo principal da tela é cinza e contém um formulário centralizado com o seguinte layout:

- Um campo de entrada rotulado "Matrícula".
- Um campo de entrada rotulado "Senha".
- Um botão azul com o texto "Entrar" localizado abaixo dos campos de entrada.

Na base da janela, há uma barra decorativa azul.

Fonte: Imagem elaborada pelo autor.

4.5.2 Protótipo de tela de cadastro de usuário

O protótipo apresentado na Figura 13 corresponde à tela de cadastro de usuários. Essa interface é destinada exclusivamente aos administradores do sistema, conforme estabelecido nos requisitos funcionais RF03 e RF12, uma vez que apenas esse perfil possui permissão para gerenciar acessos e registrar novos usuários na base de dados.

A tela é composta por um formulário com os seguintes campos obrigatórios:

- **Matrícula:** campo destinado à identificação única do usuário na instituição, gerado automaticamente pelo sistema integrado de gestão de fundações. Caso a fundação não adote esse formato de matrícula, o campo poderá ser preenchido com um *username* (nome de usuário) sem espaços ou caracteres acentuados, em conformidade com o requisito RF15.
- **Nome:** campo destinado à inserção do nome completo do usuário.
- **Senha:** campo destinado à definição da credencial de acesso do usuário. Inicialmente, o administrador do sistema atribui uma senha padrão no momento do cadastro. Após o primeiro acesso, o próprio usuário poderá alterá-la para uma senha de sua preferência.
- **Perfil:** campo do tipo *combo box* que permite a seleção do perfil de acesso do usuário. Os perfis disponíveis, conforme definido no requisito funcional RF04, são:
 1. **Coordenador de projetos:** responsável pelo monitoramento e acompanhamento dos projetos gerenciados pela fundação de apoio;

Figura 13 – Protótipo de tela de cadastro de usuário

Fonte: Imagem elaborada pelo autor.

2. Controlador de orçamento: técnicos encarregados de realizar as intervenções necessárias no plano orçamentário do projeto;
3. Administrador do sistema: responsável pela gestão e manutenção geral da aplicação.

Na parte inferior da interface, encontram-se dois botões de ação:

- Cadastrar Usuário: botão responsável por efetivar o cadastro usuário no sistema.
- Cancelar: botão que permite descartar a operação atual e retornar à tela anterior.

4.5.3 Protótipo de tela de cadastro de projeto

A Figura 14 apresenta o protótipo da tela de cadastro de projetos do sistema, em conformidade com o requisito funcional RF08. Esta interface funciona como uma primeira etapa do cadastro de projetos e tem como finalidade possibilitar o registro dos dados essenciais de um projeto gerido por uma fundação de apoio institucional.

O formulário apresentado é composto pelos seguintes campos:

Figura 14 – Protótipo de tela de cadastro de projeto

O protótipo da tela de cadastro de projeto é apresentado dentro de uma janela simulando um navegador. No topo da janela, há uma barra de endereço com o texto "Sistema de Remanejamentos - Início" e ícones de navegação. O cabeçalho da interface possui o título "Sistema de Remanejamentos" à esquerda, um ícone de menu hambúrguer rotulado "Menu" no centro, e um perfil de usuário rotulado "Nome do Usuário" com um link "Sair" à direita. O formulário principal, intitulado "Cadastro de Projeto", contém campos para: "Número" (campo de texto), "Número do Contrato" (campo de texto), "Nome" (campo de texto), "Tipo" (menu suspenso rotulado "ComboBox"), "Data de início" (campo de data com ícone de calendário), "Data de término" (campo de data com ícone de calendário), "Coordenador" (campo de texto) e "Técnico responsável" (menu suspenso rotulado "ComboBox"). Na base do formulário, há dois botões: "Próximo" com uma seta para a direita e "Cancelar" com um ícone de X.

Fonte: Imagem elaborada pelo autor.

- Número: campo destinado ao código identificador do projeto, gerado automaticamente pelo sistema integrado de gestão de fundações. Caso a fundação não adote esse formato de número, o campo poderá ser preenchido com um número sem espaços ou pontos, em conformidade com o requisito RF15.
- Número do Contrato: número associado ao contrato formal do projeto.
- Nome: nome completo do projeto.
- Tipo: campo do tipo combo box, para seleção do tipo do projeto (A, B, C ou D).
 - Tipo A: quando a universidade contratar a fundação de apoio para gestão administrativa e financeira de projetos acadêmicos, incluindo captação e recebimento de recursos.
 - Tipo B: quando a universidade contratar a fundação de apoio gestão administrativa e financeira de projetos acadêmicos com repasse de recursos da universidade, de órgãos públicos ou de convênios estaduais e municipais.

- Tipo C: quando a fundação de apoio contrata a universidade para execução de projetos de pesquisa, desenvolvimento científico e tecnológico ou fomento à inovação, por parceria ou encomenda com ressarcimento.
- Tipo D: quando houver celebração de contrato tripartite entre a universidade, a fundação de apoio e instituições contratantes, incluindo agências de fomento, empresas públicas, organizações sociais, entidades privadas e demais órgãos governamentais.
- Data de início e Data de término: campos com seleção por calendário, responsáveis pelo controle da vigência do projeto.
- Coordenador: campo para o nome do coordenador responsável pelo projeto.
- Técnico responsável: campo do tipo combo box para seleção do usuário técnico Controlador de Orçamento.

Na parte inferior da tela estão posicionados os botões de ação:

- Próximo: avança para a próxima etapa do cadastro, caso os dados validados.
- Cancelar: permite a interrupção do processo e o retorno à tela anterior.

4.5.4 Protótipo de tela de adição de rubrica e itens

A Figura 15 demonstra o protótipo da tela de adição de rubrica e itens ao projeto. Esta tela funcionará como a segunda etapa do cadastro de projetos, essa funcionalidade é essencial para o registro da estrutura orçamentária do projeto, permitindo o vínculo entre rubricas orçamentárias e seus respectivos itens de despesa.

Na parte superior da interface, é exibido o título “Adicionar Rubrica ao Projeto X”, contextualizando o usuário sobre o projeto que está sendo editado. Em seguida, é apresentado um campo do tipo combo box para seleção da rubrica a ser associada, contendo o código e o nome descritivo da rubrica.

O sistema contará também com um módulo de gerenciamento de rubricas orçamentárias, no qual será possível adicionar, atualizar e remover rubricas, conforme especificado no requisito RF13. Essas rubricas poderão ser selecionadas nesta etapa do cadastro de projetos.

Abaixo, é exibida uma tabela que apresenta os itens orçamentários adicionados à rubrica selecionada, organizados em duas colunas:

- Descrição: nome do item (ex: Item a, Item b, etc.).
- Orçamento inicial: valor inicialmente previsto para o item, expresso em reais, conforme definido no plano orçamentário do projeto encaminhado à fundação.

Figura 15 – Protótipo de tela de adição de rubrica e itens

Sistema de Remanejamentos - Início

Sistema de Remanejamentos

Menu Nome do Usuário Sair

Adicionar Rubrica ao Projeto X

Rubrica

000 - Rubrica Exemplo

Adicionar Item +

Descrição	Orçamento inicial
Item a	R\$ 10.000,00
Item b	R\$ 60.000,00
Item c	R\$ 100.000,00
Item d	R\$ 20.000,00

Adicionar Rubrica + Cancelar X

Fonte: Imagem elaborada pelo autor.

Acima da tabela, há um botão “Adicionar Item”, permitindo ao usuário incluir novas linhas com informações adicionais à rubrica. Ao final da tela, dois botões de ação são disponibilizados:

- Adicionar Rubrica: grava as informações da rubrica e dos itens inseridos, vinculando-os ao projeto.
- Cancelar: descarta as alterações e retorna à tela anterior.

Após o preenchimento de todas as rubricas e seus respectivos itens, o projeto estará formalmente cadastrado na base de dados do sistema.

4.5.5 Protótipos de tela de cadastro de ofício

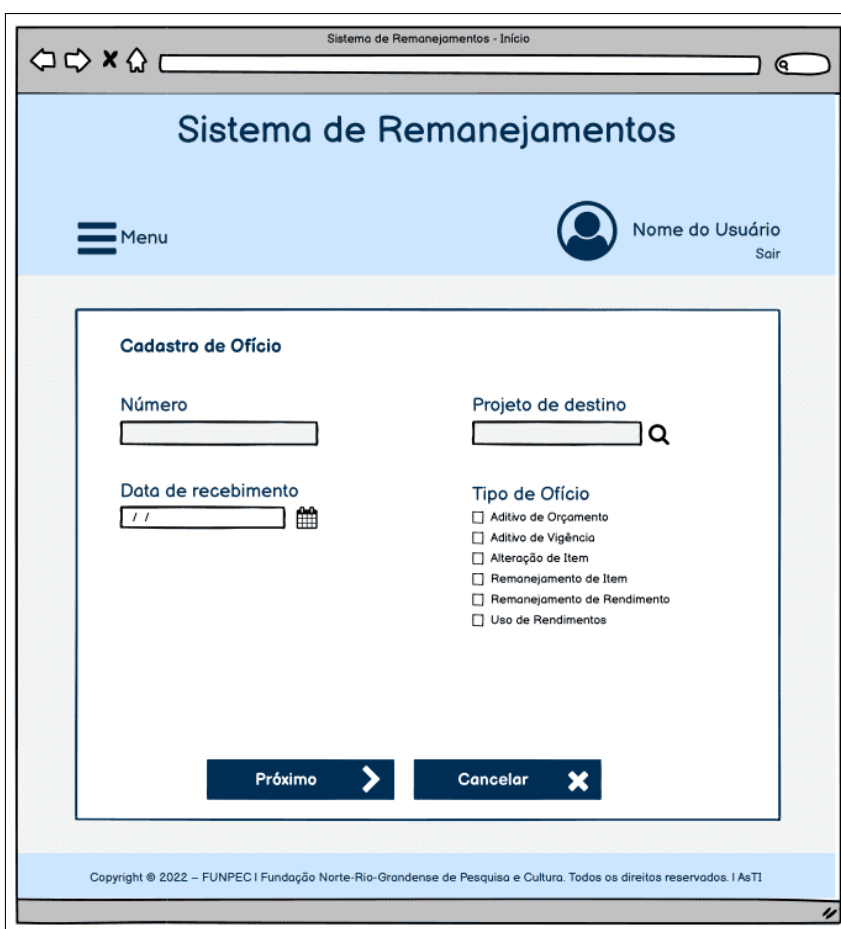
Com o projeto devidamente cadastrado, os remanejamentos serão realizados mediante solicitação formal por meio de ofício, visando à reformulação ou alteração no plano de aplicação. Para isso, o coordenador do projeto encaminha à fundação de apoio institucional um ofício contendo a justificativa da solicitação e a descrição

detalhada das alterações pretendidas, especificando-as por rubrica e item, para assim ser efetuado o remanejamento (CULTURA, 2019).

4.5.5.1 Protótipo de tela de cadastro de tipo de ofício

A seguir, apresentamos o protótipo da tela de cadastro de ofício (Figura 16). Essa interface foi desenvolvida para registrar e categorizar os ofícios recebidos pela fundação de apoio, os quais estão diretamente relacionados à gestão e às alterações realizadas na estrutura do projeto.

Figura 16 – Protótipo de tela de cadastro de ofício



O protótipo da tela de cadastro de ofício é apresentado dentro de uma janela simulando um navegador. No topo da janela, há uma barra de endereço com o texto "Sistema de Remanejamentos - Início" e ícones de navegação. O cabeçalho da interface é azul claro e contém o título "Sistema de Remanejamentos", um ícone de menu hambúrguer à esquerda e, à direita, um perfil de usuário com o nome "Nome do Usuário" e o link "Sair".

O formulário principal, intitulado "Cadastro de Ofício", está centralizado e possui os seguintes campos:

- Número:** Um campo de texto para a inserção do número identificador.
- Projeto de destino:** Um campo de texto com um ícone de lupa à direita para busca.
- Data de recebimento:** Um campo de data com o formato " / / " e um ícone de calendário.
- Tipo de Ofício:** Uma lista de opções com caixas de seleção:
 - ☐ Aditivo de Orçamento
 - ☐ Aditivo de Vigência
 - ☐ Alteração de Item
 - ☐ Remanejamento de Item
 - ☐ Remanejamento de Rendimento
 - ☐ Uso de Rendimentos

Na base do formulário, há dois botões: "Próximo" com uma seta para a direita e "Cancelar" com um ícone de X.

Na barra de rodapé da janela, há o texto: "Copyright © 2022 – FUNPEC | Fundação Norte-Rio-Grandense de Pesquisa e Cultura. Todos os direitos reservados. | AsTI".

Fonte: Imagem elaborada pelo autor.

A tela é composta por um formulário com os seguintes campos:

- **Número:** campo para inserção do número identificador do ofício, gerado pelo sistema integrado de gestão de fundações.
- **Projeto de destino:** campo para associar o ofício a um projeto previamente cadastrado no sistema. Ao lado do campo há um ícone de busca, permitindo a seleção do projeto.

- Data de recebimento: campo com funcionalidade de calendário, destinado ao registro da data em que o ofício foi oficialmente recebido pela fundação.
- Tipo de ofício: conjunto de opções do tipo *checkbox*, que permite marcar uma ou mais naturezas do ofício recebido: Aditivo de Orçamento, Aditivo de Vigência, Alteração de Item, Remanejamento de Item, Remanejamento de Rendimento, Uso de Rendimentos.

Na parte inferior da tela, dois botões de ação são disponibilizados:

- Próximo: permite avançar para a próxima etapa do processo de cadastro.
- Cancelar: cancela a operação atual e retorna à tela anterior.

4.5.5.2 Protótipo de tela de ofício de remanejamento de itens

Apresentaremos a seguir o protótipo referente à etapa seguinte do cadastro de ofício (Figura 17), considerando o cenário em que o usuário indicou que o ofício recebido é do tipo Remanejamento de Item. Essa interface foi projetada para possibilitar o registro de operações de transferência de recursos entre diferentes rubricas e itens de um projeto.

O formulário é composto por campos que permitem a seleção dos elementos envolvidos no processo de remanejamento:

- Rubrica de origem: campo destinado à seleção da rubrica de origem dos recursos, previamente cadastrada durante o processo de registro do projeto.
- Item de origem: campo destinado à seleção do item vinculado à rubrica escolhida, do qual os recursos serão remanejados.
- Rubrica de destino: campo destinado à seleção da rubrica de destino, para a qual serão alocados os recursos remanejados da rubrica de origem.
- Item de destino: campo para seleção do item vinculado à rubrica de destino escolhida, do qual os recursos serão alocados.
- Valor remanejado: campo para valor da transferência (formato monetário).

Abaixo do formulário, há uma tabela que lista os remanejamentos já adicionados ao ofício, apresentando as seguintes colunas: Rubrica de origem, Item de origem, Rubrica de destino, Item de destino, Valor remanejado.

Um botão “Remanejar” com ícone de adição permite inserir o registro na lista, possibilitando múltiplos remanejamentos antes da confirmação final.

Na parte inferior da tela, os botões de ação são:

Figura 17 – Protótipo de tela de remanejamento de itens

Sistema de Remanejamentos

Sistema de Remanejamentos - Início

Menu Nome do Usuário Sair

Remanejamento de Itens

Rubrica de origem
Selecionar rubrica

Rubrica de destino
Selecionar rubrica

Item de origem
Selecionar item

Item de destino
Selecionar item

Valor remanejado (R\$)

Remanejar +

Rubrica de Or	Item de Ori	Rubrica de De	Item de De	Valor remanej
Rubrica Exempl	Item b	Rubrica Exemplo	Item a	R\$ 20.000,00

Próximo > < Voltar

Fonte: Imagem elaborada pelo autor.

- Próximo: avança para a etapa final de confirmação do cadastro do ofício.
- Voltar: retorna à tela anterior sem confirmar alterações.

4.5.5.3 Protótipo de tela de conclusão do cadastro de ofício

A Figura 18 apresenta um protótipo de interface do sistema. Nessa tela, o usuário visualiza um resumo do ofício antes da confirmação do cadastro. Os campos exibidos incluem o número do ofício, o projeto de destino, a data de recebimento e o tipo de ofício, sendo este último categorizado em múltiplas opções como Aditivo de Orçamento, Aditivo de Vigência, Alteração de Item, Remanejamento de Item, Remanejamento de Rendimento e Uso de Rendimentos.

Na parte inferior da interface, são disponibilizados dois botões de ação principais: “Cadastrar Ofício”, que confirma a operação, e “Cancelar”, que retorna à etapa anterior. Ressalta-se que, durante esta *sprint*, não foi possível validar junto aos colaboradores internos da fundação estudada quais dados deveriam, de fato, ser exibidos nessa tela, permanecendo como uma decisão de projeto a ser refinada em etapas posteriores do desenvolvimento.

Figura 18 – Protótipo de tela de conclusão do cadastro de ofício

Sistema de Remanejamentos - Início

Sistema de Remanejamentos

Menu Nome do Usuário Sair

Resumo do Ofício

Número
013/2022 - PR

Projeto de destino
Projeto X

Data de recebimento
02/10/2022

Tipo de Ofício

- ☒ Aditivo de Orçamento
- ☒ Aditivo de Vigência
- ☒ Alteração de Item
- ☒ Remanejamento de Item
- ☒ Remanejamento de Rendimento
- ☒ Uso de Rendimentos

Cadastrar Ofício ✓ Cancelar ✕

Copyright © 2022 – FUNPEC | Fundação Norte-Rio-Grandense de Pesquisa e Cultura. Todos os direitos reservados. | AsTI

Fonte: Imagem elaborada pelo autor.

Concluída essa etapa, o remanejamento ficará disponível para análise e aprovação do gerente de projetos, se aprovado, as alterações passarão a vigorar no projeto referenciado.

4.6 *Sprint 4* - Definição de ferramentas e início de desenvolvimento

A quarta *sprint* do projeto teve como foco a definição das ferramentas de desenvolvimento a serem utilizadas, etapa essencial para dar início à implementação do código-fonte do sistema. O *Scrum Master* do projeto concedeu total liberdade ao *Development Team* para a escolha das tecnologias, estabelecendo como sendo os principais critérios: familiaridade com as ferramentas, aplicabilidade no contexto do projeto, confiabilidade e robustez das ferramentas escolhidas.

- PHP: Linguagem de programação voltada para o desenvolvimento de aplicações web, escolhida por sua ampla utilização no mercado e compatibilidade com diferentes servidores.
- Laravel: Framework baseado em PHP, adotado para estruturar o projeto dentro da arquitetura MVC, promovendo organização do código e agilidade no desenvolvimento.

- PostgreSQL: Sistema gerenciador de banco de dados relacional robusto e de código aberto, utilizado para garantir integridade, consistência e desempenho no armazenamento das informações.
- JavaScript: Linguagem essencial para a construção de interfaces interativas e dinâmicas, ampliando a experiência do usuário dentro do sistema.
- HTML e CSS: Tecnologias fundamentais para a estruturação e estilização das páginas do sistema, promovendo usabilidade e boas práticas de design responsivo.
- Bootstrap: *Framework front-end* que facilitou a criação de interfaces responsivas e padronizadas, reduzindo o tempo de desenvolvimento e garantindo compatibilidade entre diferentes dispositivos.

Para o versionamento e controle colaborativo do código-fonte, foi utilizado o Git, integrado ao GitLab. O GitLab é uma plataforma que possibilitou não apenas o armazenamento do repositório remoto, mas também o gerenciamento de tarefas, acompanhamento do progresso do projeto, integração contínua e colaboração eficiente entre os envolvidos no processo de desenvolvimento.

O Git é um sistema de controle de versão que você usa para rastrear alterações no seu código e colaborar com outras pessoas. O GitLab é um gerenciador de repositórios Git baseado na web que fornece CI/CD e outros recursos para ajudar você a gerenciar o ciclo de vida do desenvolvimento de software (PASSAMA, 2016).

4.7 Considerações finais do capítulo

A metodologia apresentada neste capítulo estruturou todas as etapas necessárias para o desenvolvimento do sistema proposto. A abordagem adotada permitiu integrar fundamentos teóricos, práticas consolidadas da engenharia de software e técnicas de alinhamento contínuo com os usuários finais, buscando que o sistema atendesse às necessidades reais da fundação de apoio institucional estudada.

As quatro primeiras *sprints* desempenharam um papel fundamental no amadurecimento da solução. Na *Sprint 1*, o levantamento de requisitos — tanto funcionais quanto não funcionais — estabeleceu a base conceitual que guiou as etapas subsequentes. A *Sprint 2* permitiu a modelagem dos processos por meio de fluxogramas, promovendo uma visão estruturada das atividades críticas do sistema. Já na *Sprint 3*, a elaboração dos protótipos de tela facilitou a validação visual das interfaces e forneceu um direcionamento sólido para a camada de apresentação do sistema. Por fim, a *Sprint 4* consolidou a seleção das ferramentas tecnológicas e marcou o início efetivo da implementação, garantindo alinhamento entre requisitos, arquitetura e viabilidade técnica.

5 DESENVOLVIMENTO

Este capítulo apresenta o processo de desenvolvimento do sistema proposto, descrevendo a implementação das funcionalidades planejadas a partir das definições realizadas na etapa metodológica. O trabalho foi conduzido de forma incremental, seguindo os princípios do *framework* Scrum, em que cada *sprint* resultou em um conjunto de entregas validadas com os colaboradores da fundação de apoio estudada.

5.1 *Sprint* 5 - Elaboração de diagrama de classes do sistema

Na quinta *sprint* do projeto ficou-se na modelagem do diagrama de classes (Figura 19) que compõe a estrutura do sistema desenvolvido. Para fins de clareza e organização, cada um dos elementos representados será esmiuçado e explicado em detalhe ao longo deste capítulo, abrangendo atributos e relacionamentos entre classes.

Pedimos desculpas ao leitor caso não seja possível visualizar de forma legível o diagrama completo apresentado neste trecho do documento. Reiteramos, porém, que ele será explicado detalhadamente ao longo deste capítulo.

A classe *Projeto* funciona como elemento central do modelo, agregando informações fundamentais como orçamento, vigência, coordenador, situação e demais características administrativas. Ela mantém associações diretas com classes de apoio, como *Usuario*, *TipoProjeto*, *SituacaoProjeto*, além de relacionar-se com coleções de *RubricaProjeto* e *Oficio*.

As classes *TipoProjeto*, *SituacaoProjeto*, *Perfil*, *TipoCadastro*, *SituacaoAprovacao*, *SituacaoOficio* e *SituacaoAditivo* possuem estrutura simples, servindo como tabelas auxiliares para classificação e controle de estados no sistema.

A classe *Usuario*, associada a um *Perfil*, representa os atores que interagem com o sistema, incluindo informações de autenticação e permissões.

O fluxo financeiro do projeto é estruturado a partir da classe *Rubrica*, que é vinculada à classe *RubricaProjeto*, responsável por relacionar rubricas específicas aos projetos. Cada *RubricaProjeto*, por sua vez, pode conter múltiplos *ItemRubrica*, que representam os itens orçamentários detalhados, com valores unitários, quantidades, descrições e situações de aprovação.

O gerenciamento documental ocorre por meio da classe *Oficio*, que representa solicitações, registros e documentos formais vinculados aos projetos. Cada ofício pode originar diferentes tipos de aditivos, representados por classes especializadas:

- *AditivoVigencia*: alterações na vigência do projeto.
- *AditivoAlteracaoItem*: modificação de itens específicos da rubrica.

- *AditivoOrcamento*: ajustes de valores orçamentários.
- *AditivoUsoRendimento*: solicitações de uso de rendimento financeiro.
- *AditivoRemanejamentoItem*: movimentação orçamentária entre itens.
- *AditivoRemanejamentoRendimento*: remanejamento envolvendo rendimentos de itens.

Cada tipo de aditivo mantém relação direta com *Oficio*, *RubricaProjeto*, *Item-Rubrica* e uma *SituacaoAditivo*, permitindo acompanhamento do fluxo de aprovação e execução das solicitações.

Esse diagrama, portanto, sintetiza a arquitetura lógica do sistema e define uma base conceitual para implementação. As próximas seções aprofundarão cada classe apresentada, descrevendo sua função no sistema, seus atributos, métodos e a justificativa de seus relacionamentos.

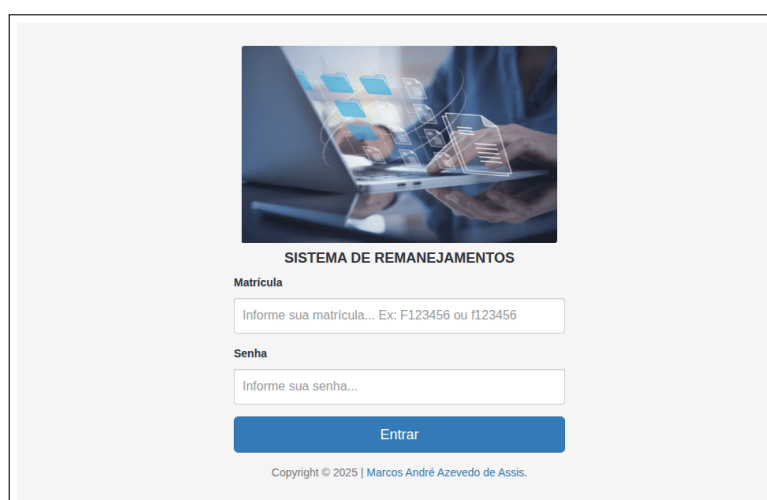
5.2 *Sprint* 6 - Implementação de autenticação e cadastro de usuários

A sexta *sprint* concentrou-se na construção da base do sistema, responsável pela autenticação de usuários e pelo gerenciamento dos usuários e perfis de acesso.

5.2.1 Autenticação de usuários

Os funcionários da fundação de apoio ao acessar o sistema por meio de seu navegador de internet, são redirecionados para a tela de autenticação (Figura 20), conforme o requisito RF01, onde inserem suas credenciais de acesso - a matrícula na instituição e a senha correspondente, de acordo com o requisito RF02.

Figura 20 – Tela de Autenticação.



A imagem mostra a interface de autenticação do 'SISTEMA DE REMANEJAMENTOS'. No topo, há uma imagem de uma pessoa usando um laptop com gráficos de barras e linhas sobrepostos. Abaixo, o título 'SISTEMA DE REMANEJAMENTOS' é seguido pelo rótulo 'Matricula'. Um campo de entrada contém o texto 'Informe sua matrícula... Ex: F123456 ou f123456'. Abaixo disso, o rótulo 'Senha' precede um campo de entrada com o texto 'Informe sua senha...'. Um botão azul com o texto 'Entrar' está posicionado abaixo dos campos. Na base da interface, uma linha de copyright indica 'Copyright © 2025 | Marcos André Azevedo de Assis.'

Fonte: Imagem elaborada pelo autor.

O botão “Entrar” realiza a validação das credenciais informadas na base de dados da aplicação. Caso estejam corretas, o usuário é redirecionado para o painel principal do sistema. Em caso de falha na autenticação, uma mensagem de erro é exibida, solicitando nova tentativa.

O trecho de código a seguir (Figura 21) define a classe *LoginFormRequest*, responsável pela validação dos dados de autenticação enviados pelo usuário antes de serem processados pelo *LoginController*.

Figura 21 – Código para validação de valores de autenticação.

```
5  class LoginFormRequest extends FormRequest {
6      public function authorize() { return true; }
7
8      public function rules() {
9          return ['matricula' => 'required|min:7', 'senha' => 'required|min:6'];
10     }
11
12     public function messages() {
13         return [
14             'matricula.min' => 'Matrícula deve ter no mínimo 7 caracteres.',
15             'senha.min' => 'Senha deve ter no mínimo 6 caracteres.',
16         ];
17     }
18 }
```

Fonte: Imagem elaborada pelo autor.

A classe herda de *FormRequest*, recurso do *framework* Laravel que facilita a validação de formulários. O método *rules()* define as regras de validação, exigindo que o campo matrícula seja obrigatório e possua no mínimo sete caracteres, e que o campo senha também seja obrigatório com pelo menos seis caracteres.

Por fim, o método *messages()* personaliza as mensagens de erro apresentadas ao usuário caso as regras de validação não sejam atendidas, promovendo maior clareza no processo de autenticação.

O próximo trecho de código que veremos (Figura 22) foi retirado do arquivo *LoginController* e apresenta o método responsável por realizar o processo de autenticação de usuários no sistema.

O código inicia recebendo a requisição que foi validada pela classe *LoginFormRequest*, contendo as credenciais informadas pelo usuário (matrícula e senha). Em seguida, é realizada uma consulta ao banco de dados, buscando um registro na tabela *usuarios* que corresponda à matrícula informada (convertida para minúsculas) e à senha criptografada com o algoritmo SHA-256, que é um algoritmo desenvolvido pela NSA (Agência de Segurança Nacional dos EUA) (PELLEGRINI, 2025).

Caso um registro seja encontrado, significa que existe um usuário com as devidas credenciais, então o sistema autentica este usuário através do método *Auth::loginUsingId()* e o redireciona para o painel principal do sistema. Caso contrário, se as credenciais não forem encontradas na base de dados, o sistema retorna para tela de

Figura 22 – Código para autenticação de usuário.

```

21 public function autenticar_usuario(LoginFormRequest $request) {
22     $usuario = DB::table('usuarios')
23         ->select('id')
24         ->where('matricula', '=', strtolower($request->matricula))
25         ->where('senha', '=', hash('sha256', $request->senha))
26         ->first();
27
28     if($usuario != null) {
29         Auth::loginUsingId($usuario->id);
30
31         return redirect()->to('projeto');
32     }
33
34     return redirect()->back()->withErrors(
35         ['msg' => 'Matrícula e/ou senha incorreta(s)! Tente novamente.']);
36 }

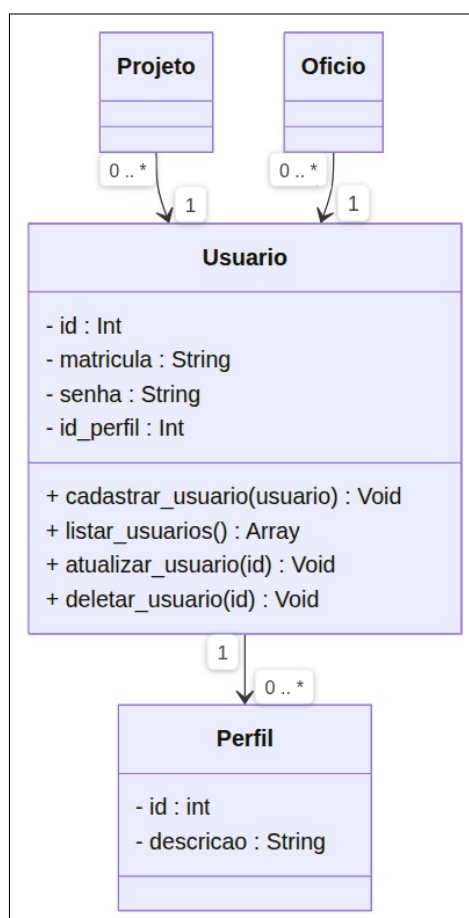
```

Fonte: Imagem elaborada pelo autor.

autenticação exibindo uma mensagem de erro informando que as credenciais estão incorretas.

5.2.2 Modelagem do diagrama de classe de Usuário

Figura 23 – Classe de Usuário.



Fonte: Imagem elaborada pelo autor.

No início do desenvolvimento do módulo de gerenciamento de usuários foi elaborado o diagrama de classes (Figura 23), que representa a modelagem das entidades responsáveis pelo gerenciamento de usuários no sistema.

A classe *Usuario* centraliza as informações e operações relacionadas aos usuários cadastrados, possuindo os atributos *id*, *matricula*, *senha* e *id_perfil*. Esta classe também define os principais métodos de manipulação de dados, como *cadastar_usuario()*, *listar_usuarios()*, *atualizar_usuario()* e *deletar_usuario()*.

A classe *Perfil* está associada à classe *Usuario* em uma relação de um para muitos, permitindo que cada perfil seja vinculado a diversos usuários. Ela contém os atributos *id* e *descricao*, responsáveis por identificar e caracterizar o tipo de perfil.

Além disso, observa-se que a classe *Usuario* possui associações com as classes *Projeto* e *Oficio*, indicando que cada usuário pode estar relacionado à criação ou gerenciamento de múltiplos projetos e ofícios no sistema.

5.2.3 Cadastro de usuários

A Figura 24 apresenta a tela de cadastro de usuários do sistema, responsável por permitir que administradores registrem novos perfis de acesso. Essa funcionalidade é essencial para o controle e a segurança do sistema, garantindo que apenas usuários autorizados possam interagir com as funcionalidades internas.

Esta funcionalidade está disponível no sistema pelo caminho *USUÁRIOS > Cadastrar Usuário* e é acessível apenas para usuários com perfil de Administrador do sistema, conforme RF12.

Figura 24 – Tela de cadastro de usuários.

Fonte: Imagem elaborada pelo autor.

O formulário de cadastro é composto pelos seguintes campos:

- Nome: destinado à identificação nominal do usuário;
- Matrícula: campo que armazena o identificador institucional do usuário, utilizado também no processo de autenticação;
- Perfil do Usuário: campo do tipo *select*, que permite a definição do nível de acesso do novo usuário (Administrador do Sistema, Gerente de Projetos ou Controlador Orçamentário);
- Senha: campo protegido, no qual o administrador define a senha inicial do usuário, que posteriormente o usuário deverá alterar para uma senha personalizada.

Na parte inferior do formulário, há dois botões principais:

- Voltar: retorna à tela anterior sem salvar as alterações;
- Cadastrar usuário: envia os dados do formulário para inserção no banco de dados.

O trecho de código a seguir (Figura 25) foi retirado do arquivo *UsuarioController* e apresenta o método responsável por realizar o cadastro de novo usuário no sistema.

Figura 25 – Código para cadastro de usuário.

```
32 public function cadastrar_usuario(UsuarioFormRequest $request) {  
33     $usuario = new Usuario;  
34  
35     $usuario->nome = mb_strtoupper($request->get('nome'));  
36     $usuario->matricula = strtolower($request->get('matricula'));  
37     $usuario->perfil = $request->get('perfil');  
38     $usuario->senha = hash('sha256', $request->get('senha'));  
39  
40     $usuario->save();  
41  
42     return redirect()->to('usuario')  
43         ->with('message', 'Usuário cadastrado com sucesso!');  
44 }
```

Fonte: Imagem elaborada pelo autor.

O método recebe os dados enviados pelo formulário de cadastro, validados previamente pela classe *UsuarioFormRequest*. Em seguida, é criada uma nova instância do modelo *Usuario*, na qual os atributos são preenchidos com as informações fornecidas: o nome é convertido para letras maiúsculas, a matrícula para letras minúsculas, o perfil é definido conforme a escolha do usuário e a senha é criptografada utilizando o algoritmo SHA-256.

Após a atribuição dos dados, o método executa o comando `save()`, que persiste o novo registro no banco de dados. Por fim, o sistema redireciona o usuário para a

página de listagem de usuários, exibindo uma mensagem de sucesso confirmando o cadastro realizado.

5.2.4 Listagem e gerenciamento de usuários

A tela de listagem de usuários do sistema (Figura 26), também é acessível apenas aos usuários com perfil de administrador do sistema. Essa tela tem como principal objetivo permitir o controle centralizado dos perfis cadastrados, possibilitando a visualização, edição e exclusão de registros existentes, bem como o acesso ao formulário para criação de novos usuários.

Figura 26 – Tela de listagem de usuários.

NOME DO USUÁRIO	MATRÍCULA	PERFIL	OPÇÕES
MARCOS ANDRÉ AZEVEDO DE ASSIS	f005500	ADMINISTRADOR DO SISTEMA	[Editar] [Excluir]
[REDAZIDO]	f005136	ADMINISTRADOR DO SISTEMA	[Editar] [Excluir]
[REDAZIDO]	f000850	GERENTE DE PROJETOS	[Editar] [Excluir]
[REDAZIDO]	f002950	CONTROLADOR ORÇAMENTARIO	[Editar] [Excluir]
[REDAZIDO] ERREIRA DE LIMA	f002640	GERENTE DE PROJETOS	[Editar] [Excluir]
[REDAZIDO] E ARAUJO FILHO	f002519	CONTROLADOR ORÇAMENTARIO	[Editar] [Excluir]
[REDAZIDO] AGA DE SOUSA	f003271	CONTROLADOR ORÇAMENTARIO	[Editar] [Excluir]

Fonte: Imagem elaborada pelo autor.

Na parte superior da tela, encontra-se o botão “Cadastrar novo Usuário”, que redireciona para o formulário de cadastro apresentado anteriormente. Abaixo, há um campo de busca que permite localizar rapidamente usuários pelo nome ou matrícula, otimizando o processo de administração de contas.

A listagem exibe uma tabela contendo as seguintes colunas:

- Nome do Usuário: identifica o colaborador cadastrado no sistema;
- Matrícula: código institucional ou personalizado utilizado para autenticação;
- Perfil: define o tipo de permissão atribuída (por exemplo, Administrador do Sistema, Gerente de Projetos ou Controlador Orçamentário);
- Opções: apresenta ícones de ação que permitem editar (ícone azul) ou excluir (ícone vermelho) o respectivo registro.

Na exclusão de usuários, é realizada uma validação, permitindo a remoção apenas quando o usuário não possui vínculos ou movimentações associadas a ofícios ou projetos. Essa verificação garante a integridade dos dados e evita a exclusão de registros que estejam relacionados a outras informações essenciais do sistema.

O trecho de código a seguir (Figura 27) apresenta o método responsável por realizar a exclusão de usuários do sistema.

Figura 27 – Código para deletar usuário.

```
public function deletar_usuario($id) {
    $oficio = DB::table('oficios')
        ->where('id_usuario_responsavel', $id)
        ->get();
    $projeto = DB::table('projetos')
        ->where('id_tecnico_responsavel', $id)
        ->orWhere('id_usuario', $id)
        ->get();

    if(count($oficio) > 0) {
        return redirect()->back()->withErrors(['msg' =>
            'Não foi possível deletar o usuário pois o mesmo tem movimentação em ofício(s)!']);
    } else if(count($projeto) > 0) {
        return redirect()->back()->withErrors(['msg' =>
            'Não foi possível deletar o usuário pois o mesmo tem movimentação em projeto(s)!']);
    }
    $usuario = Usuario::findOrFail($id)->delete();

    return redirect()->back()->with('message', 'Usuário deletado com sucesso!');
}
```

Fonte: Imagem elaborada pelo autor.

Inicialmente, o método recebe o identificador do usuário (id) como parâmetro e realiza duas consultas: a primeira na tabela oficios, verificando se o usuário está associado como responsável em algum registro de ofício, e a segunda na tabela projetos, para identificar se o usuário possui vínculo como controlador orçamentário ou como usuário que efetuou o cadastro do projeto no sistema.

Caso o usuário possua registros relacionados a ofícios ou projetos, o sistema impede a exclusão e retorna uma mensagem de erro apropriada, informando o motivo. Essa verificação evita inconsistências e perda de dados relacionados.

Se não houver vínculos entre o usuários e ofícios ou projetos, o método localiza o usuário por meio do método `findOrFail()` e realiza sua exclusão com o comando `delete()`. Após a remoção bem-sucedida, o sistema retorna exibindo uma mensagem de confirmação da exclusão.

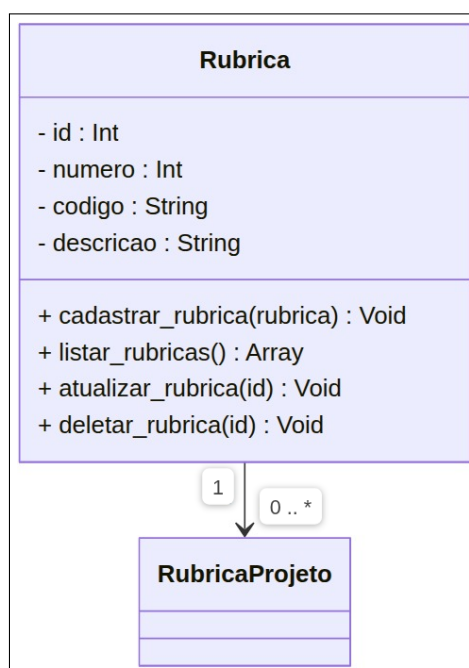
5.3 *Sprint 7* - Implementação de módulos de rubricas e projetos.

A sétima *sprint* concentrou-se na implementação das funcionalidades de gerenciamento de rubricas e projetos. Nesta etapa, foram desenvolvidas as interfaces e

lógicas de controle responsáveis por permitir o registro, validação e armazenamento das informações relacionadas às rubricas e aos projetos vinculados às fundações.

5.3.1 Modelagem do diagrama de classe de Rubrica

Figura 28 – Classe de Rubrica



Fonte: Imagem elaborada pelo autor.

O diagrama de classes apresentado na Figura 28 ilustra a modelagem da entidade responsável pelo controle das rubricas orçamentárias no sistema.

A classe *Rubrica* contém os atributos *id*, *numero*, *codigo* e *descricao*, que armazenam as informações para identificação e caracterização de cada rubrica.

Os seus métodos são: *cadastrar_rubrica()*, *listar_rubricas()*, *atualizar_atualizar()* e *deletar_deletar()*, que implementam as operações básicas de manipulação dos registros, garantindo o gerenciamento completo das rubricas cadastradas no sistema.

A classe *RubricaProjeto* mantém uma relação de dependência com *Rubrica*, representando a associação entre as rubricas e os projetos aos quais pertencem. Essa relação é de um para muitos, ou seja, uma rubrica pode estar vinculada a vários projetos.

5.3.2 Cadastro de rubricas

A Figura 29 apresenta a tela de cadastro de rubricas do sistema, atendendo assim ao requisito RF13. Essa interface tem como finalidade permitir o registro de rubricas orçamentárias utilizadas nos projetos administrados pela fundação. As rubricas

representam categorias de despesa e receita, sendo fundamentais para a estruturação e o controle financeiro de cada projeto.

Figura 29 – Tela de cadastro de rubricas.

Fonte: Imagem elaborada pelo autor.

Esta funcionalidade está disponível no sistema pelo caminho RUBRICAS > Cadastrar Rubrica e é acessível apenas para usuários com perfil de Administrador do sistema e Gerente de Projetos.

O formulário exibido é composto por três campos principais:

- Código: identifica a rubrica de forma única dentro do sistema;
- Número: campo para numeração da rubrica;
- Descrição: permite detalhar a finalidade ou natureza da rubrica cadastrada.

Abaixo dos campos, encontram-se dois botões de ação: “Voltar”, que retorna à tela anterior sem salvar os dados, e “Cadastrar Rubrica”, responsável por submeter as informações inseridas ao servidor para validação e gravação no banco de dados.

Apenas usuários com o perfil de gerente de projetos ou administrador do sistema possuem permissão para acessar essa funcionalidade.

O trecho de código abaixo (Figura 30) corresponde ao *front-end* que gera o formulário da tela apresentada na figura 29.

O código representa o formulário de cadastro de rubricas no sistema, desenvolvido em HTML e integrado à rota responsável pelo envio dos dados no endereço através do método POST.

O formulário é estruturado utilizando classes do Bootstrap, o que garante um *layout* responsivo e organizado. Ele contém três campos principais, todos definidos

Figura 30 – Código *front-end* do formulário de cadastro de rubrica.

```

23 <form method="POST" action="http://localhost:8080/rubrica"
24   accept-charset="UTF-8" autocomplete="off">
25   <div class="row">
26     <div class="col-lg-3 col-sm-3 col-xs-12">
27       <div class="form-group">
28         <label>Código</label>
29         <input type="text" name="codigo" value="{{ old('codigo') }}" class="form-control"
30           placeholder="Informe o código da Rubrica..." required>
31       </div>
32     </div>
33     <div class="col-lg-3 col-sm-3 col-xs-12">
34       <div class="form-group">
35         <label>Número</label>
36         <input type="number" name="numero" value="{{ old('numero') }}" class="form-control"
37           placeholder="Informe o número da Rubrica..." required>
38       </div>
39     </div>
40     <div class="col-lg-6 col-sm-6 col-xs-12">
41       <div class="form-group">
42         <label>Descrição</label>
43         <input type="text" name="descricao" value="{{ old('descricao') }}" class="form-control"
44           placeholder="Informe a descrição da Rubrica..." required>
45       </div>
46     </div>
47   </div>
48   <hr>
49   <div class="form-group">
50     <a class="btn btn-warning" href="{{ url('rubrica') }}" role="button">
51       <i class="fa fa-arrow-left" aria-hidden="true"></i> Voltar
52     </a>
53     <button class="btn btn-primary" type="submit">
54       <i class="fa fa-floppy-o" aria-hidden="true"></i> Cadastrar Rubrica
55     </button>
56   </div>
57 </form>

```

Fonte: Imagem elaborada pelo autor.

como obrigatórios por meio do atributo `required`. Cada campo possui também um `placeholder` que orienta o usuário sobre o tipo de informação a ser inserida.

Na parte inferior do formulário, há dois botões: um botão de voltar e um botão de submissão, responsável por enviar as informações preenchidas por meio da rota estabelecida no `action` que envia para efetuar o cadastro da nova rubrica.

5.3.3 Listagem e gerenciamento de rubricas

A tela de listagem de rubricas (Figura 31) foi desenvolvida com o objetivo de exibir e gerenciar as rubricas cadastradas no sistema. Esta interface apresenta uma tabela dinâmica que organiza as rubricas em colunas correspondentes ao código, número e descrição, permitindo ao usuário uma visualização das rubricas disponíveis.

Na parte superior da tela, está localizado o botão “Cadastrar nova Rubrica”, que direciona o usuário para o formulário de inserção de novas rubricas. Logo abaixo, há um campo de busca interativa, possibilitando a filtragem das rubricas por número ou descrição, o que facilita a localização de informações dentro da base de dados.

Cada registro exibido na tabela apresenta duas opções de ação: o ícone de edição, representado por um botão na cor azul, e o ícone de exclusão, identificado por um botão na cor vermelha. Essas funcionalidades estão disponíveis exclusivamente

Figura 31 – Tela de listagem de rubricas.

CÓDIGO	NÚMERO	DESCRIÇÃO DA RUBRICA	OPÇÕES
31.90.11	11	VENCIMENTO E VANTAGENS FIXAS	[Edit] [Delete]
31.90.13	13	OBRIGACOES PATRONAIS	[Edit] [Delete]
33.90.14	14	DIARIAS PESS.CIV E MIL.	[Edit] [Delete]
33.90.18	18	AUXILIO FINANCEIRO A ESTUDANTES	[Edit] [Delete]
33.90.20	20	AUXILIO FINANCEIRO A PESQUISADORES	[Edit] [Delete]
33.90.30	30	MATERIAL DE CONSUMO (NACIONAL)	[Edit] [Delete]
33.90.31	31	PREMIACOES	[Edit] [Delete]

Fonte: Imagem elaborada pelo autor.

para usuários com perfil de administrador do sistema ou gerente de projetos.

Na funcionalidade de exclusão de rubricas, foi implementada uma validação que impede a remoção de registros vinculados a projetos existentes. Caso uma rubrica já esteja associada ao plano orçamentário de algum projeto, o sistema bloqueia sua exclusão da base de dados, garantindo a consistência e confiabilidade das informações cadastradas. Essa verificação evita que dados essenciais sejam removidos inadvertidamente, preservando a integridade das relações entre as tabelas do sistema. O funcionamento dessa validação pode ser observado no trecho de código a seguir (Figura 32).

Figura 32 – Código para deletar rubrica.

```
public function deletar_rubrica($id) {
    $rubricaProjeto = DB::table('rubricas_projeto')
        ->select('id')
        ->where('id_rubrica', $id)
        ->get();

    if(count($rubricaProjeto) > 0) {
        return redirect()->back()->withErrors(['msg' =>
            'Rubrica não pode ser deletada, pois está vinculada a projeto(s)!']);
    }
    $rubrica = Rubrica::findOrFail($id)->delete();

    return redirect()->back()->with('message', 'Rubrica deletada com sucesso!');
}
```

Fonte: Imagem elaborada pelo autor.

O código apresenta o método recebe como parâmetro o identificador da rubrica (id) e, inicialmente, realiza uma consulta na base de dados. Essa consulta verifica se a

rubrica está associada a algum projeto cadastrado, buscando registros que possuam o mesmo identificador de rubrica.

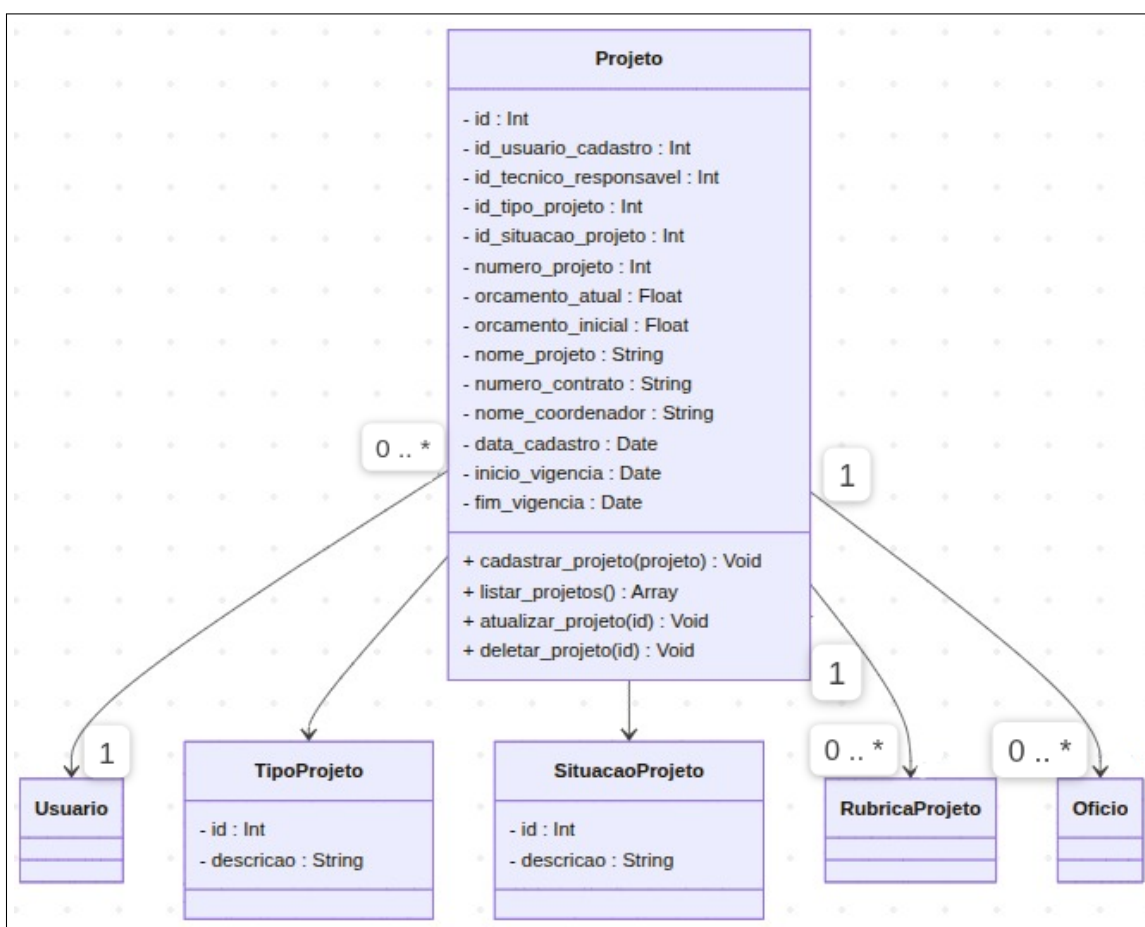
Caso existam vínculos com projetos (ou seja, se o resultado da consulta retornar um ou mais registros), o sistema impede a exclusão e retorna com uma mensagem de erro, informando que a rubrica não pode ser removida devido à sua associação com um ou mais projetos.

Se a rubrica não estiver vinculada a nenhum projeto, o método prossegue com a exclusão, localizando o registro correspondente por meio do método `findOrFail()` e executando o comando `delete()`. Após a operação bem-sucedida, o sistema retorna exibindo uma mensagem de confirmação da exclusão.

5.3.4 Modelagem do diagrama de classe de Projeto

O diagrama de classes apresentado na Figura 33 representa a estrutura do módulo de gerenciamento de projetos e sua relação com os demais componentes.

Figura 33 – Classe de Projeto.



Fonte: Imagem elaborada pelo autor.

A classe principal, *Projeto*, contém atributos que armazenam informações essenciais como identificadores de vínculo (*id_usuario_cadastro*, *id_tecnico_responsavel*,

id_tipo_projeto, *id_situacao_projeto*), além de dados administrativos e financeiros, como número do projeto, orçamento inicial e atual, nome do coordenador, vigência e datas de cadastro. Os métodos *cadastrar_projeto()*, *listar_projetos()*, *atualizar_projeto()* e *deletar_projeto()* compõem as operações básicas de CRUD, permitindo o gerenciamento das informações de projeto.

A classe *Usuario* mantém uma relação de um para muitos com *Projeto*, indicando que um usuário pode ser responsável pelo cadastro de vários projetos ou ser o técnico controlador orçamentários de vários projetos.

A classe *TipoProjeto* relaciona-se de forma semelhante, permitindo classificar os projetos de acordo com sua natureza. Já a classe *SituacaoProjeto* identifica o status em que o projeto se encontra, como em andamento, finalizado ou cancelado.

A classe *RubricaProjeto* estabelece uma relação de dependência com *Projeto*, representando os vínculos orçamentários entre projetos e rubricas financeiras. Por fim, a classe *Oficio* está associada a *Projeto* em uma relação de um para muitos, indicando que cada projeto pode possuir diversos ofícios vinculados.

5.3.5 Cadastro de projeto

Esta funcionalidade está disponível no sistema pelo caminho PROJETOS > Cadastrar Projeto e é acessível para todos os perfis.

A tela de cadastro de projeto (Figura 34) tem como objetivo permitir o registro informações relacionadas aos projetos, cumprindo com o requisito RF08. Nessa interface, o usuário pode inserir os dados do projeto, como nome, número, número do contrato, datas de início e término da vigência, além de selecionar o tipo de projeto, o coordenador responsável e o técnico de referência.

Figura 34 – Tela de cadastro de projeto.

Fonte: Imagem elaborada pelo autor.

Na seção subsequente, o sistema disponibiliza um painel para a associação de rubricas e itens orçamentários ao projeto, permitindo que o usuário selecione uma rubrica cadastrada previamente, informe a quantidade, o valor unitário e a descrição do item, compondo assim o plano de aplicação orçamentária do projeto. Cada item adicionado é exibido em uma tabela inferior, que apresenta as colunas de rubrica, item, quantidade, valor unitário, valor total e ações, como edição ou exclusão.

Ao final do formulário, há uma opção que define o nível de cadastramento do projeto, possibilitando ao usuário marcar se o cadastro está completo — concluindo o processo — ou se está em elaboração, permitindo o salvamento parcial das informações para finalização posterior, como descrito no requisito RF09.

5.3.6 Cadastro de projeto – Funcionalidade de salvamento parcial

A funcionalidade de salvamento parcial (Figura 35) apresenta uma extensão da função da interface anterior (cadastro de projetos), permitindo ao usuário registrar informações de forma progressiva e salvar parcialmente os dados inseridos. Essa funcionalidade foi desenvolvida com o objetivo de oferecer maior flexibilidade durante o processo de inserção de informações complexas, especialmente em projetos que exigem o preenchimento de um grande volume de dados.

Figura 35 – Tela de cadastro de projeto - Funcionalidade de salvamento parcial.

Coordenador do Projeto: Informe o nome do Coordenador do Projeto...

Técnico Responsável: Selecione...

Adicionar Rubricas e Itens do Projeto

Adicionar Rubrica: Selecione...

Descrição do Item: Informe a Descrição do Item...

Quantidade do Item: Informe a quantidade de Itens...

Valor Unitário do Item (R\$): 0,00

+ Adicionar

Rubrica	Item	Qtd.	Valor Unit.	Total	Ações
30 - MATERIAL DE CONSUMO (NACIONAL)	TESTE ITEM DEMONSTRATIVO	2	R\$ 100,00	R\$ 200,00	[Editar] [Excluir]

Orçamento total do Projeto: R\$ 200,00

Em que nível cadastral está o Projeto?

☐ Cadastro completo. Finalizar.

☒ Em Elaboração. Continuar o cadastro mais tarde.

[Voltar](#) [Manter Projeto em Elaboração](#)

Fonte: Imagem elaborada pelo autor.

Na parte inferior da tela, o sistema apresenta a seguinte pergunta: “Em qual nível de cadastro o projeto se encontra?”, seguida das opções: “Cadastro concluído. Finalizar” — que habilita o botão azul “Cadastrar Projeto” e realiza o salvamento definitivo do projeto na base de dados — e “Em elaboração. Continuar o cadastro mais tarde”, que habilita o botão verde “Manter o Projeto em Elaboração”.

Essa segunda opção permite salvar o estado atual do projeto sem a necessidade de concluir todas as etapas de cadastro, sendo útil em situações em que o usuário ainda não possui todos os dados necessários. Assim, as informações já registradas são preservadas no banco de dados e podem ser retomadas posteriormente para finalização do cadastro.

Foi implementada uma regra segundo a qual, enquanto o projeto estiver em fase de elaboração, não será possível o recebimento de ofícios de remanejamento.

5.3.7 Listagem e gerenciamento de projetos

A Figura 36 apresenta a tela principal do sistema, exibida após a autenticação bem-sucedida do usuário. Nessa interface, é possível visualizar a lista de projetos cadastrados, bem como realizar ações relacionadas a cada um deles.

Figura 36 – Tela principal do sistema - Listagem de projetos.

Nº	NOME PROJETO	COORDENADOR	TIPO	CONTROLADOR ORC.	OPÇÕES
372022	SE DE BALANÇOS	BEZERRA	C	MACEDO DA ROCHA	[Visualizar] [Editar] [Imprimir] [Excluir]
262023	ANATOMIA	NASCIMENTO JUNIOR	A	ARAUJO FILHO	[Visualizar] [Editar] [Imprimir] [Excluir]
182023	IZAÇÃO EM ENFERMAGEM NEONATAL E PEDIÁTRICA	NASCIMENTO KLUCZYNIK	A		[Visualizar] [Editar] [Imprimir] [Excluir]
432023	AValiação DE BLOCOS FUNCIONAIS COMPATÍVEIS COM A NORMA IEC 61499	MAITELLI	D	PONTES REGO SARMENTO	[Visualizar] [Editar] [Imprimir] [Excluir]
1272022	SA APLICADO À COOPERAÇÃO INTERNACIONAL: ECONOMIA DA SAÚDE NA PERSPECTIVA DE AVALIAÇÃO DAS POLÍTICAS PÚBLICAS	ALEXSANDRO DE MEDEIROS VALENTIM	B	RODRIGUES DE CARVALHO DOS SANTOS	[Visualizar] [Editar] [Imprimir] [Excluir]

Fonte: Imagem elaborada pelo autor.

No menu lateral esquerdo, o sistema disponibiliza opções de navegação para os módulos *Projetos*, *Ofícios*, *Rubricas*, *Usuários*, *Resolução*, além das seções *Reportar Erro* e *Sobre*. Cada módulo direciona o usuário a funcionalidades do sistema.

Na área central da tela, encontra-se o título “Lista de Projetos” e o botão “Cadastrar novo Projeto”, que permite a inclusão de novos registros. Logo abaixo, há um campo de busca, que possibilita a filtragem de projetos por nome ou número.

Observa-se que alguns dados na imagem foram ocultados por meio de tarjas. Essa medida foi adotada por se tratarem de informações reais, uma vez que a captura da tela ocorreu durante uma reunião diária envolvendo colaboradores da fundação.

A listagem é apresentada em formato de tabela, contendo as colunas:

- Nº: número identificador do projeto;

- Nome projeto: nome completo do projeto;
- Coordenador: responsável principal pelo projeto;
- Tipo: categoria ou classificação do projeto (A, B, C ou D);
- Controlador Orc.: técnico vinculado ao gerenciamento do projeto;
- Opções: conjunto de botões:
 - Visualizar detalhes do projeto (ícone de olho);
 - Editar informações do projeto (ícone de lápis);
 - Cadastrar ofício ao projeto (ícone de documento);
 - Excluir projeto (ícone de lixeira);

No canto superior direito da interface, é exibido o nome do usuário autenticado, acompanhado de sua função no sistema (por exemplo, “ADMINISTRADOR”).

É importante destacar que esta tela corresponde à interface de um usuário administrador do sistema, o qual possui acesso completo de visualização a todas as funcionalidades disponíveis. Em contrapartida, quando o usuário logado for um Controlador de Orçamento (técnico), seu acesso será restrito, permitindo-lhe visualizar apenas os projetos que o mesmo gerencia, mantendo controle exclusivamente, conforme os requisitos RF03 e RF05.

Por sua vez, o usuário Gerente dispõe de uma visão abrangente de todos os projetos, conforme definido no requisito RF06.

A funcionalidade de exclusão de projetos possui uma característica específica, pois sua remoção implica automaticamente na exclusão dos ofícios vinculados a ele. Essa abordagem foi adotada para evitar inconsistências na base de dados, garantindo que, ao excluir um projeto, todos os ofícios relacionados também sejam removidos, preservando assim a integridade referencial do sistema.

O código a seguir (Figura 37) corresponde à funcionalidade que realiza a validação da exclusão de um projeto.

O trecho de código apresentado o mostra o método `deletar_projeto()`, responsável por executar o processo de exclusão de projetos do sistema, buscando que a operação ocorra de forma segura e consistente em relação às demais entidades relacionadas no banco de dados.

O método recebe como parâmetro o identificador do projeto (*id_projeto*) e, inicialmente, realiza uma consulta à tabela *oficios*, verificando se há algum ofício vinculado ao projeto em questão. Caso exista um ou mais registros relacionados, o sistema impede a exclusão e retorna uma mensagem de erro, informando que o projeto não pode ser removido por possuir ofícios associados.

Figura 37 – Código para deletar projeto.

```

public function deletar_projeto($id) {
    $oficioVinculado = DB::table('oficios')->where('id_projeto_destino', $id)->get();

    if(count($oficioVinculado) == 0) {
        $rubricasProjeto = RubricaProjeto::where('id_projeto', $id)->get();

        if(count($rubricasProjeto) > 0) {
            foreach($rubricasProjeto as $rp) {
                $itensRubrica = ItensRubrica::where('id_rubricas_projeto', $rp->id)->get();

                if(count($itensRubrica) > 0) {
                    foreach($itensRubrica as $ir) {
                        $itens = ItensRubrica::findOrFail($ir->id)->delete();
                    }
                }
                $rubrica = RubricaProjeto::findOrFail($rp->id)->delete();
            }
        }

        $projeto = Projeto::findOrFail($id)->delete();

        return Redirect::back()->with('message', 'Projeto deletado com sucesso!');
    }
    return Redirect::back()->withErrors(['msg' => 'Existe Ofício(s) cadastrado ao Projeto!']);
}

```

Fonte: Imagem elaborada pelo autor.

Se o projeto não estiver vinculado a nenhum ofício, o sistema prossegue verificando se há rubricas associadas ao projeto na tabela `rubricas_projeto`. Para cada rubrica encontrada, o método busca e remove os itens vinculados na tabela `itens_rubrica`, garantindo que todos os registros dependentes sejam excluídos antes da exclusão da rubrica. Após isso, cada rubrica associada é também removida, concluindo o processo de limpeza das relações hierárquicas.

Por fim, o próprio projeto é localizado por meio do método `findOrFail()` e excluído com o comando `delete()`. Em caso de sucesso, o sistema redireciona o usuário à página anterior, exibindo a mensagem de confirmação da exclusão.

5.4 Sprint 8 - Implementação do módulo de cadastro de ofícios e relatórios.

A oitava *sprint* concentrou-se na implementação das funcionalidades de ofícios e a parte de relatório de ofícios do sistema. Nesta etapa, foram desenvolvidas as interfaces e lógicas de controle responsáveis por permitir o registro das informações relacionadas à ofícios e aos relatórios de ofícios.

5.4.1 Modelagem do diagrama de ofícios

O diagrama de classes apresentado na Figura 38 representa a estrutura de modelagem da entidade *Oficio* e suas relações com outras classes do sistema. Esse modelo tem como objetivo organizar as informações referentes aos ofícios vinculados aos projetos geridos pela fundação.

Figura 38 – Classe de ofício.



Fonte: Imagem elaborada pelo autor.

A classe *Ofício* contém atributos que armazenam dados essenciais como identificadores do projeto de destino, do usuário responsável e da situação do ofício, além de informações administrativas como número, observação e data de recebimento. Essa estrutura possibilita o registro detalhado e o acompanhamento do fluxo de documentos oficiais associados aos projetos.

A relação entre *Projeto* e *Ofício* é de um para muitos, permitindo que um mesmo projeto possua vários ofícios vinculados.

A classe *Usuario* está associada a *Ofício* por meio de uma relação de um para muitos, indicando que um usuário pode ser responsável por diversos ofícios cadastrados.

Por fim, a classe *SituacaoOficio* define o estado atual do ofício (como “Aprovado”, “Negado”, “Pendente de Aprovação” ou “Parcialmente Aprovado”) e relaciona-se com *Ofício*.

5.4.2 Cadastro de ofício

O cadastro de ofícios é realizado em projetos previamente registrados no sistema. Na tela inicial mostrada anteriormente (Figura 36), a coluna de opções apresenta

um botão verde que permite acessar a funcionalidade de cadastro de ofícios vinculados ao projeto selecionado. A tela correspondente a esse processo pode ser observada na Figura 39.

Figura 39 – Tela de cadastro de ofício.

Fonte: Imagem elaborada pelo autor.

A tela de cadastro de ofício foi desenvolvida para permitir o registro de novos ofícios vinculados a projetos já existentes no sistema. Seu objetivo é efetivar alterações orçamentárias e administrativas realizadas no projeto.

Na parte superior da interface, o sistema exibe o nome do projeto selecionado, permitindo ao usuário confirmar o contexto do cadastro. Em seguida, são apresentados os campos “*Plano de Aplicação Inicial*”, que disponibiliza o botão “*Plano de Aplicação Atual do Projeto*” para consulta do plano vigente, e a opção de indicar se o ofício consiste na adição de novas rubricas e/ou itens, através dos botões de seleção “Consiste” e “Não Consiste”.

Abaixo, encontram-se os campos “*Número do Ofício*” e “*Data do Ofício*”, que são de preenchimento obrigatório e garantem a correta identificação e versionamento dos documentos registrados.

A interface é composta ainda por uma série de seções expansíveis, destacadas em verde, que representam os diferentes tipos de ofícios. Entre essas opções estão: “*Aditivo de Vigência*”, “*Alteração de Item*”, “*Aditivo de Orçamento*”, “*Uso de Rendimento*”, “*Remanejamento de Item*” e “*Remanejamento de Rendimento*”. Cada uma dessas seções permite o detalhamento de informações específicas conforme o tipo de modificação selecionada.

Na parte inferior da tela, são exibidos dois botões de ação: o botão laranja “*Voltar*”, que retorna à lista de projetos, e o botão azul “*Cadastrar Ofício*”, responsável por salvar as informações inseridas e registrar o novo ofício na base de dados.

5.4.3 Cadastro de ofício - Plano de aplicação atual do projeto

A tela apresentada na Figura 40 corresponde à interface de consulta do plano de aplicação atual do projeto, acessível a partir do módulo de cadastro de ofício. Essa funcionalidade tem como objetivo exibir, de forma mais organizada a estrutura orçamentária vigente do projeto selecionado, permitindo ao usuário verificar as rubricas e itens cadastrados antes de realizar o cadastro de ofícios.

Figura 40 – Tela de cadastro de ofício - Plano de aplicação atual do projeto.

Tipo	Descrição	Quantidade	Valor Unitário	Valor Total
Rubrica	30 - MATERIAL DE CONSUMO (NACIONAL)			
Item	MATERIAL DE LIMPEZA	1	300,00	300,00
Item	MATERIAIS DESCARTAVEIS	1	3.000,00	3.000,00

Tipo	Descrição	Quantidade	Valor Unitário	Valor Total
Rubrica	39 - OUTROS SERVICOS TERCEIROS - PJ			
Item	SERVIÇOS GRÁFICOS	101	133.247,03	1.345.795,00

Tipo	Descrição	Quantidade	Valor Unitário	Valor Total
Rubrica	52 - EQUIPAMENTOS E MATERIAL PERMANENTE (NACIONAL)			
Item	TV	1	9.000,00	9.000,00
Item	MESA	2	2.500,00	5.000,00

Fonte: Imagem elaborada pelo autor.

A interface é apresentada em formato de janela modal sobreposta à tela de cadastro de ofícios, o que permite ao usuário visualizar as informações sem perder o contexto do cadastro em andamento.

Na tabela exibida, os dados são organizados em colunas que representam o *Tipo* (Rubrica ou Item), a *Descrição*, a *Quantidade*, o *Valor Unitário* e o *Valor Total*. As rubricas são apresentadas como linhas principais, seguidas dos respectivos itens vinculados, que representam as despesas ou recursos específicos previstos no orçamento. Essa hierarquia visual foi pensada buscando facilitar a compreensão da composição orçamentária e o relacionamento entre as rubricas e seus itens.

5.4.4 Modelagem do diagrama de Aditivo de vigência

O diagrama de classes apresentado na Figura 41 representa a estrutura de modelagem da entidade de *AditivoVigencia* e suas relações no sistema. Essa classe tem como objetivo gerenciar os aditivos de vigência vinculados a um ofício, permitindo o controle das alterações de prazo dos projetos.

Figura 41 – Classe de aditivo de vigência.



Fonte: Imagem elaborada pelo autor.

A classe *AditivoVigencia* armazena informações como o identificador do ofício ao qual está associado, a situação atual do aditivo (“Aprovado” ou “Negado”), as datas de encerramento do projeto antes e após a prorrogação, além de um campo para justificativa da solicitação. Além dos atributos, a classe possui métodos que possibilitam cadastrar, listar, atualizar e excluir registros de aditivos de vigência, garantindo a manutenção eficiente dos dados.

A relação entre *Oficio* e *AditivoVigencia* é de um para muitos, indicando que um ofício pode conter diversos aditivos vinculados a ele.

5.4.5 Cadastro de ofício - Aditivo de vigência

A Figura 42 apresenta a interface do sistema responsável pelo registro de ofícios que envolvem a prorrogação da vigência de um projeto. Essa funcionalidade permite que o usuário cadastre aditivos de vigência vinculados ao projeto.

Figura 42 – Tela de cadastro de ofício - Aditivo de vigência.

Fonte: Imagem elaborada pelo autor.

A seção destacada em verde corresponde ao módulo de *Aditivo de Vigência*, onde o usuário pode informar a *Nova data final de vigência do projeto* e inserir uma *Justificativa* detalhando os motivos da prorrogação. O campo “*Data Final Vigência do Projeto*” apresenta a data da vigência atual do projeto, enquanto o campo “*Nova Data Final Vigência do Projeto*” permite definir o novo prazo de término.

Na parte inferior da tela, o sistema disponibiliza os botões “*Voltar*” e “*Cadastrar Ofício*”, que permitem respectivamente retornar à tela anterior ou salvar as informações do aditivo na base de dados.

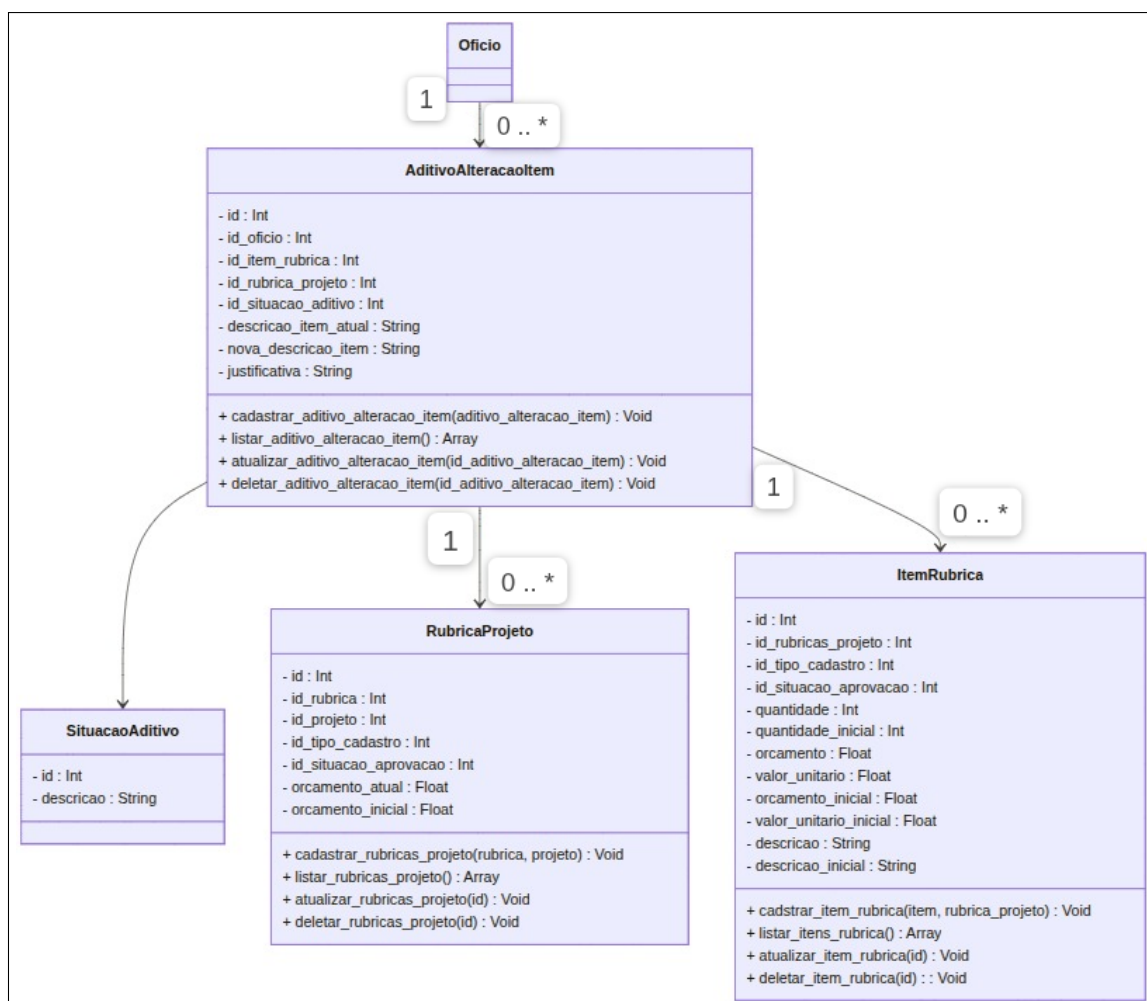
5.4.6 Modelagem do diagrama de Aditivo de alteração de item

O diagrama de classes apresentado na Figura 43 representa a modelagem da entidade *AditivoAlteracaoItem* e suas relações. Essa estrutura tem como objetivo gerenciar as solicitações de alteração em itens vinculados às rubricas de um projeto.

A classe *AditivoAlteracaoItem* contém atributos que registram informações como o ofício de origem, o item de rubrica afetado, a rubrica de projeto relacionada, a situação do aditivo, além das descrições anterior e nova do item e a justificativa para a alteração. Ela também inclui métodos para cadastrar, listar, atualizar e excluir aditivos, garantindo a manipulação e o controle eficiente desses registros no sistema.

A relação entre *Oficio* e *AditivoAlteracaoItem* é de um para muitos, indicando que um ofício pode conter diversas solicitações de alteração de itens. A classe *RubricaProjeto* também se relaciona com *AditivoAlteracaoItem* por meio de uma associação de um para muitos, pois cada rubrica pode possuir múltiplos aditivos vinculados. Já a

Figura 43 – Tela de cadastro de ofício - Aditivo de alteração de item.



Fonte: Imagem elaborada pelo autor.

classe *ItemRubrica* representa os itens que compõem as rubricas do projeto, estando igualmente associada a *AditivoAlteracaoItem* em uma relação de um para muitos.

5.4.7 Cadastro de ofício - Aditivo de alteração de item

A Figura 44 apresenta a interface destinada ao cadastro de ofícios relacionados à alteração de itens vinculados a projetos geridos pela fundação. Essa funcionalidade tem como objetivo permitir as modificações nas descrições de itens pertencentes às rubricas orçamentárias de um projeto.

A seção intitulada “*Alteração de Item*”, destacada em verde, concentra os campos principais desta operação. O usuário deve inicialmente selecionar a *Rubrica do Projeto* e, em seguida, escolher o *Item da Rubrica* que sofrerá alteração. Após a seleção, é possível inserir uma *nova descrição para o item*, campo que registra a atualização textual proposta. Ao clicar no botão “+ Adicionar”, a alteração é listada na tabela exibida logo abaixo, que contém as colunas “*Número - Rubrica*”, “*Descrição*

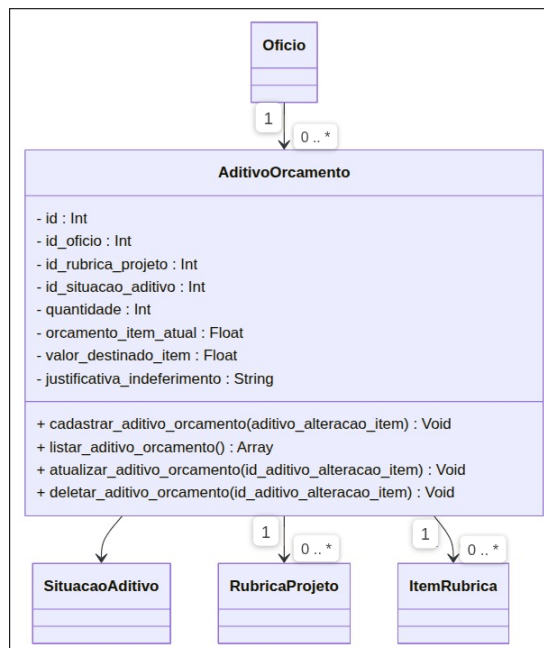
Figura 44 – Tela de cadastro de ofício - Aditivo de alteração de item.

Fonte: Imagem elaborada pelo autor.

Atual do Item”, “*Descrição Proposta*” e “*Ações*”. Essa estrutura facilita a visualização das modificações pendentes antes do envio.

5.4.8 Modelagem do diagrama de Aditivo de orçamento

Figura 45 – Classe de aditivo de orçamento.



Fonte: Imagem elaborada pelo autor.

O diagrama de classes apresentado na Figura 45 representa a estrutura da entidade *AditivoOrçamento* e suas relações dentro do sistema.

A classe *AditivoOrçamento* possui atributos que armazenam informações essenciais, como o identificador do ofício associado, a rubrica de projeto, a situação do aditivo, a quantidade de recursos, o orçamento atual do item, o valor destinado após a alteração e eventuais justificativas para indeferimentos.

A relação entre *Ofício* e *AditivoOrçamento* é de um para muitos, indicando que um ofício pode conter diversas solicitações de aditivos orçamentários. Já a associação entre *AditivoOrçamento* e *RubricaProjeto* é de um para muitos, uma vez que cada rubrica de projeto pode ter múltiplos aditivos vinculados. A classe *ItemRubrica*, que representa os itens específicos dentro das rubricas, também se relaciona com *AditivoOrçamento* em uma relação de um para muitos.

5.4.9 Cadastro de ofício - Aditivo de orçamento

A Figura 46 apresenta a interface destinada ao cadastro de ofícios que envolvem a criação de um aditivo de orçamento em um projeto já registrado no sistema. Essa funcionalidade tem como finalidade registrar formalmente acréscimos ou ajustes no valor orçamentário de itens vinculados a rubricas.

Figura 46 – Tela de cadastro de ofício - Aditivo de orçamento.

Fonte: Imagem elaborada pelo autor.

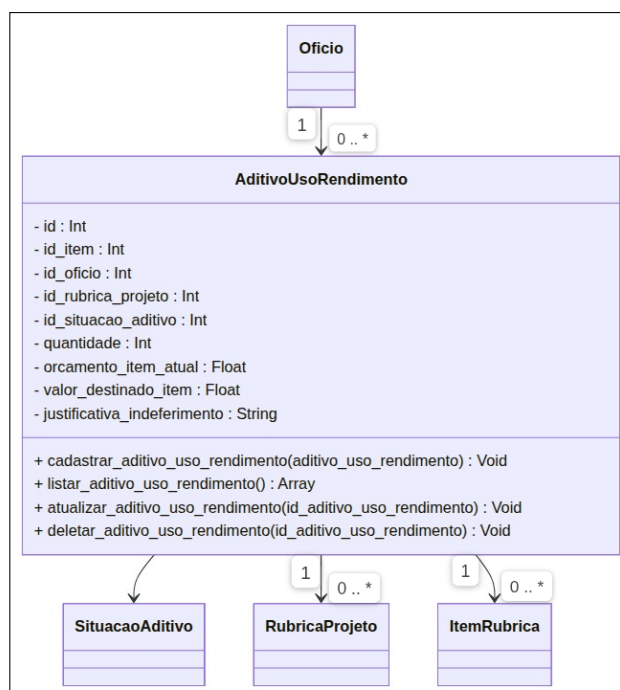
A seção destacada em verde sob o título “Aditivo de Orçamento” apresenta os campos principais para o lançamento das alterações orçamentárias. O usuário deve selecionar a rubrica do Projeto e, posteriormente, o item da rubrica que receberá o

aditivo. Em seguida, é necessário preencher os campos de quantidade e valor orçado, que representam o montante e o custo associado ao item adicional ou reajustado. Após o preenchimento, o botão “+ Adicionar” permite incluir a informação em uma tabela exibida logo abaixo, onde são mostradas as colunas “Número - Rubrica”, “Item”, “Qtd.”, “Valor Orçado” e “Ações”. Essa estrutura facilita o controle visual dos itens adicionados antes do salvamento definitivo.

5.4.10 Modelagem do diagrama de Aditivo de uso de rendimento

O diagrama de classes apresentado na Figura 47 representa a estrutura da entidade *AditivoUsoRendimento* e suas relações com outras classes do sistema.

Figura 47 – Classe de aditivo de uso de rendimento.



Fonte: Imagem elaborada pelo autor.

A classe *AditivoUsoRendimento* armazena informações como o identificador do item, do ofício e da rubrica de projeto associados, a situação atual do aditivo, a quantidade de recursos, o orçamento do item antes da operação, o valor destinado e uma possível justificativa em casos de indeferimento.

No que diz respeito aos relacionamentos, o Ofício está ligado à classe *AditivoUsoRendimento* em uma relação de um para muitos, indicando que um ofício pode estar associado a diversas solicitações de uso de rendimentos.

A entidade *RubricaProjeto* está associada a *AditivoUsoRendimento* também em uma relação de um para muitos, permitindo que diferentes aditivos estejam vinculados a uma mesma rubrica orçamentária do projeto. Já a classe *ItemRubrica*

representa os itens financeiros detalhados dentro de uma rubrica e se relaciona com *AditivoUsoRendimento*.

5.4.11 Cadastro de ofício - Aditivo de uso de rendimento

Figura 48 – Tela de cadastro de ofício - Uso de rendimento.

A interface de cadastro de ofício - Uso de rendimento apresenta o seguinte layout:

- Menu Lateral:**
 - REPORTAR ERRO (botão vermelho)
 - SOBRE... (botão verde)
 - DETALHES (botão verde)
- Seções de Ação:**
 - ADITIVO DE VIGÊNCIA
 - ALTERAÇÃO DE ITEM
 - ADITIVO DE ORÇAMENTO
 - USO DE RENDIMENTO
 - REMANEJAMENTO DE ITEM
 - REMANEJAMENTO DE RENDIMENTO
- Formulário de Cadastro:**
 - Rubricas do Projeto:** Seleção de rubrica.
 - Itens da Rubrica:** Seleção de item da rubrica.
 - Quantidade:** Campo para informar a quantidade a ser rer.
 - Valor Rendido (R\$):** Campo para informar o valor rendido, atualmente 0,00.
 - + Adicionar:** Botão para adicionar o registro.
- Tabela de Registro:**

Rubrica	Item	Quantidade	Valor Rendido	Ações
---------	------	------------	---------------	-------
- Botões de Navegação:**
 - Voltar (botão amarelo)
 - Cadastrar Ofício (botão azul)

Fonte: Imagem elaborada pelo autor.

A Figura 48 apresenta a interface destinada ao registro de ofícios relacionados ao uso de rendimentos em projetos cadastrados no sistema. Essa funcionalidade tem como propósito permitir a aplicação de valores provenientes de rendimentos financeiros gerados durante a execução do projeto, garantindo que os recursos adicionais sejam corretamente vinculados às rubricas.

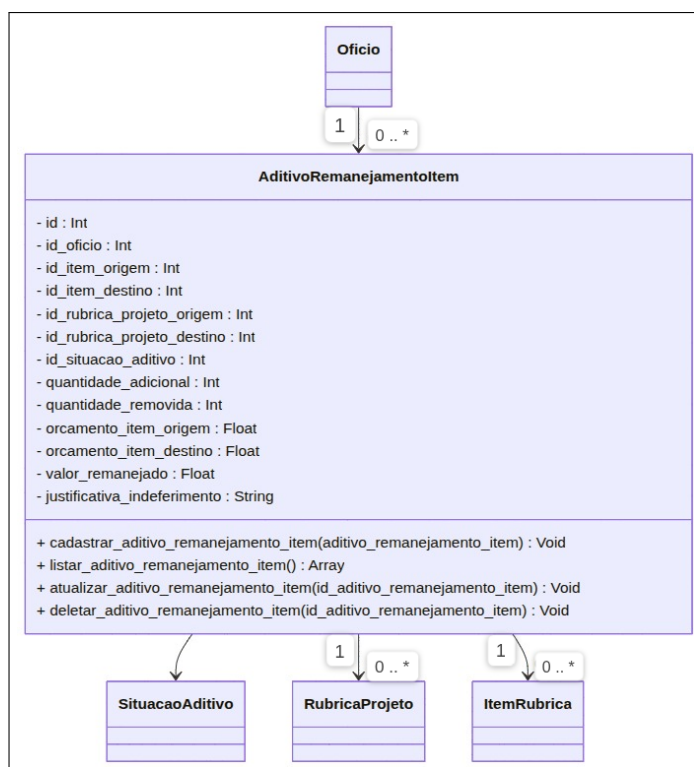
Dentro da seção de uso de rendimento, o usuário deve selecionar a rubrica do projeto e o item da rubrica que receberá a aplicação do valor rendido. Em seguida, é necessário informar a quantidade e o valor rendido a ser aplicado. Após o preenchimento dos campos, o botão “+ Adicionar” permite incluir o registro na tabela exibida logo abaixo. Essa tabela apresenta os dados organizados nas colunas “*Rubrica*”, “*Item*”, “*Quantidade*”, “*Valor Rendido*” e “*Ações*”, permitindo que o usuário visualize, edite ou remova lançamentos antes da conclusão do cadastro.

5.4.12 Modelagem do diagrama de aditivo de remanejamento de item

O diagrama de classes apresentado na Figura 49 representa a estrutura da entidade *AditivoRemanejamentoItem*, responsável por gerenciar as movimentações de recursos entre itens orçamentários dentro de um mesmo projeto por meio de um Ofício.

A classe *AditivoRemanejamentoItem* contém atributos que identificam tanto o item e a rubrica de origem quanto o item e a rubrica de destino, além de informações

Figura 49 – Classe de ofício de remanejamento de item.



Fonte: Imagem elaborada pelo autor.

referentes à quantidade removida e adicionada, aos valores de orçamento antes de efetuar o remanejamento e ao valor total remanejado. Também armazena uma justificativa para indeferimentos, caso o processo não seja aprovado.

No que diz respeito aos relacionamentos, a entidade *Ofício* possui uma relação de um para muitos com *AditivoRemanejamentoItem*, uma vez que um ofício pode estar vinculado a diversas solicitações de remanejamento de itens.

Já as entidades *RubricaProjeto* e *ItemRubrica* mantêm relações de um para muitos com *AditivoRemanejamentoItem*, indicando que várias solicitações de remanejamento podem estar associadas a uma mesma rubrica ou item orçamentário, tanto de origem quanto de destino.

5.4.13 Cadastro de ofício - Aditivo de remanejamento de item

A tela apresentada na Figura 50 representa a seção do sistema responsável pelo cadastro de remanejamentos de itens dentro de um ofício. Essa funcionalidade permite que o usuário realize a redistribuição de recursos entre rubricas e itens do mesmo projeto.

Os campos de entrada incluem a seleção da rubrica de origem e da rubrica de destino, seguidas das respectivas opções de item de origem e item de destino. Em seguida, é possível informar as quantidades a serem retiradas e adicionadas, bem como o valor total do remanejamento em reais. O botão azul “+ Adicionar” permite

Figura 50 – Tela de cadastro de ofício - Remanejamento de item.

Fonte: Imagem elaborada pelo autor.

incluir a operação na tabela de controle exibida logo abaixo, onde as informações são organizadas por colunas — apresentando a rubrica e item de origem, a rubrica e item de destino, as quantidades movimentadas, o valor total e as ações disponíveis.

5.4.14 Modelagem do diagrama de Aditivo de remanejamento de rendimento

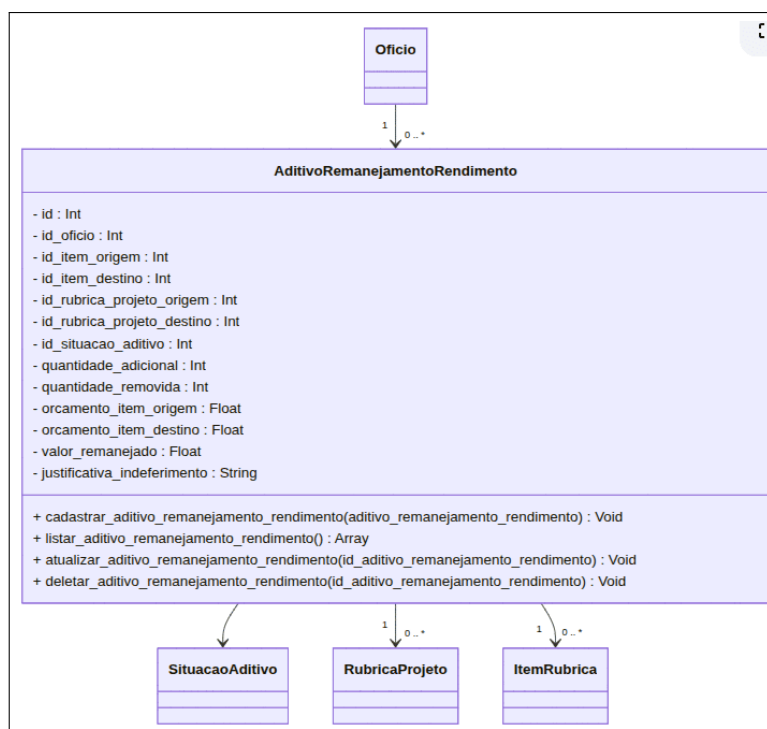
O diagrama de classes apresentado na Figura 51 representa a estrutura da entidade *AditivoRemanejamentoRendimento*, responsável por controlar as operações de remanejamento de recursos provenientes de rendimentos financeiros entre diferentes itens e rubricas de um projeto, formalizadas por meio de um Ofício.

A classe *AditivoRemanejamentoRendimento* possui atributos que armazenam informações essenciais, como identificadores de origem e destino de itens e rubricas, quantidades removidas e adicionadas, valores orçamentários antes e depois do remanejamento e o valor total transferido. Também inclui um campo de justificativa para casos de indeferimento.

Em relação aos relacionamentos, a entidade Ofício se conecta à classe *AditivoRemanejamentoRendimento* em uma relação de um para muitos, pois um mesmo ofício pode conter vários aditivos relacionados a rendimentos.

As entidades *RubricaProjeto* e *ItemRubrica* também estão associadas por meio de relações de um para muitos, indicando que uma mesma rubrica ou item pode estar vinculado a diversos aditivos de remanejamento de rendimento.

Figura 51 – Classe de ofício de remanejamento de rendimento.



Fonte: Imagem elaborada pelo autor.

5.4.15 Cadastro de ofício - Remanejamento de rendimento

Figura 52 – Tela de cadastro de ofício - Remanejamento de rendimento.

The screenshot shows a web interface for recording revenue reallocation. On the left is a dark sidebar with links: 'REPORTAR ERRO' (with an error icon), 'ERRO', 'SOBRE...', and 'DETALHES'. The main area has a list of green buttons: 'ADITIVO DE VIGÊNCIA', 'ALTERAÇÃO DE ITEM', 'ADITIVO DE ORÇAMENTO', 'USO DE RENDIMENTO', 'REMANEJAMENTO DE ITEM', and 'REMANEJAMENTO DE RENDIMENTO'. Below these are input fields for 'Rubrica de Origem', 'Item de Origem', 'Rubrica de Destino', and 'Item Destino', each with a 'Selecione...' dropdown. There are also fields for 'Retirar quantidade do Item Origem' and 'Adicionar quantidade do Item Destino' with 'Informe a quantidade..' prompts. A 'Valor TOTAL do Remanejamento (R\$)' field shows '0,00' and a '+ Adicionar' button. At the bottom is a table with columns: 'Rubrica Origem', 'Item Origem', 'Rubrica Destino', 'Item Destino', 'Retirar Qtd. Ori.', 'Adiciona Qtd. Des.', 'Valor', and 'Ações'.

Fonte: Imagem elaborada pelo autor.

A Figura 52 apresenta a interface destinada ao registro de remanejamentos de rendimento entre rubricas e itens de um projeto. Essa funcionalidade possibilita

o controle e a redistribuição de valores provenientes de rendimentos financeiros já aplicados, permitindo que sejam realocados conforme a necessidade do projeto.

Na seção “*Remanejamento de Rendimento*”, o usuário deve selecionar a rubrica de origem e o item de origem de onde o valor será retirado, bem como a rubrica de destino e o item de destino que receberão o montante remanejado. Os campos seguintes permitem informar a quantidade a ser retirada do item de origem e a quantidade a ser adicionada ao item de destino, além do valor total do remanejamento.

Após o preenchimento dos dados, o botão “+ Adicionar” insere o registro na tabela localizada na parte inferior da tela, onde são exibidas as colunas “*Rubrica Origem*”, “*Item Origem*”, “*Rubrica Destino*”, “*Item Destino*”, “*Retirar Qtd. Ori.*”, “*Adiciona Qtd. Des.*”, “*Valor*” e “*Ações*”. Essa tabela fornece uma visão consolidada das movimentações realizadas, permitindo que o usuário revise ou exclua lançamentos antes de concluir o cadastro do ofício.

5.4.16 Listagem de ofícios

A tela de Listagem de Ofícios do Projeto (Figura 53) disponível pelo caminho OFÍCIOS > Lista de ofícios no sistema, apresenta todos os ofícios vinculados ao projeto selecionado, permitindo ao usuário visualizar rapidamente o histórico de solicitações realizadas, bem como o status atual de cada documento.

Figura 53 – Tela de listagem de ofícios.

Nº	Data Ofício	Projeto	Tipo(s)	Status	Ações
01	21/09/2023	[REDACTED] BALANÇOS	RI	Aprovado	[View] [Download]
150	20/09/2023	[REDACTED] NACIONAL: ECONOMIA DA SAUDE NA PERSPECTIVA DE AVALIAÇÃO DAS POLITICAS PÚBLICAS	RI	Aprovado	[View] [Download]
20.2023	20/09/2023	[REDACTED] EM ENFERMAGEM NEONATAL E PEDIÁTRICA	RI	Aprovado	[View] [Download]
009	05/09/2023	[REDACTED] BALANÇOS	UR	Aprovado	[View] [Download]
005	20/09/2023	[REDACTED] BALANÇOS	RI	Aprovado	[View] [Download]
032023	06/09/2023	[REDACTED] BALANÇOS	RI	Aprovado	[View] [Download]
001/2022	07/07/2022	[REDACTED] BALANÇOS	RI	Aprovado	[View] [Download]
12023	06/09/2023	[REDACTED] MIA	RI	Parcialmente Aprovado	[View] [Download]
22023	06/09/2023	[REDACTED] RMAÇÃO PARA O CENTRO DE REABILITAÇÃO INFANTIL E ADULTO - CRI/CRA, POLICLÍNICA REGIONAL DE CAICÓ, SAD E NO HEMONORTE DO ESTADO DO RIO GRANDE DO NORTE.	RI	Pendente de Aprovação	[View] [Download] [Edit] [Refresh]

Fonte: Imagem elaborada pelo autor.

Na parte superior da tela, são exibidos o número e o título do projeto. Logo abaixo, há um botão para acesso ao Plano Inicial do Projeto, permitindo a consulta rápida dessa informação sempre que necessário.

A listagem é apresentada em formato de tabela, contendo as seguintes colunas:

- Nº: número identificador do ofício.
- Data Ofício: data de emissão do documento.
- Tipo(s): categoria ou finalidade do ofício, como pendente de aprovação, negado ou aprovado.
- Status: situação do remanejamento relacionado ao ofício (ex.: RI – Remanejamento de Item).
- Ações: conjunto de ícones que possibilitam visualizar o ofício ou baixar o arquivo correspondente

Cada linha representa um ofício associado ao projeto, permitindo ao usuário acompanhar o andamento e o resultado das solicitações realizadas. Na lateral direita da tabela, encontra-se uma legenda, que explica o significado das siglas dos tipos de ofícios, como AV (Aditivo de Vigência), AI (Alteração de Item), AO (Aditivo de Orçamento) e RR (Remanejamento de Rendimento), entre outras.

5.4.17 Visualizar ofício

A tela de Visualizar Detalhes do Ofício (Figura 54) exibe todas as informações relevantes sobre um ofício específico, permitindo ao usuário acompanhar de forma completa os dados cadastrais, o contexto do documento e os remanejamentos associados.

Figura 54 – Tela de visualização de ofício.

REMANEJAMENTOS ☰ MARCOS ANDRÉ AZEVEDO DE ASSIS Administrador

OFÍCIOS > Lista de Ofícios > Visualizar Detalhes do Ofício

Detalhes do Ofício: Nº 22023

Ofício Referente ao Projeto
 [REDACTED] INFORMAÇÃO PARA O CENTRO DE REABILITAÇÃO INFANTIL E ADULTO CRI/CRA, POLICLÍNICA REGIONAL DE CAICÓ, SAD E NO HEMONORTE DO ESTADO DO RIO GRANDE DO NORTE.

Número do Projeto
1362022

Data Ofício
06/09/2023

Status do Ofício
Pendente de Aprovação

Usuário Responsável pela Operação
[REDACTED] JO FILHO

Remanejamento de Item

Rubrica Origem	Item Origem	Rubrica Destino	Item Destino	Retirar Qtd. Ori.	Adiciona Qtd. Des.	Valor	Status	Justificativa
18 - AUXÍLIO FINANCEIRO A ESTUDANTES	JOÃO VICTOR SILVA DE OLIVEIRA	18 - AUXÍLIO FINANCEIRO A ESTUDANTES	[REDACTED] NASCIMENTO		1	2.500,00	Pendente de Aprovação	

[← Voltar](#) [→ Atualizar Status do Ofício](#)

Fonte: Imagem elaborada pelo autor.

Na parte superior da interface, é apresentado o número do ofício, facilitando sua identificação dentro do sistema. Em seguida, são exibidos os dados gerais do documento, distribuídos em campos informativos:

- Ofício Referente ao Projeto: apresenta o título completo do projeto ao qual o ofício está vinculado.
- Número do Projeto: exibe o código identificador do projeto.
- Status do Ofício: informa a situação atual, como Pendente de Aprovação, Aprovado ou Negado.
- Data do Ofício: data em que o documento foi registrado.
- Usuário Responsável pela Operação: identifica o usuário que realizou o cadastro.

Abaixo dessa seção, encontra-se o bloco do tipo do ofício, no caso mostrado é de Remanejamento de Item, que detalha todas as movimentações de recursos relacionadas ao ofício. As informações são apresentadas em formato de tabela, contendo:

- Rubrica Origem e Rubrica Destino: categorias orçamentárias envolvidas.
- Item Origem e Item Destino: itens específicos que sofreram remanejamento.
- Retirar Qtd. Ori. e Adiciona Qtd. Des.: quantidades removidas e adicionadas.
- Valor: valor monetário do remanejamento.
- Status: situação da solicitação de alteração.
- Justificativa: campo destinado à motivação da operação (quando aplicável).

5.4.18 Relatório de ofícios

A tela apresentada na Figura 55 corresponde ao relatório em PDF gerado pelo sistema para detalhamento das reformulações de plano de aplicação associadas a um ofício. Esse documento tem como finalidade consolidar, em um único layout padronizado, todas as movimentações orçamentárias realizadas no âmbito de um projeto específico, seguindo o modelo institucional adotado pelas fundações.

Na parte superior do relatório, encontram-se as informações gerais do projeto e do contrato: número do contrato, nome da contratada, período de execução e valores totais dos remanejamentos classificados como Propostos, Aprovados ou de Rendimento. Esses campos são automaticamente preenchidos a partir dos dados cadastrados no sistema, garantindo integridade e alinhamento com a base de dados do projeto.

Figura 55 – Tela de relatório de ofícios.

REFORMULAÇÃO DE PLANO DE APLICAÇÃO

Contrato Nº: 10179211323
Contratada: Fundação N
Período de Execução do Contrato: 16/05/2023 a 01/08/2025

Valor Total do Remanejamento Proposto: R\$ 0,00
Valor Total do Remanejamento de Rendimentos Proposto: R\$ 0,00
Valor Total do Rendimento Proposto: R\$ 0,00

ELEMENTOS DE DESPESA	ORÇAMENTO APROVADO	RENDIMENTO APROVADO	ALTERAÇÃO ORÇAMENTÁRIA PROPOSTA (+) ou (-)			ORÇAMENTO PROPOSTO (sem rendimentos)
			Rendimento Proposto	Remanejamento de Rendimentos Aprovado	Remanejamento Proposto	
DESPESAS CORRENTES	1.363.095,00	-	-	-	-	-
30 - MATERIAL DE CONSUMO (NACIONAL)	3.540,00	-	-	-	-	-
39 - OUTROS SERVIÇOS TERCEIROS - PJ	1.345.555,00	-	-	-	-	-
52 - EQUIPAMENTOS E MATERIAL PERMANENTE (NACIONAL)	14.000,00	-	-	-	-	-

Demonstrativo dos Itens de Rubricas com Alterações Propostas

1/2

19/04/2024, 17:17

RELATÓRIO DE OFÍCIO

33.90.30 - MATERIAL DE CONSUMO (NACIONAL)							
Descrição	Orçamento Aprovado (Verba)	Rendimento Aprovado	Rendimento Proposto	Remanejamento de Rendimentos Aprovado	Remanejamento Proposto	Quantidade	Orçamento Proposto (Com rendimentos)
MATERIAL DE LIMPEZA	300,00	-	-	-	-	1	-
MATERIAIS DESCARTÁVEIS	3.000,00	-	-	-	-	1	-
Total	3.300,00	-	-	-	-	-	-

33.90.39 - OUTROS SERVIÇOS TERCEIROS - PJ							
Descrição	Orçamento Aprovado (Verba)	Rendimento Aprovado	Rendimento Proposto	Remanejamento de Rendimentos Aprovado	Remanejamento Proposto	Quantidade	Orçamento Proposto (Com rendimentos)
SERVIÇOS GRÁFICOS	1.345.795,00	-	-	-	-	101	-
Total	1.345.795,00	-	-	-	-	-	-

44.90.52 - EQUIPAMENTOS E MATERIAL PERMANENTE (NACIONAL)							
Descrição	Orçamento Aprovado (Verba)	Rendimento Aprovado	Rendimento Proposto	Remanejamento de Rendimentos Aprovado	Remanejamento Proposto	Quantidade	Orçamento Proposto (Com rendimentos)
TV	9.000,00	-	-	-	-	1	-
MESA	5.000,00	-	-	-	-	2	-
Total	14.000,00	-	-	-	-	-	-

DATA: 06/09/2023

RAUIJO FILHO
Controlador de Orçamento

Incluir, Alterar, Excluir.
Coluna de Remanejamento: Entrada, Saída.

Fonte: Imagem elaborada pelo autor.

Logo abaixo, o relatório apresenta o quadro consolidado de Elementos de Despesa, onde são dispostos os valores originalmente aprovados, os rendimentos, e as alterações orçamentárias propostas. Esse quadro sintetiza as movimentações por categoria de despesa, facilitando a visualização das rubricas impactadas pelas mudanças.

Em seguida, o documento exhibe o Demonstrativo dos Itens de Rubrica com Alterações Propostas, organização que detalha cada rubrica e seus respectivos itens de forma estruturada. Para cada categoria, o relatório apresenta:

- Descrição do item
- Orçamento Aprovado
- Rendimento Aprovado

- Remanejamento Proposto e Aprovado
- Quantidade remanejada
- Orçamento Proposto (com rendimento)

Os itens são agrupados conforme seus códigos orçamentários, tais como 33.90.30 – Material de Consumo (Nacional), 33.90.39 – Outros Serviços Terceiros – PJ, ou 44.90.52 – Equipamentos e Material Permanente. Cada tabela apresenta subtotais e totais calculados automaticamente pelo sistema, evitando divergências manuais.

Ao final do documento, o relatório inclui a data de emissão e a assinatura digital do responsável pelo orçamento, assegurando validade operacional. Também são exibidas observações informativas, como instruções de inclusão, alteração ou exclusão e a legenda referente à coluna de remanejamento.

Essa funcionalidade de geração de relatórios foi implementada para oferecer um documento formal e padronizado, compatível com as exigências administrativas da fundação, permitindo que os remanejamentos sejam apresentados de forma clara, organizada e pronta para análise ou envio oficial.

5.5 Considerações finais do capítulo

O capítulo de Desenvolvimento detalhou todas as etapas de implementação do sistema, evidenciando a evolução contínua do projeto ao longo das *sprints* definidas na metodologia. Foram apresentados os módulos essenciais que compõem a solução proposta — autenticação e gerenciamento de usuários, rubricas, projetos, ofícios e seus respectivos aditivos — bem como os modelos conceituais, interfaces e trechos de código que sustentam o funcionamento de cada componente.

A construção incremental permitiu validar, a cada ciclo, funcionalidades diretamente relacionadas às necessidades da fundação de apoio institucional, contemplando desde o controle de perfis de acesso até o complexo fluxo de remanejamento orçamentário. Os diagramas de classes apresentados ao longo do capítulo demonstram como a modelagem orientada a objetos foi fundamental para garantir organização ao sistema.

6 CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

O presente trabalho teve como objetivo principal solucionar a ineficiência, a vulnerabilidade a erros e a falta de segurança no processo de gerenciamento de remanejamentos em projetos administrados por fundações de apoio institucional, apresentando-se como uma alternativa diante da carência de ferramentas voltadas a essa finalidade. A proposta surgiu a partir da constatação de que essas instituições ainda dependem de planilhas eletrônicas para o controle dos remanejamentos, o que acarreta os riscos mencionados.

Durante o desenvolvimento do projeto, foram aplicadas tecnologias robustas e consolidadas, como PHP, JavaScript, Laravel, PostgreSQL e Bootstrap, integradas por meio da arquitetura MVC, o que garantiu modularidade e facilidade de manutenção do código-fonte. Além disso, a adoção da metodologia ágil Scrum foi essencial para organizar as etapas de desenvolvimento em ciclos iterativos (*Sprints*), promovendo entregas incrementais e permitindo constantes ajustes com base no *feedback* dos colaboradores da fundação estudada.

Como resultado, a ferramenta desenvolvida oferece uma alternativa para a gestão de remanejamentos em fundações de apoio, centralizando os dados em um único ambiente — o que antes era realizado por meio de diversas planilhas — e automatizando processos anteriormente executados de forma manual, como somas de saldos e validações de orçamentos. Com a ferramenta, espera-se obter benefícios como a redução de erros humanos, a melhoria da integridade dos dados e o aumento da produtividade nas rotinas de cadastro, acompanhamento e geração de relatórios.

Apesar de sua aplicabilidade ter sido testada desde o seu surgimento, o trabalho apresenta algumas limitações. O sistema foi avaliado em apenas uma fundação de apoio, o que pode restringir a abrangência de sua aplicabilidade. Ainda assim, a versão inicial e funcional desenvolvida cumpre os objetivos propostos e demonstra a viabilidade da solução.

Como trabalhos futuros, sugere-se:

- Efetuar o teste de aplicabilidade em mais fundações de apoio;
- A integração com sistemas contábeis e financeiros utilizados pelas fundações;
- A incorporação de ferramentas analíticas para geração de *dashboards* e indicadores de gestão;
- A implementação de um módulo de notificações automáticas para alertas de prazos;
- Melhoria contínua dos fluxos das funcionalidades do sistema;

- Atualização constante das tecnologias utilizadas para as versões mais recentes;
- Evolução do código-fonte do sistema tanto do *back-end* quanto do *front-end*, sempre aplicando as boas práticas de programação;
- Aprimorar a documentação do sistema.

Conclui-se, portanto, que o sistema proposto representa um incremento em uma das diversas áreas da gestão de projetos em fundações de apoio institucional, ao oferecer uma ferramenta desenvolvida para o controle de remanejamentos. Além de atender às necessidades operacionais da instituição, o sistema contribui para a modernização e digitalização dos processos administrativos.

Dessa forma, o presente trabalho abre espaço para futuras pesquisas e aprimoramentos, consolidando-se como uma contribuição para a área de gestão de projetos e desenvolvimento de sistemas web aplicados à problemas contemporâneos.

REFERÊNCIAS

- AUDY, Jorge. **Scrum 360: Um guia completo e prático de agilidade**. [S.l.]: Editora Casa do Código, 2015.
- BAKKEN, Stig Sæther; SURASKI, Zeev; SCHMID, Egon. **PHP Manual: Volume 2**. [S.l.]: iUniverse, Incorporated, 2000.
- BEIGHLEY, Lynn. **Use a cabeça! SQL**. [S.l.]: Alta Books, 2009.
- BRASIL. **Lei nº 8.958, de 20 de dezembro de 1994**. Brasília, DF: [s.n.], 1994. *Diário Oficial da União*, seção 1. Dispõe sobre a relação entre as instituições federais de ensino superior e de pesquisa científica e tecnológica e as fundações de apoio.
- _____. **Lei nº 9.609, de 19 de fevereiro de 1998**. Brasília, DF: [s.n.], 1998. Dispõe sobre a proteção da propriedade intelectual de programa de computador, sua comercialização no País, e dá outras providências.
- CHACON, Scott; STRAUB, Ben. **Pro git**. [S.l.]: Springer Nature, 2014.
- CONVENIA. **Como convencer seu chefe a abandonar a planilha de Excel?** 2023. Disponível em: <<https://blog.convenia.com.br/como-convencer-seu-chefe-a-abandonar-a-planilha-de-excel-para-controlar-o-rh.>>. Acesso em 25 Janeiro 2025.
- CRUZ, Fábio. **Scrum e PMBOK unidos no Gerenciamento de Projetos**. [S.l.]: Brasport, 2013.
- CULTURA, Fundação Norte-Rio-Grandense de Pesquisa e. **Caderno de orientações para coordenador de curso de especialização**. [S.l.]: FUNPEC – Fundação Norte-Rio-Grandense de Pesquisa e Cultura, 2019.
- DAL'OGGIO, Pablo. **PHP Programando com Orientação a Objetos-4ª Edição: Inclui Design Patterns**. [S.l.]: Novatec Editora, 2018.
- DEBASTIANI, Carlos Alberto. **Definindo escopo em projetos de software**. [S.l.]: Novatec Editora, 2016.
- FARANELLO, Scott. **Balsamiq wireframes quickstart guide**. [S.l.]: Packt Publishing Ltd, 2012.
- FARINELLI, Fernanda. Conceitos básicos de programação orientada a objetos. **Instituto Federal Sudeste de Minas Gerais**, 2007.
- FERREIRA, Brenda. **Framework Laravel: Um Estudo de Caso Full Stack Development**. [S.l.]: Editora Universidade Federal de Ouro Preto, 2021.
- FIELDING, Roy T.; KAISER, Gail. The apache http server project. **IEEE Internet Computing**, IEEE, v. 1, n. 4, p. 88–90, 2002.
- FILHO, João Eriberto Mota. **Descobrimdo o Linux-3ª Edição: Entenda o sistema operacional GNU/Linux**. [S.l.]: Novatec Editora, 2012.

- GABARDO, Ademir C. **Laravel para ninjas**. [S.l.]: Novatec Editora, 2017.
- HAGEN, William Von. **Ubuntu Linux Bible**. [S.l.]: John Wiley & Sons, 2007. v. 352.
- HICKSON, Anthony. **HTML, The living standard**. [S.l.]: Lulu. com, 2014.
- JÚNIOR, Altino Alves et al. Entendendo o engajamento das comunidades front-end e back-end nos repositórios do github. In: SBC. **Workshop de Visualização, Evolução e Manutenção de Software (VEM)**. [S.l.], 2022. p. 26–30.
- KABIR, Mohammed J. **Apache Server 2 Bible**. [S.l.]: Wiley Publishing, 2002.
- KRAUSE, Walther. **ISO 21500-Orientações sobre Gerenciamento de Projetos: Diretrizes para o Sucesso**. [S.l.]: Brasport, 2014.
- LARAVEL. **Laravel: The PHP Framework for Web Artisans**. 2025. Disponível em: <<https://github.com/laravel/laravel>>. Acesso em 18 Maio 2025.
- MILANI, André. **Postgresql-guia do programador**. [S.l.]: Novatec Editora, 2008.
- MINDTEK. **O risco de Excel**. 2021. Disponível em: <<https://www.mindtek.com.br/2021/10/o-risco-de-excel/>>. Acesso em 25 Janeiro 2025.
- MORETO, Silvio. **Bootstrap 4 By Example**. [S.l.]: Packt Publishing Ltd, 2016.
- NORMANDO, Célio. **Arquitetura MVC e Princípios de Projeto**. 2021. Acesso em 14 de maio de 2025. Disponível em: <<https://medium.com/@celionormando/arquitetura-mvc-e-princ%C3%ADpios-de-projeto-3d0b278ef910>>.
- O'REGAN, Gerard. Xerox parc. In: **Pillars of Computing: A Compendium of Select, Pivotal Technology Firms**. [S.l.]: Springer, 2015. p. 225–229.
- PANKO, Raymond R. What we know about spreadsheet errors. **Journal of End User Computing**, 2005.
- PASSAMA, Robin. **Introduction to Git & GitLab**. Montpellier: LIRMM, 2016.
- PELLEGRINI, Jerônimo. **Criptografia Compacto**. [S.l.]: UFPR, 2025.
- PEREIRA, Geordy Souza et al. **Identificação e análise de variáveis de desempenho e sucesso que mais influenciam no gerenciamento de projetos nas fundações de apoio**. [S.l.]: Revista de Gestão de Projetos, 2022.
- PEREIRA, Paulo; TORREÃO, Paula; MARÇAL, Ana Sofia. Entendendo scrum para gerenciar projetos de forma ágil. **Mundo PM**, v. 1, n. 14, p. 64–71, 2007.
- PMI. **Um Guia do Conhecimento em Gerenciamento de Projetos (Guia PMBOK®)**. 5. ed. Newtown Square, PA: Project Management Institute, 2013. Tradução da 5ª edição norte-americana.
- PRODANOV, Cleber Cristiano; FREITAS, Ernani Cesar De. **Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico-2ª Edição**. [S.l.]: Editora Feevale, 2013.
- PROPLAN. **Pró-Reitoria de Planejamento - PROPLAN**. 2025. Disponível em: <<https://proplan.ufrn.br/>>. Acesso em 25 Janeiro 2025.

REENSKAUG, Trygve. **The original MVC reports**. 2006. Acesso em 15 maio 2025. Disponível em: <<https://folk.universitetetioslo.no/trygver/themes/mvc/mvc-index.html>>.

ROCHA, José Gladistone. Arquitetura em camadas com uso do paradigma mvc e processo unificado na programação de software orientado a objetos. **Tecnologias em Projeção**, v. 9, n. 1, p. 31–49, 2018.

SCHWABER, Ken; SUTHERLAND, Jeff. The scrum guide. **Scrum Org, November**, 2020.

SCRUM.ORG. **What is Scrum?** 2025. Disponível em: <<https://www.scrum.org/>>. Acesso em 04 Fevereiro 2025.

SELLTIZ, Claire et al. Métodos de pesquisa nas relações sociais. In: **Métodos de pesquisa nas relações sociais**. [S.l.]: São Paulo: EPU, 1975. p. 690–690.

SENADO. **Dotação orçamentária**. 2025. Disponível em: <<https://www12.senado.leg.br/manualdecomunicacao/guia-de-economia/dotacao-orcamentaria>>. Acesso em 25 Janeiro 2025.

SIGO, Sistemas. **Por que abandonar as planilhas Excel?** 2023. Disponível em: <<https://www.sigoweb.com.br/site/blog/categoria/porque-abandonar-as-planilhas-excel/>>. Acesso em 25 Janeiro 2025.

SILVA, Erikson Dutra de Miranda. Desenvolvimento de uma ferramenta de gestão para construção civil utilizando o framework laravel mvc. **Editora Universidade Federal de Goiás**, 2019.

SILVA, Maurício Samy. **JavaScript-Guia do Programador: Guia completo das funcionalidades de linguagem JavaScript**. [S.l.]: Novatec Editora, 2010.

_____. **jQuery-A Biblioteca do Programador JavaScript-3ª Edição: Aprenda a criar efeitos de alto impacto em seu site com a biblioteca JavaScript mais utilizada pelos desenvolvedores web**. [S.l.]: Novatec Editora, 2013.

SOMMERVILLE, Ian. **Software Engineering, 9/E**. [S.l.]: Pearson Education India, 2011.

STACKOVERFLOW. **2024 Developer Survey**. 2024. Disponível em: <<https://survey.stackoverflow.co/2024/>>. Acesso em 29 Janeiro 2025.

THE PHP FOUNDATION. **A Nova Vida do PHP – A Fundação PHP**. 2025. Disponível em: <<https://thephp.foundation/>>. Acesso em 26 Janeiro 2025.

TRENDS, Google. **Google Trends**. 2025. Disponível em: <<https://trends.google.com.br/>>. Acesso em 03 Fevereiro 2025.

TURINI, Rodrigo. **PHP e Laravel: crie aplicações web como um verdadeiro artesão**. [S.l.]: Editora Casa do Código, 2015.

VERGARA, Sylvia Constant. Projetos e relatórios de pesquisa. **São Paulo: Atlas**, v. 34, p. 38, 2006.

W3TECHS. **W3Techs - World Wide Web Technology Surveys**. 2025. Disponível em: <<https://w3techs.com/>>. Acesso em 27 Janeiro 2025.

WATRALL, Ethan. **Use a cabeça! Web design.** [S.l.]: Alta Books, 2009.